

KI-unterstützte Erstellung von ETL-Prozessen - Eine experimentelle Evaluierung

Bachelorarbeit

zur Erlangung des Grades

Bachelor of Science

der Fachrichtung Informatik

an der Dualen Hochschule Baden-Württemberg (DHBW) Stuttgart

von

Luis Geiger

02.05.2026

Matrikelnummer:

9144849

Kurs:

TINF23CE

Gutachter an der DHBW Stuttgart: Prof. Dr. Name

Erklärung

Ich erkläre mich hiermit ehrenwörtlich, dass ich die vorliegende Arbeit Bachelorarbeit mit dem Thema „*KI-unterstützte Erstellung von ETL-Prozessen - Eine experimentelle Evaluierung*“ selbständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Aus den benutzten Quellen, direkt oder indirekt, übernommene Gedanken habe ich als solche kenntlich gemacht. Diese Arbeit wurde bisher in gleicher oder ähnlicher Form oder auszugsweise noch keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Stuttgart, 02.05.2026

Ort, Datum

L.Geiger

Luis Geiger

Inhaltsverzeichnis

Abbildungsverzeichnis	V
Tabellenverzeichnis	V
Skriptverzeichnis	V
Abkürzungsverzeichnis	VI
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Zielsetzung und Forschungsfragen	2
1.3 Methodisches Vorgehen	3
1.4 Aufbau der Arbeit	4
2 Grundlagen	4
2.1 ETL-Prozesse	5
2.1.1 Definition und Phasen	5
2.1.2 Typische Herausforderungen	6
2.1.3 Klassische Werkzeuge und Verfahren	6
2.2 Datenqualität und Datenbereinigung	7
2.2.1 Dimensionen der Datenqualität	7
2.2.2 Häufige Datenprobleme	8
2.3 Künstliche Intelligenz und maschinelles Lernen	8
2.4 Large Language Models	9
2.4.1 Funktionsweise und Architektur	9
2.4.2 Überblick aktueller Modelle	10
2.4.3 Stärken und Limitationen bei strukturierten Daten	10
2.5 Prompt Engineering	11
2.5.1 Grundlegende Prompting-Strategien	11
2.5.2 Einflussfaktoren auf die Ergebnisqualität	12
3 Stand der Forschung	12
3.1 KI-Einsatz in der Datenverarbeitung	13
3.2 LLMs für strukturierte Daten und Tabellen	13
3.3 Verwandte Arbeiten zur Automatisierung von ETL	14
3.4 Forschungslücke und Abgrenzung	15

4	Konzeption der experimentellen Evaluierung	16
4.1	Forschungsdesign und Hypothesen	16
4.2	Auswahl der Large Language Models	20
4.3	Definition der ETL-Aufgaben	20
4.3.1	Datenbereinigung	21
4.3.2	Datentransformation	21
4.3.3	Duplikaterkennung	22
4.4	Prompting-Strategien	22
4.5	Evaluationskriterien	23
4.5.1	Genauigkeit	23
4.5.2	Vollständigkeit	24
4.5.3	Konsistenz	24
4.5.4	Effizienz	24
4.6	Vergleich gegen eine manuelle ETL-Pipeline	25
5	Erstellung des synthetischen Datensatzes	26
5.1	Anforderungen an den Datensatz	26
5.2	Datenmodell und -struktur	27
5.3	Eingebaute Datenqualitätsprobleme	28
5.4	Generierungsverfahren und verwendete Werkzeuge	32
5.5	Validierung des Datensatzes	32
6	Implementierung	33
6.1	Systemarchitektur und Versuchsumgebung	34
6.2	Eingesetzte Technologien und Bibliotheken	35
6.3	Umsetzung der klassischen ETL-Pipeline	35
6.4	Integration der LLMs in die Pipeline	36
6.5	Umsetzung der Prompting-Strategien	38
6.6	Mess- und Auswertungsverfahren	39
7	Durchführung der Experimente	40
7.1	Versuchsablauf	40
7.2	Erhobene Messdaten	41
7.3	Reproduzierbarkeit und Versuchsbedingungen	42
8	Ergebnisse und Auswertung	43
8.1	Ergebnisse je ETL-Aufgabe	44
8.1.1	Datenbereinigung	46
8.1.2	Duplikaterkennung	48
8.1.3	Datentransformation	49

8.2	Vergleich der Modelle	50
8.3	Vergleich der Prompting-Strategien	52
8.4	Ergebnisse der verketteten Pipeline	54
8.4.1	Qualitätsverlauf über die Kette	55
8.4.2	Trennung von eigener Leistung und Fehlerfortpflanzung	56
8.5	Reproduzierbarkeit der Ergebnisse	58
8.6	Bestätigungslauf auf einem unbekannten Datensatz	59
8.6.1	Verkettete und isolierte Ausführung	60
8.6.2	Reproduzierbarkeit über die Wiederholungen	62
8.6.3	Kosten, Token-Verbrauch und Laufzeit	63
8.6.4	Prompting-Strategien im Bestätigungslauf	65
8.6.5	Prompting-Strategien ohne Fehlerfortpflanzung	67
8.7	Vergleich KI-basiert vs. klassisch	69
8.8	Diskussion der Ergebnisse	71
8.8.1	Stärken des LLM-Einsatzes	71
8.8.2	Schwächen und Risiken	72
8.8.3	Einordnung in den Stand der Forschung	73
9	Handlungsempfehlungen für die Praxis	75
9.1	Eignung von LLMs für unterschiedliche ETL-Szenarien	75
9.2	Empfehlungen zur Modell- und Prompt-Auswahl	77
9.3	Hybride Architekturen	79
10	Fazit und Ausblick	81
10.1	Zusammenfassung der Ergebnisse	81
10.2	Beantwortung der Forschungsfragen	83
10.3	Limitationen der Arbeit	85
10.4	Ausblick auf weitere Forschung	87
	Literatur	90

Abbildungsverzeichnis

8.1	Gesamtüberblick der Genauigkeit	45
8.2	Duplikaterkennung je Strategie	48
8.3	Modellvergleich über alle Läufe	50
8.4	Vergleich der Prompting-Strategien	53
8.5	Qualitätsverlauf der verketteten Pipeline	55
8.6	Verkettete Pipeline gegen unabhängige Einzelläufe	56
8.7	Fehlerfortpflanzung in der Pipeline	57
8.8	Reproduzierbarkeit der Ergebnisse	58
8.9	Bestätigungslauf: Genauigkeit je Pipeline-Schritt	60
8.10	Reproduzierbarkeit im Bestätigungslauf	62
8.11	Aufwand der beiden Ausführungsmodi	64
8.12	Prompting-Strategien im Bestätigungslauf	66
8.13	Prompting-Strategien mit und ohne Fehlerfortpflanzung	68
8.14	Ansätze im Vergleich je Kategorie	69

Tabellenverzeichnis

5.1	Beispiele verfälschter Werte im Datensatz	30
5.2	Beispiele eingefügter Duplikate	31
8.1	Ergebnisse je ETL-Aufgabe	46
9.1	Eignungsmatrix nach Aufgabentyp	77

Skriptverzeichnis

8.1	Zwei lauffähige Lösungen derselben Bereinigungsaufgabe: Die obere Variante bleibt in der eingesetzten Pandas-Version wirkungslos, weil Textspalten nicht mehr als Objektspalten geführt werden; die untere benennt die Spalten explizit.	47
-----	--	----

Abkürzungsverzeichnis

ETL Extract, Transform, Load

LLM Large Language Model

KI Künstliche Intelligenz

ML maschinelles Lernen

1. Einleitung

1.1 Motivation und Problemstellung

Die Verfügbarkeit, Qualität und zeitnahe Bereitstellung von Daten sind zu einem zentralen Erfolgsfaktor für datengetriebene Organisationen geworden. Im Zentrum der dafür notwendigen Dateninfrastruktur stehen seit Jahrzehnten Extract, Transform, Load (ETL)-Prozesse, die Daten aus heterogenen Quellsystemen extrahieren, in eine einheitliche Struktur überführen und in Zielsysteme wie Data Warehouses oder Analyseplattformen laden [1, 2]. Trotz ihrer langen Etablierung gelten klassische ETL-Pipelines zunehmend als Engpass: Sie sind in der Regel regelbasiert, statisch modelliert und erfordern erheblichen manuellen Aufwand bei Entwurf, Anpassung und Wartung [3, 4]. Mit dem Zuwachs von Datenvolumen, der Heterogenität der Quellen und steigenden Anforderungen an Aktualität geraten herkömmliche ETL-Architekturen an ihre Grenzen [2, 5]. Parallel dazu haben sich in den letzten Jahren Large Language Models (LLMs) als leistungsfähige Werkzeuge etabliert, die neben natürlicher Sprache auch strukturierte und semi-strukturierte Daten verarbeiten können [6]. Arbeiten zeigen, dass LLMs im Kontext der Datenverarbeitung Aufgaben wie Datenbereinigung, Schema-Mapping, Transformationslogik oder Anomalieerkennung übernehmen oder zumindest unterstützen können [7–9]. Damit eröffnet sich die Perspektive, klassische ETL-Prozesse durch Künstliche Intelligenz (KI)-gestützte Komponenten zu ergänzen oder partiell zu ersetzen, um manuellen Aufwand zu reduzieren und die Anpassungsfähigkeit der Pipelines zu erhöhen. Der praktische Anreiz liegt dabei weniger in der technischen Machbarkeit an sich als vielmehr in der Aussicht auf Effizienz- und Kostenvorteile: Übernehmen Modelle Routineschritte der Transformationslogik ganz oder unterstützend, lässt sich der zeitintensive und damit kostenträchtige manuelle Entwicklungs- und Wartungsaufwand spürbar verringern, was den Einsatz gerade im wirtschaftlichen Kontext motiviert. Gleichzeitig ist die praktische Eignung von LLMs für ETL-Aufgaben bislang nur unzureichend systematisch untersucht. Während konzeptionelle Beiträge die potenziellen Vorteile beschreiben, fehlt es an empirischen Vergleichen, die unterschiedliche Modelle und Prompting-Strategien unter kontrollierten Bedingungen gegen klassische Verfahren stellen. Offen ist insbesondere, wie zuverlässig LLMs typische ETL-Teilaufgaben wie Datenbereinigung, Datentransformation und Duplikaterkennung bearbeiten, wie stabil und reproduzierbar ihre Ergebnisse aufgrund der probabilistischen Modellarchitektur ausfallen, wie sich verschiedene Prompting-Ansätze auf die Ergebnisqualität auswirken und unter welchen Bedingungen ein Einsatz gegenüber etablierten regel-

basierten Methoden sinnvoll ist. Offen bleibt zudem, wie sich ein solcher Einsatz zum heutigen De-facto-Standard der Softwareentwicklung verhält, bei dem KI nicht autonom, sondern als Assistenz einer menschlichen Fachkraft wirkt: Inwieweit die Modelle den manuellen Aufwand tatsächlich verringern, statt ihn lediglich zu verlagern, ist bislang kaum systematisch beleuchtet. Genau an diesem Forschungsdefizit setzt die vorliegende Arbeit an.

1.2 Zielsetzung und Forschungsfragen

Ziel dieser Arbeit ist eine experimentelle Evaluierung, inwieweit sich aktuelle LLMs für ausgewählte ETL-Aufgaben eignen – sei es als eigenständige Automatisierung oder als unterstützendes Werkzeug einer menschlichen Fachkraft. Im Zentrum steht der systematische Vergleich mehrerer aktueller LLMs und unterschiedlicher Prompting-Strategien, bewertet anhand von Genauigkeit, Vollständigkeit, Konsistenz und Effizienz bei der Lösung der definierten ETL-Teilaufgaben und ausdrücklich nicht als allgemeiner Leistungsvergleich der Modelle. Grundlage ist ein kontrolliert erstellter synthetischer Datensatz, der typische Datenqualitätsprobleme abbildet. Die Ergebnisse werden mit einer klassisch implementierten ETL-Pipeline verglichen, um Stärken, Schwächen und praktische Einsatzgrenzen KI-gestützter Ansätze fundiert einordnen zu können. Auf dieser Grundlage sollen Handlungsempfehlungen für den Einsatz von LLMs in realen ETL-Szenarien abgeleitet werden. Aus dieser Zielsetzung ergeben sich zwei leitende Forschungsfragen:

FF1: Inwieweit eignen sich aktuelle LLMs für die Bearbeitung typischer ETL-Teilaufgaben – Datenbereinigung, Datentransformation und Duplikaterkennung – im Vergleich zu einer klassischen, regelbasierten Pipeline, und unter welchen Bedingungen ist je nach Aufgabentyp ein klassischer, KI-gestützter oder hybrider Ansatz zu empfehlen?

FF2: Wie stark beeinflusst die Wahl der Prompting-Strategie die Ergebnisqualität der untersuchten LLMs, und welche Strategien sind für die betrachteten ETL-Aufgaben zu empfehlen?

Da LLMs probabilistische Ausgaben erzeugen, wird die Reproduzierbarkeit ihrer Ergebnisse nicht als eigenständige Forschungsfrage, sondern als querschnittliche Untersuchungsdimension über beide Fragen hinweg behandelt: Jede Konfiguration wird mehrfach ausgeführt, sodass die Stabilität der Ergebnisse durchgängig mitbewertet

werden kann.

1.3 Methodisches Vorgehen

Zur Beantwortung der formulierten Forschungsfragen wird ein mehrstufiges, experimentell ausgerichtetes Vorgehen gewählt, das konzeptionelle, gestalterische und empirische Elemente miteinander verbindet. Den Ausgangspunkt bildet eine strukturierte Aufarbeitung der theoretischen Grundlagen zu ETL-Prozessen, Datenqualität, LLMs und Prompt Engineering sowie eine Analyse des aktuellen Forschungsstands zum Einsatz von KI in der Datenintegration. Diese literaturbasierte Vorarbeit dient dazu, relevante Konzepte, etablierte Verfahren und bestehende Forschungslücken zu identifizieren und die anschließende Evaluierung theoretisch zu fundieren. Auf dieser Grundlage wird in einem zweiten Schritt das experimentelle Design entwickelt. Dazu zählen die Auswahl mehrerer aktueller LLMs, die Definition der zu untersuchenden ETL-Teilaufgaben Datenbereinigung, Datentransformation und Duplikaterkennung sowie die Festlegung unterschiedlicher Prompting-Strategien. Ergänzend werden Evaluationskriterien operationalisiert, anhand derer die Ergebnisse aller Verfahren vergleichbar gemessen werden können. Da LLMs aufgrund ihrer probabilistischen Natur nicht-deterministische Ausgaben erzeugen können, wird zusätzlich die Reproduzierbarkeit der Ergebnisse als eigenständige Untersuchungsdimension berücksichtigt. Als Vergleichsmaßstab dient eine klassische, regelbasiert implementierte ETL-Pipeline. Im dritten Schritt wird ein synthetischer Datensatz erstellt, der typische Datenqualitätsprobleme realer Datenbestände kontrolliert abbildet. Die gezielte Konstruktion erlaubt es, die Eigenschaften der Eingangsdaten exakt zu kennen und damit reproduzierbare sowie aussagekräftige Vergleiche zwischen den untersuchten Ansätzen zu ermöglichen. Anschließend erfolgt die technische Umsetzung der Versuchsumgebung, in der sowohl die klassische ETL-Pipeline als auch die LLM-basierten Verarbeitungsschritte implementiert und über einheitliche Schnittstellen ausgeführt werden. Die eigentliche Durchführung der Experimente erfolgt unter standardisierten Bedingungen, wobei für jede Kombination aus Modell, Prompting-Strategie und ETL-Aufgabe Messdaten zu Qualität, Stabilität und Effizienz erhoben werden. Um die Reproduzierbarkeit der LLM-Ergebnisse beurteilen zu können, wird jede Konfiguration mehrfach ausgeführt. Die Auswertung kombiniert eine quantitative Analyse der definierten Kennzahlen mit einer qualitativen Einordnung beobachteter Stärken und Schwächen. Aus den Ergebnissen werden abschließend Handlungsempfehlungen abgeleitet, die eine fundierte Einschätzung über den sinnvollen Einsatz von LLMs in unterschiedlichen ETL-Szenarien ermöglichen.

1.4 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in zehn Kapitel. Nach der vorliegenden Einleitung, in der Motivation, Zielsetzung, Forschungsfragen und methodisches Vorgehen dargelegt werden, behandelt Kapitel 2 die theoretischen Grundlagen. Hier werden zentrale Konzepte zu ETL-Prozessen, Datenqualität, LLMs sowie Prompt Engineering eingeführt, die als Bezugsrahmen für die weitere Untersuchung dienen. Kapitel 3 widmet sich dem aktuellen Stand der Forschung. Es werden Arbeiten zum Einsatz von KI in der Datenverarbeitung, zur Verarbeitung strukturierter Daten durch LLMs sowie zur Automatisierung von ETL-Prozessen aufgearbeitet und gegeneinander abgegrenzt. Auf dieser Basis wird die für diese Arbeit relevante Forschungslücke präzisiert. In Kapitel 4 wird die Konzeption der experimentellen Evaluierung beschrieben. Dies umfasst das Forschungsdesign, die Auswahl der untersuchten Modelle, die Definition der ETL-Aufgaben, die eingesetzten Prompting-Strategien sowie die Evaluationskriterien einschließlich der manuellen Pipeline. Kapitel 5 dokumentiert anschließend die Erstellung des synthetischen Datensatzes, einschließlich Anforderungen, Datenmodell, eingebauter Qualitätsprobleme und der durchgeführten Validierung. Die technische Umsetzung wird in Kapitel 6 dargestellt. Es beschreibt die Systemarchitektur, die eingesetzten Technologien sowie die konkrete Implementierung der klassischen ETL-Pipeline und der LLM-basierten Komponenten. Kapitel 7 behandelt die Durchführung der Experimente und legt Versuchsablauf, erhobene Messdaten sowie die Bedingungen zur Sicherstellung der Reproduzierbarkeit offen. Kapitel 8 präsentiert und analysiert die Ergebnisse. Neben einer aufgabenspezifischen Auswertung erfolgen ein Vergleich der Modelle, der Prompting-Strategien sowie eine Gegenüberstellung von KI-basiertem und klassischem Ansatz. Die Ergebnisse werden anschließend diskutiert und in den Stand der Forschung eingeordnet. Aus den gewonnenen Erkenntnissen leitet Kapitel 9 Handlungsempfehlungen für die Praxis ab, insbesondere zur Eignung von LLMs in unterschiedlichen ETL-Szenarien sowie zur Gestaltung hybrider Architekturen. Kapitel 10 fasst die zentralen Ergebnisse zusammen, beantwortet die in Abschnitt 1.2 formulierten Forschungsfragen, reflektiert die Limitationen der Arbeit und gibt einen Ausblick auf weiterführende Forschungsfragen.

2. Grundlagen

Dieses Kapitel legt die theoretischen Grundlagen, auf denen die experimentelle Evaluierung der folgenden Kapitel aufbaut. Zunächst werden ETL-Prozesse als etablier-

tes Verfahren der Datenintegration beschrieben (Abschnitt 2.1). Anschließend wird das Konzept der Datenqualität als zentraler Bezugspunkt für die untersuchten ETL-Teilaufgaben eingeführt (Abschnitt 2.2). Darauf folgt eine Einordnung von Künstlicher Intelligenz und maschinellem Lernen als übergeordnetem Rahmen (Abschnitt 2.3), bevor Large Language Models als die in dieser Arbeit eingesetzte verarbeitende Komponente betrachtet werden (Abschnitt 2.4). Den Abschluss bildet eine Darstellung des Prompt Engineering als zentrale Steuerungsmethode für diese Modelle (Abschnitt 2.5).

2.1 ETL-Prozesse

2.1.1 Definition und Phasen

Der Begriff ETL steht für die drei aufeinanderfolgenden Phasen *Extract*, *Transform* und *Load* und bezeichnet einen der zentralen Prozesse der Datenintegration, mit dem Daten aus unterschiedlichen Quellsystemen in ein einheitliches Zielsystem überführt werden [1, 2]. Historisch ist der ETL-Prozess eng mit dem Aufbau von Data Warehouses verbunden, in denen konsolidierte und bereinigte Daten als Grundlage für Auswertungen und Entscheidungsunterstützung bereitgestellt werden [1, 10]. Der Prozess bildet damit das Bindeglied zwischen unverarbeiteten Rohdaten und den daraus abgeleiteten, nutzbaren Informationen [1, 2].

In der ersten Phase, der Extraktion, werden die relevanten Daten aus einer oder mehreren Quellen ausgelesen. Diese Quellen sind in der Praxis heterogen und reichen von relationalen Datenbanken über Dateien und Tabellen bis hin zu Schnittstellen externer Dienste, sodass bereits hier unterschiedliche Formate und Strukturen zusammengeführt werden müssen [1, 2]. Die Transformationsphase bildet den inhaltlichen Kern des Prozesses: Die extrahierten Daten werden bereinigt, vereinheitlicht und in die vom Zielsystem geforderte Struktur überführt. Typische Operationen sind die Korrektur fehlerhafter oder fehlender Werte, die Vereinheitlichung von Formaten, die Zusammenführung und Aggregation von Datensätzen, das Auflösen von Duplikaten sowie das Abbilden der Quellstrukturen auf das Zielschema [1, 2, 5]. In der abschließenden Ladephase werden die transformierten Daten schließlich in das Zielsystem geschrieben, etwa in ein Data Warehouse oder eine Analyseplattform, wobei zwischen vollständigem Laden und inkrementellem Aktualisieren unterschieden wird [1, 10].

Die strikte Abfolge der drei Phasen ist dabei nicht das einzig mögliche Vorgestaltungsmuster. Insbesondere im Kontext von Big Data hat sich mit ELT (Extract, Load,

Transform) eine Variante etabliert, bei der die Rohdaten zunächst geladen und erst innerhalb des Zielsystems transformiert werden, um von dessen Verarbeitungskapazitäten zu profitieren [2, 5]. Für die vorliegende Arbeit bleibt jedoch die klassische ETL-Reihenfolge maßgeblich, da der Schwerpunkt auf den Transformationsschritten der Datenbereinigung, Datentransformation und Duplikaterkennung liegt.

2.1.2 Typische Herausforderungen

Obwohl ETL-Prozesse seit Jahrzehnten etabliert sind, gelten sie in modernen Datenlandschaften zunehmend als anspruchsvoll und wartungsintensiv. Diese Einschätzung gründet auf mehreren miteinander verbundenen Herausforderungen. Eine zentrale davon ist die Sicherung der Datenqualität: Da die Eingangsdaten aus unterschiedlichen Quellen stammen, weisen sie häufig Fehler, Lücken und Inkonsistenzen auf, die im Transformationsschritt erkannt und behoben werden müssen, um verlässliche Analyseergebnisse zu gewährleisten [1, 11]. Eng damit verbunden ist die Heterogenität der Quellen, die sich in unterschiedlichen Formaten, Schemata und Bedeutungen gleichartiger Felder äußert und einen erheblichen Aufwand bei der Vereinheitlichung verursacht [1, 2].

Hinzu kommt die Frage der Skalierbarkeit. Mit dem stetigen Wachstum von Datenvolumen und -vielfalt geraten klassische, batch-orientierte ETL-Architekturen an ihre Grenzen und können den Anforderungen an Verarbeitungsgeschwindigkeit und Aktualität nur noch eingeschränkt gerecht werden [2, 5]. Ein weiterer kritischer Aspekt ist der hohe manuelle Aufwand bei Entwurf, Anpassung und Wartung: Klassische Pipelines sind in der Regel regelbasiert und statisch modelliert, sodass jede Änderung an den Quellstrukturen oder den fachlichen Anforderungen einen manuellen Eingriff erfordert [3, 4]. Diese geringe Anpassungsfähigkeit führt dazu, dass ein Großteil der Arbeitszeit von Datenfachleuten in die Erstellung und Pflege solcher Pipelines fließt, was sie zu einem häufig benannten Engpass datengetriebener Organisationen macht [3, 5]. Genau diese Schwachstellen motivieren die in dieser Arbeit untersuchte Frage, inwieweit lernende und sprachbasierte Verfahren den manuellen Aufwand reduzieren und die Flexibilität klassischer ETL-Prozesse erhöhen können.

2.1.3 Klassische Werkzeuge und Verfahren

Zur Umsetzung von ETL-Prozessen hat sich über die Jahre ein breites Spektrum an Werkzeugen und Verfahren herausgebildet. Auf der einen Seite stehen dedizierte ETL-

Plattformen mit grafischer Modellierung wie Talend, Pentaho oder Informatica, die den Prozess über vorkonfigurierte Komponenten abbilden und sich vor allem in betrieblichen Data-Warehouse-Umgebungen etabliert haben [1, 10]. Auf der anderen Seite werden ETL-Strecken zunehmend programmatisch umgesetzt, etwa mit der Python-Bibliothek Pandas für die Verarbeitung tabellarischer Daten oder mit verteilten Frameworks wie Apache Spark für große Datenmengen [1, 2].

Allen klassischen Ansätzen ist gemeinsam, dass die Verarbeitungslogik regelbasiert und explizit vorgegeben wird: Entwicklerinnen und Entwickler definieren die Transformations- und Bereinigungsregeln manuell, etwa in Form von Bedingungen, Zuordnungstabellen oder Skripten [3, 10]. Dieser Ansatz bietet den Vorteil hoher Nachvollziehbarkeit und deterministischer, reproduzierbarer Ergebnisse, stößt jedoch dort an Grenzen, wo Datenprobleme vielfältig, kontextabhängig oder schwer im Voraus zu formalisieren sind [3, 4]. In der vorliegenden Arbeit dient eine solche regelbasiert implementierte Pipeline als Vergleichsmaßstab, gegen den die untersuchten LLM-gestützten Verfahren gemessen werden.

2.2 Datenqualität und Datenbereinigung

2.2.1 Dimensionen der Datenqualität

Datenqualität ist ein mehrdimensionales Konstrukt, das beschreibt, inwieweit ein Datenbestand für seinen vorgesehenen Verwendungszweck geeignet ist. In der Literatur hat sich ein Kern wiederkehrender Qualitätsdimensionen herausgebildet, zu denen insbesondere Genauigkeit, Vollständigkeit, Konsistenz, Aktualität und Eindeutigkeit zählen [11, 12]. Die Genauigkeit beschreibt, inwieweit die gespeicherten Werte die reale Sachlage korrekt abbilden, während die Vollständigkeit den Grad angibt, in dem alle erforderlichen Werte tatsächlich vorhanden sind [11, 12]. Die Konsistenz bezieht sich auf die Widerspruchsfreiheit der Daten innerhalb eines Bestands und über Systemgrenzen hinweg, etwa bei einheitlichen Formaten und Bezeichnungen [11, 12]. Die Aktualität erfasst, ob die Daten den jeweils gültigen Zeitstand widerspiegeln, und die Eindeutigkeit verlangt, dass jeder reale Sachverhalt nur einmal und nicht in mehreren widersprüchlichen oder duplizierten Datensätzen abgelegt ist [11, 12].

Mit dem Aufkommen von Big Data ist dieser klassische Kanon erweitert worden, da Volumen, Geschwindigkeit und Vielfalt der Daten zusätzliche Anforderungen stellen und feinere Differenzierungen der Qualitätsdimensionen erforderlich machen [11, 12].

Für die vorliegende Arbeit sind insbesondere die Dimensionen Genauigkeit, Vollständigkeit, Konsistenz und Eindeutigkeit von Bedeutung, da sie unmittelbar mit den untersuchten ETL-Teilaufgaben der Datenbereinigung, Datentransformation und Duplikaterkennung korrespondieren.

2.2.2 Häufige Datenprobleme

Aus den Qualitätsdimensionen lassen sich die in der Praxis am häufigsten auftretenden Datenprobleme ableiten, die im Rahmen der Datenbereinigung erkannt und behoben werden müssen. Fehlende Werte gehören zu den verbreitetsten Mängeln und beeinträchtigen unmittelbar die Vollständigkeit; sie erfordern Entscheidungen über Imputation, Entfernung oder Kennzeichnung der betroffenen Datensätze [11, 13]. Inkonsistente Formate – etwa unterschiedliche Datums-, Zahlen- oder Einheitendarstellungen sowie abweichende Schreibweisen gleicher Inhalte – beeinträchtigen die Konsistenz und müssen im Transformationsschritt vereinheitlicht werden [11, 12].

Ein weiteres zentrales Problem sind Duplikate, also mehrfach vorhandene Datensätze, die denselben realen Sachverhalt beschreiben, sich aber in Schreibweise oder Vollständigkeit unterscheiden können. Ihre Erkennung ist anspruchsvoll, da sie über den exakten Abgleich hinaus häufig eine unscharfe Ähnlichkeitsbetrachtung erfordert [13, 14]. Hinzu kommen Ausreißer und fehlerhafte Werte, die durch Eingabefehler, Messfehler oder Systembrüche entstehen und die Genauigkeit des Datenbestands gefährden [11, 13]. Die Bedeutung einer gründlichen Bereinigung ergibt sich daraus, dass mangelhafte Eingangsdaten unmittelbar auf nachgelagerte Analysen und Modelle durchschlagen, was häufig mit dem Grundsatz „garbage in, garbage out“ umschrieben wird [11, 12]. Die gezielte Konstruktion solcher Datenprobleme bildet später die Grundlage des in Kapitel 5 beschriebenen synthetischen Datensatzes.

2.3 Künstliche Intelligenz und maschinelles Lernen

Die in dieser Arbeit untersuchten Sprachmodelle sind Teil eines übergeordneten Forschungs- und Anwendungsfeldes, das dieser Abschnitt einordnet, bevor die Modelle selbst betrachtet werden. Unter dem Begriff der KI wird das Bestreben verstanden, Maschinen mit Fähigkeiten auszustatten, die üblicherweise menschliche Intelligenz voraussetzen – etwa das Erkennen von Mustern, das Ziehen von Schlüssen oder das Treffen von Entscheidungen [15, 16]. KI ist damit ein weit gefasster Oberbegriff, der von regelbasierten Expertensystemen bis zu lernenden Verfahren reicht.

Den heute dominierenden Zweig der KI bildet maschinelles Lernen (ML). Anders als klassische, regelbasierte Systeme, deren Verhalten explizit durch Entwickelnde vorgegeben wird, leiten ML-Verfahren die zugrunde liegenden Zusammenhänge selbstständig aus Beispieldaten ab und übertragen sie auf neue, ungesehene Fälle [16, 17]. Genau dieser Unterschied – gelernte statt fest programmierter Verarbeitungslogik – ist für die vorliegende Arbeit zentral, da er den Kontrast zwischen den untersuchten LLMs und der klassischen, regelbasierten Vergleichspipeline begründet.

Im Kontext der Datenverarbeitung sind ML-Verfahren bereits etabliert und werden unter anderem zur Datenbereinigung, zur Imputation fehlender Werte, zum Abgleich von Entitäten und zur Anomalieerkennung eingesetzt [13, 17]. Eine besonders leistungsfähige Ausprägung des maschinellen Lernens ist das auf tiefen neuronalen Netzen beruhende *Deep Learning*, dessen Fortschritte – insbesondere die Transformer-Architektur – die Grundlage der im folgenden Abschnitt behandelten LLMs bilden [6, 18].

2.4 Large Language Models

2.4.1 Funktionsweise und Architektur

LLMs sind Sprachmodelle mit einer sehr großen Zahl an Parametern, die auf großen Mengen an Textdaten vortrainiert werden und in der Lage sind, natürliche Sprache zu verstehen und zu erzeugen [6, 18]. Technische Grundlage nahezu aller aktuellen Modelle ist die Transformer-Architektur, deren Selbstaufmerksamkeitsmechanismus es erlaubt, Abhängigkeiten zwischen weit auseinanderliegenden Textbestandteilen effizient zu erfassen [6, 18]. Eingaben werden dabei zunächst in kleinere Einheiten, sogenannte Token, zerlegt, die das Modell intern als numerische Vektoren verarbeitet; die Ausgabe entsteht, indem das Modell schrittweise das jeweils wahrscheinlichste nachfolgende Token vorhersagt [6].

Charakteristisch für LLMs ist der zweistufige Entstehungsprozess aus einem aufwendigen Vortraining auf großen, allgemeinen Datenmengen und einer anschließenden Anpassung, etwa durch Feinabstimmung oder durch ein Training mit menschlichem Feedback, das die Modelle an die Erwartungen der Nutzenden ausrichtet [6]. Ein zentraler Befund der Forschung ist, dass mit zunehmender Modell- und Datengröße neue Fähigkeiten auftreten, die kleinere Modelle nicht aufweisen. Dazu zählt insbesondere das In-Context-Lernen, also die Fähigkeit, eine Aufgabe allein anhand von Anweisungen und Beispielen im Eingabetext zu lösen, ohne dass die Modellparameter verändert werden

[6, 19]. Genau diese Eigenschaft bildet die Grundlage dafür, LLMs ohne aufgabenspezifisches Nachtraining für ETL-Teilaufgaben einzusetzen. Zugleich ist die Ausgabe der Modelle probabilistischer Natur: Da das nächste Token aus einer Wahrscheinlichkeitsverteilung gezogen wird, können identische Eingaben zu unterschiedlichen Ausgaben führen, was unmittelbare Folgen für die Reproduzierbarkeit der Ergebnisse hat [6].

2.4.2 Überblick aktueller Modelle

Seit der breiten öffentlichen Aufmerksamkeit für dialogfähige Modelle hat sich das Feld der LLMs rasant entwickelt und ausdifferenziert [6, 20]. Auf dem Markt konkurriert heute eine Reihe leistungsfähiger Modellfamilien, darunter die GPT-Modelle von OpenAI, die Claude-Modelle von Anthropic, die Gemini-Modelle von Google sowie offen verfügbare Modelle wie die Llama-Reihe [6, 20]. Diese Modelle unterscheiden sich nicht nur in ihrer Architektur und Parameterzahl, sondern auch in praxisrelevanten Eigenschaften wie der Größe des Kontextfensters, der Verarbeitungsgeschwindigkeit, den Nutzungskosten und der Verfügbarkeit als offenes oder geschlossenes Modell [6, 20].

Eine wesentliche Entwicklung der jüngeren Vergangenheit ist die Erweiterung hin zu multimodalen Modellen, die neben Text auch weitere Modalitäten wie Bilder verarbeiten können [6, 18]. Angesichts der hohen Innovationsgeschwindigkeit verschieben sich die Leistungsgrenzen zwischen den Modellen kontinuierlich, weshalb vergleichende Bewertungen anhand standardisierter Kennzahlen wie Qualität, Geschwindigkeit und Preis an Bedeutung gewonnen haben [20, 21]. Für die vorliegende Arbeit ist diese Heterogenität von unmittelbarer Relevanz, da sie die Auswahl mehrerer aktueller Modelle für den experimentellen Vergleich begründet und nahelegt, dass sich die Eignung für ETL-Aufgaben zwischen den Modellen unterscheiden kann.

2.4.3 Stärken und Limitationen bei strukturierten Daten

Im Kontext der Datenverarbeitung liegt die Stärke von LLMs in ihrer Flexibilität: Sie können semi- und unstrukturierte Inhalte interpretieren, kontextabhängige Zusammenhänge erfassen und über reine Mustererkennung hinaus auch implizites Wissen einbringen, ohne dass die zugrunde liegenden Regeln explizit vorgegeben werden müssen [6, 7]. Damit eignen sie sich potenziell für Aufgaben wie die Bereinigung uneinheitlicher Werte, die Vereinheitlichung von Formaten oder das Erkennen ähnlicher Datensätze, die mit starren Regeln nur schwer abzudecken sind [7].

Diesen Stärken stehen jedoch erhebliche Limitationen gegenüber. Eine zentrale Einschränkung ist die Neigung zu Halluzinationen, also zur Erzeugung plausibel wirkender, aber sachlich falscher oder frei erfundener Ausgaben, was bei der Verarbeitung strukturierter Daten zu schwer erkennbaren Fehlern führen kann [6, 7]. Hinzu kommt die Begrenzung durch das Kontextfenster, die der Menge an Daten, die in einem einzelnen Verarbeitungsschritt übergeben werden kann, eine technische Obergrenze setzt [6]. Ferner fallen die Nutzungskosten und die Verarbeitungszeit ins Gewicht, die insbesondere bei großen Datenmengen zu einem begrenzenden Faktor werden [6, 20]. Schließlich erschwert die bereits beschriebene probabilistische Natur der Modelle die Reproduzierbarkeit, da wiederholte Ausführungen identischer Aufgaben voneinander abweichende Ergebnisse liefern können [6, 7]. Diese Spannung zwischen Flexibilität und Verlässlichkeit bildet einen der zentralen Ausgangspunkte und zugleich einen der Untersuchungsgegenstände der vorliegenden Arbeit.

2.5 Prompt Engineering

2.5.1 Grundlegende Prompting-Strategien

Da LLMs ihre Aufgaben primär über das In-Context-Lernen lösen, kommt der Gestaltung der Eingabe – dem sogenannten Prompt – eine entscheidende Bedeutung zu. Unter Prompt Engineering versteht man das gezielte Entwerfen und Optimieren solcher Eingaben, um das Modellverhalten zu steuern, ohne die Modellparameter selbst zu verändern [22, 23]. Die einfachste Strategie ist das Zero-Shot-Prompting, bei dem das Modell eine Aufgabe allein anhand einer Anweisung und ohne weitere Beispiele bearbeitet [19, 22]. Beim Few-Shot-Prompting werden der Anweisung hingegen einige beispielhafte Ein- und Ausgabepaare beigelegt, anhand derer das Modell das gewünschte Verhalten ableitet; dieser Ansatz verbessert die Ergebnisqualität bei vielen Aufgaben spürbar [19, 22].

Darüber hinaus haben sich anspruchsvollere Strategien etabliert. Beim Chain-of-Thought-Prompting wird das Modell angeleitet, seine Lösung in nachvollziehbaren Zwischenschritten zu entwickeln, was sich insbesondere bei mehrstufigen oder schlussfolgernden Aufgaben als wirksam erwiesen hat [22, 23]. Eine weitere verbreitete Technik ist das Role-Prompting, bei dem dem Modell eine bestimmte Rolle oder Perspektive zugewiesen wird, um Stil und Inhalt der Ausgabe zu lenken [22, 23]. Angesichts der Vielzahl mittlerweile beschriebener Techniken weisen systematische Übersichten allerdings auf eine uneinheitliche Begriffsbildung und eine teils fragmentierte Forschungslage hin,

was eine sorgfältige Auswahl und Definition der eingesetzten Strategien erforderlich macht [22, 23]. Für die vorliegende Arbeit werden daher klar abgegrenzte Prompting-Strategien definiert und systematisch gegeneinander verglichen.

2.5.2 Einflussfaktoren auf die Ergebnisqualität

Die Qualität der von einem LLM erzeugten Ausgabe hängt von einer Reihe von Einflussfaktoren ab, die über die reine Wahl der Prompting-Strategie hinausgehen. Beim Few-Shot-Prompting wirken sich Auswahl, Anzahl und Reihenfolge der mitgegebenen Beispiele unmittelbar auf das Ergebnis aus, sodass schon kleine Änderungen an den Beispielen die Modellantwort verschieben können [19, 23]. Auch die genaue Formulierung und Strukturierung der Anweisung beeinflusst die Ergebnisqualität erheblich, weshalb das Auffinden geeigneter Prompts häufig einen iterativen Prozess darstellt [22, 24].

Ein technisch bedeutsamer Parameter ist die Temperatur, die steuert, wie stark die Ausgabe von der wahrscheinlichsten Vorhersage abweicht: Niedrige Werte führen zu deterministischeren und stabileren, hohe Werte zu vielfältigeren, aber weniger vorher-sagbaren Antworten, was die Reproduzierbarkeit der Ergebnisse unmittelbar berührt [6, 23]. Zudem skaliert die Leistungsfähigkeit beim Prompting mit der Größe und Leistungsfähigkeit des zugrunde liegenden Modells, sodass dieselbe Strategie je nach Modell unterschiedlich gut wirkt [19, 23]. Schließlich ist mit längeren und beispielreicheren Prompts ein höherer Verbrauch an Token und damit ein höherer Aufwand verbunden, sodass zwischen Ergebnisqualität und Effizienz abzuwägen ist [23, 24]. Diese Faktoren begründen, warum in der vorliegenden Arbeit sowohl unterschiedliche Modelle als auch unterschiedliche Prompting-Strategien unter kontrollierten Bedingungen verglichen und ihre Ergebnisse mehrfach erhoben werden.

3. Stand der Forschung

Aufbauend auf den theoretischen Grundlagen ordnet dieses Kapitel die vorliegende Arbeit in den aktuellen Forschungsstand ein. Zunächst wird der breitere Einsatz von Künstlicher Intelligenz in der Datenverarbeitung betrachtet (Abschnitt 3.1). Anschließend wird die Forschung zur Verarbeitung strukturierter und tabellarischer Daten durch LLMs aufgearbeitet (Abschnitt 3.2), bevor verwandte Arbeiten zur Automatisierung von ETL-Prozessen diskutiert werden (Abschnitt 3.3). Auf dieser Grundlage wird

abschließend die Forschungslücke präzisiert, an der die vorliegende Arbeit ansetzt (Abschnitt 3.4).

3.1 KI-Einsatz in der Datenverarbeitung

Der Einsatz von Künstlicher Intelligenz und maschinellem Lernen hat die Datenverarbeitung in den vergangenen Jahren grundlegend verändert. Schon vor dem Aufkommen großer Sprachmodelle wurden Verfahren des maschinellen Lernens eingesetzt, um Schritte der Datenaufbereitung, der Mustererkennung und der Entscheidungsunterstützung zu automatisieren und so aus großen Datenbeständen Erkenntnisse zu gewinnen [15, 16]. Insbesondere im Bereich der Datenaufbereitung haben sich lernende Verfahren als nützlich erwiesen, etwa bei der Datenbereinigung, der Imputation fehlender Werte, dem Abgleich von Entitäten und der Erkennung von Ausreißern [13, 17]. Eine systematische Übersicht zeigt dabei ein wechselseitiges Verhältnis: Maschinelles Lernen wird einerseits zur Datenbereinigung genutzt, andererseits ist eine hohe Datenqualität Voraussetzung für die Leistungsfähigkeit der Modelle selbst [13, 17].

Mit der Verfügbarkeit dialogfähiger Sprachmodelle hat sich dieser Trend noch einmal verstärkt. Frühe Arbeiten beschreiben, wie Modelle wie ChatGPT Datenfachleute bei zahlreichen Teilschritten ihres Arbeitsablaufs unterstützen können, von der Datenbereinigung und Vorverarbeitung über die Modellbildung bis hin zur Interpretation der Ergebnisse [7, 16]. Gleichzeitig betonen diese Beiträge, dass der Einsatz solcher Modelle mit Herausforderungen wie Verzerrungen, mangelnder Nachvollziehbarkeit und Fragen der Verlässlichkeit verbunden ist und daher einer kritischen Begleitung bedarf [7, 15]. Insgesamt zeichnet die Literatur das Bild einer zunehmend KI-gestützten Datenverarbeitung, in der die Automatisierung von Routineaufgaben im Vordergrund steht, eine systematische empirische Bewertung der erzielten Qualität jedoch häufig noch aussteht [13, 17].

3.2 LLMs für strukturierte Daten und Tabellen

Während LLMs ursprünglich für die Verarbeitung natürlicher Sprache entwickelt wurden, hat sich die Forschung zunehmend ihrer Anwendung auf strukturierte und tabellarische Daten zugewandt. Hierbei zeigt sich, dass die Modelle prinzipiell in der Lage sind, tabellarische Inhalte zu interpretieren, Zusammenhänge zwischen Feldern zu erfassen und auf dieser Basis Transformations- und Bereinigungsaufgaben zu unterstüt-

zen [7, 25]. Ein wichtiger Anwendungsstrang ist die semantische Interpretation komplexer Datenbeziehungen, bei der LLMs über den rein syntaktischen Abgleich hinaus den inhaltlichen Bezug zwischen Datensätzen erschließen und so etwa beim Abbilden unterschiedlicher Schemata helfen können [25, 26].

Ein zweiter, intensiv untersuchter Bereich ist die Erzeugung synthetischer Daten durch LLMs. Mehrere Übersichtsarbeiten dokumentieren, dass sich große Sprachmodelle eignen, um realistische tabellarische und textuelle Datensätze zu generieren, die etwa zur Datenanreicherung oder zum Ausgleich seltener Fälle dienen [27, 28]. Zugleich verweisen empirische Studien auf deutliche Grenzen: Die Eignung der erzeugten Daten variiert stark je nach Aufgabe, und insbesondere bei subjektiven oder komplexen Aufgabenstellungen bleibt die Qualität der synthetischen Daten hinter realen Daten zurück [27, 29]. Für tabellarische Daten im Speziellen weist die Literatur darauf hin, dass dieser Datentyp trotz seiner praktischen Bedeutung lange vernachlässigt wurde und eigene methodische Herausforderungen mit sich bringt [30, 31]. Diese Befunde sind für die vorliegende Arbeit in zweifacher Hinsicht relevant: Sie begründen einerseits die Verwendung eines synthetischen Datensatzes und mahnen andererseits zur empirischen Prüfung der tatsächlichen Verarbeitungsqualität, statt sich auf die grundsätzliche Fähigkeit der Modelle zu verlassen [29, 31]. Da eine solche empirische Prüfung im produktiven Einsatz jedoch nicht bei jeder Ausführung wiederholt werden kann, rückt zugleich die Stabilität der Ergebnisse über wiederholte Ausführungen in den Blick.

3.3 Verwandte Arbeiten zur Automatisierung von ETL

Die Automatisierung von ETL-Prozessen mit Methoden der Künstlichen Intelligenz ist Gegenstand einer wachsenden Zahl von Arbeiten. Ein erster, schon länger verfolgter Ansatz nutzt klassische Verfahren des maschinellen Lernens, um einzelne Schritte der Pipeline zu automatisieren, etwa das Abbilden von Schemata, die Erkennung von Anomalien und die Ableitung von Transformationsregeln aus den Daten [3, 26]. Darauf aufbauend beschreibt eine Reihe neuerer Beiträge umfassendere Architekturen, in denen KI-Komponenten auf allen Stufen des ETL-Prozesses eingebettet werden, um Aufgaben wie automatisierte Schemaerkennung, intelligente Qualitätssicherung und Leistungsoptimierung zu übernehmen [9, 32].

Ein wiederkehrendes Leitbild dieser Arbeiten ist die Vorstellung selbstheilender und adaptiver Datenpipelines, die Probleme proaktiv erkennen, eigenständig beheben und sich an veränderte Datenstrukturen anpassen, statt rein reaktiv auf Fehler zu reagieren

[8, 26]. In diesem Zusammenhang werden zunehmend autonome KI-Agenten diskutiert, die Datenaufnahme, Bereinigung und Integration weitgehend selbstständig ausführen sollen [8, 33]. Mehrere Beiträge widmen sich dabei spezifischen Teilaufgaben, etwa der intelligenten Anomalieerkennung in ETL-Strecken, der automatisierten Qualitätssicherung oder der automatisierten Erzeugung von Merkmalen [14, 34]. Insbesondere wird betont, dass nicht ein vollständiger Ersatz, sondern eine hybride Verknüpfung von regelbasierten Systemen und KI-Modellen erstrebenswert sei, um Nachvollziehbarkeit und Kontrolle bei gleichzeitig höherer Flexibilität zu wahren [9, 32].

Arbeiten, die explizit generative Sprachmodelle für die ETL-Automatisierung einsetzen, sind hingegen noch vergleichsweise jung. Eine Untersuchung zur generativen KI in der ETL-Automatisierung zeigt, dass sich mit Hilfe von Einbettungs- und generativen Verfahren aus natürlichsprachlichen Anfragen passende ETL-Lösungen ableiten lassen, der manuelle Aufwand dadurch deutlich reduziert, jedoch nicht vollständig beseitigt wird [4, 35]. Insgesamt überwiegen in diesem Feld konzeptionelle und architektonische Beiträge, die das Potenzial KI-gestützter ETL-Prozesse beschreiben, während belastbare, vergleichende Evaluierungen einzelner Modelle und Verarbeitungsstrategien noch selten sind [36, 37].

3.4 Forschungslücke und Abgrenzung

Aus der Aufarbeitung des Forschungsstands lassen sich mehrere Lücken ableiten, an denen die vorliegende Arbeit ansetzt. Erstens beschreiben viele Beiträge das Potenzial KI-gestützter ETL-Prozesse auf konzeptioneller oder architektonischer Ebene, ohne dieses Potenzial unter kontrollierten Bedingungen empirisch zu überprüfen [9, 36]. Zweitens stützt sich ein Großteil der bestehenden Arbeiten auf klassische Verfahren des maschinellen Lernens, während der gezielte Einsatz aktueller LLMs für typische ETL-Teilaufgaben und insbesondere dessen systematischer Vergleich mit etablierten Verfahren bislang nur unzureichend untersucht ist [3, 35]. Drittens fehlt es an Untersuchungen, die mehrere aktuelle Modelle und unterschiedliche Prompting-Strategien gemeinsam und unter einheitlichen Bedingungen gegen eine klassische, regelbasierte Pipeline stellen, obwohl die Literatur sowohl die Heterogenität der Modelle als auch den starken Einfluss der Prompt-Gestaltung deutlich herausstellt [20, 23]. Viertens wird die Reproduzierbarkeit der Ergebnisse, die sich unmittelbar aus der probabilistischen Natur der Modelle ergibt, in den vorhandenen Arbeiten kaum als eigenständige Untersuchungsdimension behandelt, obwohl sie für den praktischen Einsatz von zentraler Bedeutung ist [6, 7].

An genau diesen Lücken setzt die vorliegende Arbeit an. Sie verbindet die genannten Aspekte zu einer experimentellen Evaluierung, die mehrere aktuelle LLMs und unterschiedliche Prompting-Strategien anhand eines kontrolliert erstellten synthetischen Datensatzes für die Teilaufgaben Datenbereinigung, Datentransformation und Duplikaterkennung untersucht und die Ergebnisse mit einer klassisch implementierten ETL-Pipeline vergleicht. Durch die mehrfache Ausführung identischer Aufgaben wird die Reproduzierbarkeit als eigenständige Dimension einbezogen. Damit grenzt sich die Arbeit von rein konzeptionellen Beiträgen ab und liefert eine empirisch fundierte Grundlage, um die in Kapitel 1 formulierten Forschungsfragen zu beantworten und Handlungsempfehlungen für den praktischen Einsatz abzuleiten [9, 35].

4. Konzeption der experimentellen Evaluierung

Aufbauend auf den theoretischen Grundlagen aus Kapitel 2 und der in Kapitel 3 präzisierten Forschungslücke beschreibt dieses Kapitel die Konzeption der experimentellen Evaluierung. Es überführt die in Abschnitt 1.2 formulierten Forschungsfragen in ein konkretes Untersuchungsdesign und legt damit die methodische Grundlage für die folgende Umsetzung. Zunächst werden das übergeordnete Forschungsdesign und die daraus abgeleiteten Hypothesen dargelegt (Abschnitt 4.1). Anschließend werden die Auswahl der untersuchten LLMs (Abschnitt 4.2), die Definition der betrachteten ETL-Aufgaben (Abschnitt 4.3) sowie die verglichenen Prompting-Strategien (Abschnitt 4.4) beschrieben. Darauf folgt die Operationalisierung der Evaluationskriterien (Abschnitt 4.5), bevor abschließend die klassische ETL-Pipeline als Vergleichsbasis definiert wird (Abschnitt 4.6). Die konkrete technische Umsetzung des hier entworfenen Designs ist Gegenstand der Kapitel 5 bis 7.

4.1 Forschungsdesign und Hypothesen

Zur Beantwortung der Forschungsfragen wird ein experimentelles Untersuchungsdesign gewählt, das den Einsatz von LLMs für ausgewählte ETL-Teilaufgaben unter kontrollierten Bedingungen mit einer klassischen, regelbasierten Pipeline vergleicht. Ein experimenteller Zugang ist hier angemessen, da die Forschungslücke gerade im Fehlen belastbarer, vergleichender Evaluierungen unter einheitlichen Bedingungen besteht, während das grundsätzliche Potenzial KI-gestützter ETL-Prozesse in der Lite-

ratur bereits vielfach konzeptionell beschrieben wurde [9, 36]. Den Kern des Designs bildet ein vollständig gekreuztes Versuchsschema über die drei Faktoren Modell, Prompting-Strategie und ETL-Aufgabe: Für jede Kombination dieser Faktoren wird die Verarbeitung durchgeführt und anhand einheitlicher, in Abschnitt 4.5 definierter Kriterien bewertet, sodass sich die Einflüsse der einzelnen Faktoren systematisch isolieren lassen [6, 10].

Eine zentrale Gestaltungsentscheidung betrifft die Art und Weise, wie die Modelle in den Verarbeitungsprozess eingebunden werden. Als primärer Ausführungsmodus werden die Modelle angewiesen, einen ausführbaren ETL-Verarbeitungsschritt in Form von Python-Code zu erzeugen, der anschließend deterministisch auf den Daten ausgeführt wird, statt die Eingabedaten unmittelbar zu transformieren. Dieser Ansatz der Code-Generierung hat mehrere Vorteile: Er entkoppelt die Verarbeitung von der Datenmenge und umgeht damit die Begrenzung durch das Kontextfenster, er erzeugt eine nachvollziehbare und wiederausführbare Artefaktbasis und er stellt zugleich die unmittelbare Vergleichbarkeit mit der regelbasierten Pipeline her, die ebenfalls als programmatische Lösung auf Basis der Bibliothek Pandas umgesetzt wird [1, 4]. Der erzeugte Code wird ausgeführt, sein Ergebnis als tabellarische Ausgabedatei gespeichert und gegen eine bekannte Soll-Lösung (Ground Truth) geprüft.

Ergänzend zu diesem Vorgehen wird ein zweiter Ausführungsmodus untersucht, die direkte In-Context-Verarbeitung, bei der dem Modell die Eingabedaten unmittelbar im Prompt bereitgestellt werden und es die transformierte Ergebnistabelle ohne den Umweg über generierten Code direkt zurückgibt. Dieser Modus zielt gezielt auf die semantisch geprägten Aufgaben, bei denen ein regelbasiertes Vorgehen – ob als manuell erstellte Pipeline oder als generierter Code – an seine Grenzen stößt, weil sich das erforderliche Wissen nicht vollständig in feste Regeln überführen lässt, etwa bei der Vereinheitlichung uneinheitlich geschriebener Länderangaben oder der Erkennung unscharfer Duplikate [7, 25]. Indem das Modell sein Sprach- und Weltwissen unmittelbar auf jeden einzelnen Wert anwendet, anstatt eine verallgemeinernde Verarbeitungsvorschrift zu formulieren, lässt sich prüfen, ob die direkte Verarbeitung bei solchen Aufgaben Vorteile gegenüber der Code-Generierung erzielt. Da die Daten hierbei vollständig im Prompt mitgeführt werden, ist dieser Modus durch das Kontext- und Ausgabefenster der Modelle begrenzt und eignet sich vor allem für Datenmengen überschaubaren Umfangs; er wird daher nicht als vollständig gekreuzter Faktor, sondern als ergänzender Vergleich auf denselben Aufgaben und Modellen geführt [6, 24].

Die bis hierhin beschriebene Faktormatrix betrachtet jede Aufgabe für sich, in dem Sinne, dass jede Konfiguration unmittelbar auf den unveränderten Rohdaten ansetzt. Eine reale ETL-Strecke besteht jedoch nicht aus voneinander unabhängigen Einzelschritten, sondern aus einer Kette, in der jeder Schritt auf dem Ergebnis seines Vorgängers aufsetzt [1, 2]. Um die Übertragbarkeit der Einzelergebnisse auf diesen praktisch maßgeblichen Fall beurteilen zu können, wird das Design um eine dritte Untersuchungsebene ergänzt: Dieselben Aufgaben werden zusätzlich als durchgängige, verkettete Pipeline ausgeführt, in der die Ausgabe jedes Schritts die Eingabe des nächsten bildet und ein Abbruch die abhängigen Schritte blockiert. Damit lässt sich beobachten, wie sich lokal geringe Abweichungen über die Kette hinweg auswirken, was in der isolierten Betrachtung einer einzelnen Aufgabe grundsätzlich unsichtbar bleibt.

Damit ein in der Kette gemessener Qualitätsverlust jedoch nicht fälschlich dem betreffenden Schritt zugeschrieben wird, obwohl er aus fehlerhaften Eingabedaten resultiert, wird die verkettete Ausführung um einen zweiten, sonst identischen Durchlauf ergänzt, in dem jeder Schritt statt der Ausgabe seines Vorgängers die entsprechende Soll-Zwischentabelle als Eingabe erhält. Da die Aufgabenstellung unverändert bleibt und lediglich die Eingabe fehlerfrei ist, misst dieser isolierte Durchlauf die Leistung eines Schritts ohne Fehlerfortpflanzung; die Differenz beider Durchläufe quantifiziert unmittelbar den Anteil des Ergebnisverlusts, der auf vorgelagerte Schritte zurückgeht. Diese Trennung ist für die Beantwortung von FF1 unmittelbar bedeutsam, weil eine Empfehlung je Aufgabentyp voraussetzt, dass ein beobachteter Fehler dem verursachenden Schritt und nicht seinem Nachfolger zugerechnet wird [4, 10].

Da LLMs aufgrund ihrer probabilistischen Natur auch bei identischer Eingabe voneinander abweichende Ausgaben erzeugen können, wird die Reproduzierbarkeit als eigenständige Untersuchungsdimension in das Design aufgenommen [6, 7]. Hierzu wird jede Konfiguration aus Modell, Prompting-Strategie und Aufgabe mehrfach ausgeführt, sodass die Streuung der Ergebnisse über die Wiederholungen hinweg quantifiziert werden kann. Die mehrfache Ausführung bezieht sich dabei auf den Generierungsschritt: Für jede Wiederholung wird das Modell erneut zur Code-Erzeugung aufgefordert, sodass sich die probabilistische Variabilität der Modelle in voneinander abweichenden Code-Artefakten niederschlagen kann, während die anschließende Ausführung eines einzelnen erzeugten Skripts selbst deterministisch bleibt. Steuerbare Einflussgrößen wie die in Abschnitt 2.5.2 eingeführte Temperatur werden über alle Durchläufe konstant und auf einen niedrigen Wert festgelegt, um die nicht durch das Modell selbst bedingte Variabilität zu minimieren und vergleichbare Bedingungen zu schaffen

[6, 23]. Als unveränderliche Grundlage aller Experimente dient ein kontrolliert erzeugter synthetischer Datensatz, dessen gezielte Konstruktion in Kapitel 5 beschrieben wird und der es erlaubt, die Eigenschaften der Eingangsdaten exakt zu kennen.

Aus den Forschungsfragen und dem Forschungsstand lassen sich die folgenden Hypothesen ableiten, die im Rahmen der Auswertung in Kapitel 8 geprüft werden. Entsprechend der Struktur der Zielsetzung wird dabei je Forschungsfrage genau eine Hypothese formuliert, sodass jede von ihnen unmittelbar auf eine der beiden Fragen bezogen bleibt und deren Beantwortung vorbereitet:

H1 (zu FF1): Die Eignung von LLMs hängt stark vom Aufgabentyp ab. Bei klar formalisierbaren Aufgaben erreichen sie bestenfalls eine mit der regelbasierten Pipeline vergleichbare Ergebnisqualität, während sie bei Aufgaben mit hoher semantischer Anforderung Vorteile zeigen und mit zunehmender struktureller Komplexität an Zuverlässigkeit verlieren. Ein über alle Aufgaben hinweg überlegener Ansatz existiert daher nicht; welches Vorgehen vorzuziehen ist, ergibt sich erst aus dem Zusammenspiel von Aufgabentyp, Qualitätsanforderung und Effizienzrahmen [7, 9].

H2 (zu FF2): Aufwändigere Prompting-Strategien, die zusätzlichen Kontext, Beispiele oder Zwischenschritte bereitstellen, verbessern die Ergebnisqualität gegenüber einem einfachen Zero-Shot-Prompt, wobei der Nutzenzuwachs mit steigendem Aufwand abnimmt und der zusätzliche Kontext bei zu enger Führung auch nachteilig wirken kann [22, 23].

Die Reproduzierbarkeit wird, wie bereits in Abschnitt 1.2 festgelegt, nicht als eigenständige Forschungsfrage und entsprechend auch nicht als eigenständige Hypothese geführt. Sie wird stattdessen als querschnittliche Dimension zu beiden Hypothesen erhoben: Für H1 ist die Stabilität ein Bestandteil der Eignungsbeurteilung, da ein im Mittel gutes, aber stark schwankendes Ergebnis für den produktiven Einsatz einen anderen Wert besitzt als ein gleich gutes, aber verlässlich wiederholbares; für H2 ist sie zusätzliches Unterscheidungsmerkmal zwischen den Strategien. Erwartet wird dabei, dass wiederholte Ausführungen identischer Konfigurationen trotz niedrig gewählter Temperatur messbar voneinander abweichen und die Stabilität mit zunehmender Komplexität der Aufgabe abnimmt [6, 7].

4.2 Auswahl der Large Language Models

Die Auswahl der untersuchten Modelle folgt dem Ziel, ein für den praktischen Einsatz repräsentatives und zugleich aussagekräftiges Spektrum aktueller LLMs abzudecken. Wie in Abschnitt 2.4.2 dargestellt, unterscheiden sich die verfügbaren Modellfamilien nicht nur in Architektur und Parameterzahl, sondern auch in praxisrelevanten Eigenschaften wie Kontextfenster, Verarbeitungsgeschwindigkeit, Nutzungskosten und Verfügbarkeit als offenes oder geschlossenes Modell [6, 20]. Gerade diese Heterogenität begründet den Vergleich mehrerer Modelle, da sich die Eignung für ETL-Aufgaben zwischen ihnen unterscheiden kann und ein Einzelmodell keine verallgemeinerbaren Aussagen zuließe [20, 21].

Als Auswahlkriterien werden die Aktualität und Verbreitung der Modelle, die Abdeckung unterschiedlicher Anbieter sowie die Berücksichtigung sowohl geschlossener als auch offen verfügbarer Modelle herangezogen. Auf dieser Grundlage werden je ein Vertreter der etablierten geschlossenen Modellfamilien von OpenAI, Anthropic und Google sowie ein offen verfügbares, lokal ausführbares Modell aus der Llama-Reihe untersucht. Damit deckt die Auswahl unterschiedliche Kosten-, Leistungs- und Betriebsmodelle ab und erlaubt insbesondere den praktisch bedeutsamen Vergleich zwischen kommerziell betriebenen und lokal betreibbaren Modellen [6, 20]. Um die Modelle technisch austauschbar ansprechen zu können, werden sie hinter einer einheitlichen Schnittstelle gekapselt, sodass der Versuchsablauf unabhängig vom konkreten Anbieter erfolgt; die Details dieser Abstraktion werden in Kapitel 6 beschrieben. Da sich das Feld der Modelle mit hoher Geschwindigkeit weiterentwickelt, wird die exakte Versionskennung jedes Modells bei jeder Ausführung protokolliert, um die spätere Nachvollziehbarkeit und Reproduzierbarkeit der Ergebnisse sicherzustellen [6, 20].

4.3 Definition der ETL-Aufgaben

Die untersuchten ETL-Aufgaben orientieren sich an den in Abschnitt 2.2.2 beschriebenen, in der Praxis am häufigsten auftretenden Datenproblemen und decken die drei in dieser Arbeit fokussierten Teilbereiche der Transformationsphase ab: die Datenbereinigung, die Datentransformation und die Duplikaterkennung. Um nicht nur die grundsätzliche Eignung, sondern auch deren Grenzen in Abhängigkeit von der Aufgabenkomplexität zu erfassen, wird jeder Teilbereich in drei aufeinander aufbauenden Schwierigkeitsstufen operationalisiert. Diese reichen von einer einfachen, klar formalisierbaren Aufgabe über eine mittlere Stufe bis hin zu einer anspruchsvollen Variante,

die semantisches Verständnis oder mehrstufige Verarbeitung erfordert. Daraus ergibt sich eine Matrix aus neun Aufgaben, die zugleich die Schwierigkeitsachse als systematische Vergleichsdimension einführt und so eine differenzierte Beurteilung der Modelle erlaubt [12, 13]. Alle Aufgaben beziehen sich auf den synthetischen Datensatz aus Kapitel 5, der ein kleines E-Commerce-Szenario mit den Tabellen Kunden, Produkte und Bestellungen abbildet, und sind so gestaltet, dass für jede von ihnen eine eindeutige Soll-Lösung existiert.

4.3.1 Datenbereinigung

Die Aufgaben zur Datenbereinigung adressieren die Beseitigung von Mängeln, die unmittelbar die Genauigkeit, Vollständigkeit und Konsistenz des Datenbestands beeinträchtigen [11, 13]. Die einfache Stufe umfasst grundlegende Bereinigungsoperationen wie das Entfernen überflüssiger Leerzeichen in Textspalten und das einheitliche Kennzeichnen fehlender Werte; diese Aufgabe ist mit wenigen Regeln vollständig beschreibbar und dient als Referenzpunkt für die grundsätzliche Verarbeitungsfähigkeit der Modelle. Die mittlere Stufe verlangt die Vereinheitlichung uneinheitlich formatierter Datumsangaben, die in mehreren unterschiedlichen Darstellungen vorliegen, auf ein einheitliches Zielformat und prüft damit die Fähigkeit zur formatübergreifenden Normalisierung. Die anspruchsvolle Stufe schließlich erfordert die semantische Vereinheitlichung von Länderangaben, die in verschiedenen Sprachen, Abkürzungen und Schreibweisen einschließlich Tippfehlern vorliegen, auf einen standardisierten Code. Gerade diese Aufgabe stellt eine für LLMs besonders aufschlussreiche Achse dar, da sie über den rein syntaktischen Abgleich hinausgeht und ein inhaltliches Verständnis der Werte voraussetzt, das mit starren Regeln nur schwer abzubilden ist [7, 25].

4.3.2 Datentransformation

Die Aufgaben zur Datentransformation betreffen die Überführung der Daten in die geforderte Struktur und Typisierung sowie die Ableitung neuer Informationen aus vorhandenen Werten. Die einfache Stufe umfasst Typkonvertierungen, etwa die Umwandlung uneinheitlich formatierter Preisangaben in einen numerischen Wert und textueller Wahrheitswerte in einen Boolean-Datentyp, und prüft damit die korrekte Behandlung von Datentypen. Die mittlere Stufe verlangt die Berechnung abgeleiteter Spalten aus bestehenden Feldern, beispielsweise die Ermittlung eines Gesamtbetrags sowie die Extraktion einzelner Datumsbestandteile, und adressiert damit zeilenweise Berechnungslogik. Die anspruchsvolle Stufe bildet die komplexeste Aufgabe der gesamten

Untersuchung und verbindet mehrere Verarbeitungsschritte: Sie erfordert die Verknüpfung der drei Tabellen über ihre Schlüsselbeziehungen, eine begleitende Bereinigung der beteiligten Felder sowie eine anschließende Gruppierung und Aggregation der Ergebnisse. Damit prüft sie, ob die Modelle eine mehrstufige, aus Join, Bereinigung und Aggregation zusammengesetzte Verarbeitungslogik korrekt umsetzen können, wie sie für reale ETL-Strecken charakteristisch ist [1, 2].

4.3.3 Duplikaterkennung

Die Aufgaben zur Duplikaterkennung zielen auf die Eindeutigkeit des Datenbestands und damit auf die Identifikation und Auflösung mehrfach vorhandener Datensätze [13, 14]. Die einfache Stufe beschränkt sich auf exakte Duplikate, bei denen vollständig identische Datensätze zu entfernen sind, und ist mit einem deterministischen Abgleich vollständig lösbar. Die mittlere Stufe verlangt die schlüsselbasierte Entdoppelung anhand eines eindeutigen Merkmals, wobei bei mehreren übereinstimmenden Datensätzen anhand einer definierten Regel der zu behaltende Datensatz ausgewählt werden muss. Die anspruchsvolle Stufe adressiert schließlich unscharfe Duplikate (Fuzzy-Duplikate), bei denen derselbe reale Sachverhalt in abweichenden Schreibweisen vorliegt, etwa durch Tippfehler, unterschiedliche Groß- und Kleinschreibung oder geringfügig abweichende Formatierungen. Da hier ein exakter Abgleich nicht ausreicht und stattdessen eine Ähnlichkeitsbetrachtung erforderlich ist, bildet diese Aufgabe die zweite zentrale Achse, an der sich die semantischen Fähigkeiten der Modelle gegenüber regelbasierten Verfahren zeigen [7, 13].

4.4 Prompting-Strategien

Da LLMs ihre Aufgaben primär über das In-Context-Lernen lösen, kommt der Gestaltung der Eingabe eine entscheidende Bedeutung für die Ergebnisqualität zu, weshalb die Prompting-Strategie als eigenständiger Untersuchungsfaktor in das Design aufgenommen wird [22, 23]. Angesichts der in Abschnitt 2.5.1 angesprochenen uneinheitlichen Begriffsbildung im Feld werden für diese Arbeit klar abgegrenzte Strategien definiert, die sich in ihrem Aufwand und dem Umfang der bereitgestellten Hilfestellung systematisch unterscheiden und gegeneinander verglichen werden.

Den Ausgangspunkt bildet das Zero-Shot-Prompting, bei dem das Modell die Aufgabe allein anhand einer präzisen Anweisung und ohne weitere Beispiele bearbeitet. Diese Strategie dient als Referenzbasis, die misst, was ein Modell ohne zusätzliche Hilfe-

stellung aus einer reinen Aufgabenbeschreibung erzeugt [19, 22]. Darauf aufbauend wird das Few-Shot-Prompting untersucht, das der Anweisung einige beispielhafte Ein- und Ausgabepaare beifügt, anhand derer das Modell das gewünschte Verhalten ableitet und das die Ergebnisqualität bei vielen Aufgaben spürbar verbessert [19, 23]. Als dritte Strategie wird das Chain-of-Thought-Prompting einbezogen, bei dem das Modell zur Entwicklung seiner Lösung in nachvollziehbaren Zwischenschritten angeleitet wird, was sich insbesondere bei den mehrstufigen Aufgaben dieser Untersuchung als wirksam erweisen könnte [22, 23]. Ergänzend wird eine schema-angereicherte Variante betrachtet, die der Anweisung eine explizite Beschreibung des erwarteten Zielschemas beifügt, um die strukturelle Korrektheit der Ausgabe zu unterstützen. Um faire Vergleichsbedingungen zu gewährleisten, werden die Strategien für alle Aufgaben nach einem einheitlichen Aufbau formuliert und versioniert vorgehalten, sodass jede eingesetzte Prompt-Variante eindeutig dokumentiert und wiederverwendbar bleibt [23, 24].

4.5 Evaluationskriterien

Damit die Ergebnisse aller Verfahren über Modelle, Prompting-Strategien und Aufgaben hinweg vergleichbar gemessen werden können, werden die Bewertungskriterien aus den in Abschnitt 2.2.1 eingeführten Dimensionen der Datenqualität abgeleitet und operationalisiert [10, 12]. Grundlage jeder Bewertung ist der Abgleich der erzeugten Ausgabedatei mit der für jede Aufgabe bekannten Soll-Lösung. Die folgenden Unterabschnitte beschreiben die vier herangezogenen Kriterien Genauigkeit, Vollständigkeit, Konsistenz und Effizienz. Die Stabilität der Ergebnisse – als querschnittliche Reproduzierbarkeitsdimension über beide Forschungsfragen hinweg – wird nicht als separates Einzelkriterium erhoben, sondern daraus abgeleitet, wie stark die genannten Qualitätskennzahlen über die wiederholten Ausführungen einer Konfiguration streuen: Eine geringe Streuung entspricht dabei einer hohen, eine große Streuung einer geringen Stabilität [6, 7].

4.5.1 Genauigkeit

Die Genauigkeit erfasst, inwieweit die erzeugten Werte mit der Soll-Lösung übereinstimmen, und bildet das zentrale Korrektheitskriterium [11, 12]. Sie wird als Anteil der korrekt verarbeiteten Werte am Gesamtbestand operationalisiert, sodass auch teilweise korrekte Ergebnisse differenziert bewertet werden können und nicht jede Abweichung gleichermaßen ins Gewicht fällt. Voraussetzung für diesen Vergleich ist die

Übereinstimmung der grundlegenden Struktur, also der Spaltenmenge und der Datensatzanzahl; weicht diese ab, wird dies gesondert ausgewiesen, da in einem solchen Fall kein sinnvoller Abgleich möglich ist. Auf diese Weise lässt sich die Genauigkeit als kontinuierliche Kennzahl zwischen vollständiger Übereinstimmung und vollständiger Abweichung abbilden.

4.5.2 Vollständigkeit

Die Vollständigkeit gibt an, inwieweit das Ergebnis alle geforderten Datensätze und Felder enthält, ohne dass erwartete Werte fehlen oder unzulässig entfernt werden [11, 12]. Sie wird über den Abgleich der erzeugten Struktur mit der Soll-Lösung erfasst, insbesondere über das Vorhandensein aller erwarteten Spalten und die korrekte Anzahl der Datensätze. Dieses Kriterium ist gerade für die Aufgaben der Duplikaterkennung von Bedeutung, bei denen sowohl ein unzureichendes als auch ein übermäßiges Entfernen von Datensätzen die Vollständigkeit verletzt, und es ergänzt die Genauigkeit um die Frage, ob der Umfang des Ergebnisses dem erwarteten Umfang entspricht.

4.5.3 Konsistenz

Die Konsistenz bezieht sich auf die Widerspruchsfreiheit und Einheitlichkeit des Ergebnisses, insbesondere auf die Übereinstimmung der Datentypen und Formate mit dem erwarteten Zielschema [11, 12]. Bewertet wird, ob die Ausgabe das geforderte Schema einhält, ob die einzelnen Felder die erwarteten Datentypen aufweisen und ob Werte einheitlich dargestellt werden. Dieses Kriterium ist vor allem für die Transformationsaufgaben relevant, bei denen die korrekte Typisierung und Formatierung ein wesentliches Ergebnismerkmal darstellt, und es prüft die strukturelle Verlässlichkeit der erzeugten Daten unabhängig von der inhaltlichen Korrektheit einzelner Werte.

4.5.4 Effizienz

Die Effizienz erfasst den für die Verarbeitung erforderlichen Aufwand und ergänzt die qualitätsbezogenen Kriterien um die für den praktischen Einsatz entscheidende Wirtschaftlichkeitsdimension [20, 24]. Erhoben werden die Verarbeitungszeit eines Aufrufs, der Verbrauch an Token sowie die daraus resultierenden Nutzungskosten. Diese Kennzahlen sind insbesondere für die vergleichende Bewertung der Prompting-Strategien relevant, da aufwändigere Strategien mit längeren Eingaben einen höheren Token-

Verbrauch verursachen und somit zwischen Ergebnisqualität und Effizienz abzuwägen ist [23, 24]. Zugleich ermöglichen sie den praxisnahen Vergleich zwischen kommerziell betriebenen und lokal ausgeführten Modellen sowie die Gegenüberstellung mit der klassischen Pipeline, deren Ausführung keine vergleichbaren Modellkosten verursacht.

4.6 Vergleich gegen eine manuelle ETL-Pipeline

Um die Ergebnisse der LLM-gestützten Verarbeitung einordnen zu können, wird als Vergleichsmaßstab eine klassische, regelbasiert implementierte ETL-Pipeline herangezogen, die dieselben neun Aufgaben auf demselben Datensatz löst. Diese Pipeline bildet den etablierten Stand der Technik ab, bei dem die Verarbeitungslogik manuell und explizit in Form von Regeln und Skripten vorgegeben wird, und sie wird ebenfalls programmatisch auf Basis der Bibliothek Pandas umgesetzt, um eine technisch vergleichbare Ausgangslage herzustellen [3, 10]. Sie dient als Referenzpunkt, gegen den sich die Stärken und Schwächen der KI-gestützten Ansätze messen lassen, und liefert damit die Grundlage zur Beantwortung von FF1.

Charakteristisch für diese Vergleichsbasis ist, dass sie deterministische und vollständig nachvollziehbare Ergebnisse erzeugt: Identische Eingaben führen stets zu identischen Ausgaben, und die zugrunde liegende Logik ist explizit einsehbar [3, 4]. Damit bildet die regelbasierte Pipeline insbesondere bei den klar formalisierbaren Aufgaben die Referenz, während gerade bei den semantisch geprägten Aufgaben der Länder-Vereinheitlichung und der Fuzzy-Duplikaterkennung erwartet wird, dass sich die Grenzen rein regelbasierter Verfahren zeigen. Diese arbeiten zwar auch bei solchen Aufgaben deterministisch, erfassen die zugrunde liegende semantische Ähnlichkeit jedoch nur eingeschränkt. Die Pipeline wird mit den identischen Evaluationskriterien aus Abschnitt 4.5 bewertet, sodass ein unmittelbarer Vergleich zwischen klassischem und KI-gestütztem Vorgehen unter gleichen Bedingungen möglich ist und sich auf dieser Grundlage in Kapitel 9 Handlungsempfehlungen ableiten lassen. Ein hybrider Ansatz, der beide Verfahren kombiniert, ist dabei nicht selbst Gegenstand der Evaluierung; da sich die Nachvollziehbarkeit regelbasierter und die Flexibilität KI-gestützter Verfahren in der Praxis jedoch häufig ergänzen, wird er als naheliegende Konsequenz des Vergleichs in den Handlungsempfehlungen aufgegriffen [9, 32].

5. Erstellung des synthetischen Datensatzes

Das in Kapitel 4 entworfene Untersuchungsdesign setzt einen Datensatz voraus, dessen Eigenschaften exakt bekannt sind: Nur wenn die enthaltenen Datenqualitätsprobleme gezielt eingebracht wurden und für jede Aufgabe eine eindeutige Soll-Lösung existiert, lassen sich die in Abschnitt 4.5 definierten Kriterien überhaupt objektiv messen. Dieses Kapitel dokumentiert daher die Erstellung des synthetischen Datensatzes, der in Abschnitt 4.1 bereits als unveränderliche Grundlage aller Experimente eingeführt wurde. Es schließt unmittelbar an die in Abschnitt 4.3 definierten neun ETL-Aufgaben an und zeigt, wie das kleine E-Commerce-Szenario mit den Tabellen Kunden, Produkte und Bestellungen so konstruiert wird, dass genau die dort geforderten Probleme in kontrolliertem Umfang vorliegen. Zunächst werden die aus dem Forschungsdesign abgeleiteten Anforderungen begründet (Abschnitt 5.1), anschließend das Datenmodell und die saubere Zielstruktur beschrieben (Abschnitt 5.2). Darauf folgt die detaillierte Zuordnung der gezielt eingebauten Qualitätsprobleme zu den einzelnen Aufgaben (Abschnitt 5.3), bevor das zweistufige Generierungsverfahren (Abschnitt 5.4) und die abschließende Validierung des Datensatzes (Abschnitt 5.5) dargelegt werden. Der so erzeugte Datensatz bildet die Eingabe für die technische Umsetzung in Kapitel 6 und die Durchführung der Experimente in Kapitel 7.

5.1 Anforderungen an den Datensatz

Die Anforderungen an den Datensatz ergeben sich unmittelbar aus den Forschungsfragen und dem in Kapitel 4 festgelegten Design. Eine erste Anforderung betrifft die Repräsentativität: Der Datensatz muss die in Abschnitt 2.2.2 beschriebenen, in der Praxis typischen Datenqualitätsprobleme abbilden, damit die Ergebnisse über das konkrete Szenario hinaus aussagekräftig sind und die Aufgaben nicht künstlich vereinfacht wirken [12, 13]. Zugleich soll er nicht beliebige, sondern gerade jene Probleme enthalten, die den neun in Abschnitt 4.3 definierten Aufgaben entsprechen, sodass jede Aufgabe an einem tatsächlich vorhandenen Mangel ansetzt.

Eine zweite, für das gewählte experimentelle Vorgehen zentrale Anforderung ist die vollständige Kontrollierbarkeit. Da die Qualität der Verarbeitung durch den Abgleich mit einer bekannten Soll-Lösung gemessen wird, müssen sowohl der fehlerfreie Soll-

zustand als auch Art und Umfang der eingebrachten Fehler exakt bekannt sein. Bei realen Datenbeständen wäre diese eindeutige Referenz nur mit erheblichem manuellem Annotationsaufwand und verbleibender Unsicherheit herstellbar, weshalb die Literatur den Einsatz synthetischer Daten gerade dann empfiehlt, wenn eine kontrollierte und exakt bekannte Ausgangslage benötigt wird [29, 38]. Die in Abschnitt 3.2 referierten Vorbehalte gegenüber synthetischen Daten betreffen primär deren Einsatz als Trainingsmaterial; für die hier verfolgte Rolle als kontrollierte Testgrundlage überwiegt hingegen der Vorteil der bekannten Ground Truth [27, 31].

Eine dritte Anforderung ist die Reproduzierbarkeit des Datensatzes selbst. Da die Reproduzierbarkeit der Modellergebnisse eine eigenständige, querschnittliche Untersuchungsdimension bildet, darf die Datengrundlage keine zusätzliche Variabilität einführen; die Generierung muss deterministisch sein, sodass sie jederzeit identisch wiederholt werden kann [6, 7]. Schließlich soll der Datensatz eine vierte Anforderung erfüllen, die zwischen Vielfalt und Handhabbarkeit vermittelt: Er muss groß und vielfältig genug sein, um die drei Schwierigkeitsstufen jeder Aufgabe sinnvoll auszuprägen, zugleich aber überschaubar genug bleiben, um die in Abschnitt 4.5.4 erhobenen Aufwandskennzahlen nicht durch reine Datenmenge zu dominieren und die wiederholte Ausführung aller Konfigurationen praktikabel zu halten.

5.2 Datenmodell und -struktur

Als Anwendungsszenario dient ein bewusst einfach gehaltenes E-Commerce-Datenmodell, das aus den drei Tabellen Kunden, Produkte und Bestellungen besteht. Diese Wahl ist kein Selbstzweck, sondern folgt aus den Aufgaben: Das Szenario ist allgemein verständlich, deckt sämtliche in Abschnitt 4.3 geforderten Problemtypen ab und stellt über die Beziehung der Bestellungen zu Kunden und Produkten zugleich die Schlüsselstruktur bereit, die für die anspruchsvolle Transformationsaufgabe aus Join, Bereinigung und Aggregation benötigt wird [1, 2]. Die Tabelle der Bestellungen verweist über Fremdschlüssel auf Kunden und Produkte und verbindet die drei Tabellen so zu einem kleinen relationalen Modell.

Grundlage des Modells ist die Definition eines sauberen Zielschemas für jede Tabelle, das den fehlerfreien Sollzustand festlegt. Für die Kundentabelle umfasst dieses Schema eine eindeutige Kundennummer, den vollständigen Namen, die E-Mail-Adresse, einen zweistelligen Länder-Code nach ISO-3166-1-alpha-2 sowie das Registrierungsdatum. Die Produkttabelle besteht aus einer Produktnummer, der Bezeichnung, einer

Kategorie, dem Preis als nichtnegativem Dezimalwert und einem Wahrheitswert zur Verfügbarkeit. Die Bestelltabelle schließlich verknüpft Bestell-, Kunden- und Produkt-nummer mit der bestellten Menge, dem Bestelldatum und dem Gesamtbetrag. Dieses saubere Schema erfüllt eine doppelte Funktion: Es beschreibt einerseits das von den Modellen zu erreichende Zielformat und dient andererseits, in seiner konkret erzeugten Ausprägung, als bekannte Referenz für die Bewertung. Die typisierte, maschinenlesbare Festlegung des Schemas stellt dabei sicher, dass die als Referenz erzeugten Daten strikt schemakonform sind und damit als verlässliche Ground Truth taugen [11, 12].

Der Umfang des Datensatzes ist so gewählt, dass er die genannte Abwägung zwischen Vielfalt und Handhabbarkeit einlöst. Die Kundentabelle umfasst in ihrer sauberen Fassung fünfhundert Datensätze, die Produkttabelle einhundert und die Bestelltabelle zweitausend Datensätze. Diese Größenordnung ist einerseits ausreichend, um Datenqualitätsprobleme in statistisch merklichem, aber nicht triviales Ausmaß einzustreuen und die Aggregation über Länder und Kategorien mit belastbaren Häufigkeiten zu füllen, andererseits klein genug, um die in Abschnitt 4.6 vorgesehene wiederholte Ausführung über alle Kombinationen aus Modell, Prompting-Strategie und Aufgabe ohne unverhältnismäßigen Zeit- und Kostenaufwand zu ermöglichen.

5.3 Eingebaute Datenqualitätsprobleme

Kern der Datensatzkonstruktion ist das gezielte Einbringen von Qualitätsproblemen, das dieser Abschnitt entlang der drei in Abschnitt 4.3 definierten Aufgabenkategorien beschreibt. Jedes Problem ist dabei nicht beliebig gewählt, sondern korrespondiert mit genau einer Aufgabe und einer der in Abschnitt 2.2.1 eingeführten Qualitätsdimensionen, sodass sich die spätere Bewertung lückenlos auf einen bewusst gesetzten Mangel zurückführen lässt [12, 13]. Auf diese Weise ist die in Abschnitt 4.3 eingeführte Matrix aus neun Aufgaben und drei Schwierigkeitsstufen unmittelbar in der Struktur des Datensatzes verankert.

Die Probleme der *Datenbereinigung* sind ausschließlich in der Kundentabelle verortet und in aufsteigender Schwierigkeit gestaffelt. Für die einfache Stufe werden in einem Teil der Namensfelder zusätzliche Leerzeichen vorangestellt und angehängt und in einem Teil der Länderangaben die Werte gänzlich entfernt, wodurch die für diese Stufe geforderte Beseitigung überflüssiger Leerzeichen und die Kennzeichnung fehlender Werte adressiert werden [11, 13]. Die mittlere Stufe entsteht dadurch, dass das Regis-

trierungsdatum in vier unterschiedlichen Darstellungen abgelegt wird, nämlich als ISO-Datum, im deutschen Format, im US-amerikanischen Format mit ausgeschriebenem Monat sowie als Unix-Zeitstempel; die Aufgabe der formatübergreifenden Normalisierung besitzt damit im Datensatz eine konkrete Entsprechung. Die anspruchsvolle Stufe schließlich wird durch die Verfälschung der Länderangaben erzeugt, indem der saubere ISO-Code durch wechselnde Schreibweisen, Sprachen, Abkürzungen und gezielte Tippfehler ersetzt wird, etwa „Deutschland“, „Germany“ oder „Deutshcland“ anstelle von „DE“. Gerade diese Verfälschung bildet die in Abschnitt 4.3.1 hervorgehobene semantische Achse ab, da ihre Auflösung ein inhaltliches Verständnis der Werte und nicht nur einen syntaktischen Abgleich erfordert [7, 25].

Die Probleme der *Datentransformation* betreffen die Produkt- und die Bestelltabelle und knüpfen an die Dimension der Konsistenz an. In der Produkttabelle werden die Preisangaben von ihrem numerischen Sollwert in uneinheitliche textuelle Schreibweisen überführt, etwa mit Dezimalkomma, vorangestelltem Euro-Zeichen oder nachgestelltem Währungskürzel, und die Verfügbarkeit wird von einem Wahrheitswert in wechselnde textuelle Entsprechungen umgeschrieben. Damit besitzt die einfache Transformationsaufgabe der Typkonvertierung eine unmittelbare Grundlage. Die mittlere Stufe erfordert keine eigene Verfälschung, sondern nutzt die vorhandene Struktur der Bestelltabelle, aus der sich der Gesamtbetrag sowie einzelne Datumsbestandteile ableiten lassen, und prüft so die zeilenweise Berechnungslogik. Die anspruchsvolle Stufe schließlich ergibt sich aus dem Zusammenspiel aller drei Tabellen: Sie verlangt deren Verknüpfung über die Schlüsselbeziehungen, eine begleitende Bereinigung der zuvor beschriebenen Mängel und eine abschließende Aggregation über Land und Produktkategorie. Diese Aufgabe bündelt damit bewusst mehrere der eingebauten Probleme in einem einzigen, mehrstufigen Verarbeitungsschritt, wie er für reale ETL-Strecken charakteristisch ist [1, 2].

Die Probleme der *Duplikaterkennung* zielen auf die Eindeutigkeit der Kundentabelle und werden durch das Hinzufügen zweier Arten von Doppelungen erzeugt. Für die einfache und die mittlere Stufe werden vollständig identische Datensätze eingefügt, die sowohl als exakte Duplikate zu erkennen sind als auch, über die mehrfach vergebene E-Mail-Adresse, die schlüsselbasierte Entdoppelung ermöglichen. Für die anspruchsvolle Stufe werden unscharfe Duplikate gebildet, bei denen derselbe Kunde mit geringfügig abweichendem Namen und abweichender E-Mail-Schreibweise erneut auftritt, etwa durch vertauschte Buchstaben, veränderte Groß- und Kleinschreibung, eingefügte Mittelinitialen oder zusätzliche Punkte im lokalen Teil der Adresse. Da die-

se Fuzzy-Duplikate nicht durch exakten Abgleich, sondern nur über eine Ähnlichkeitsbetrachtung auflösbar sind, bilden sie die in Abschnitt 4.3.3 beschriebene zweite semantische Achse der Untersuchung [7, 13]. Der Anteil der eingebrachten Probleme ist über alle Kategorien hinweg so bemessen, dass er einerseits merkliche Auswirkungen auf die Ergebniskennzahlen hat, andererseits den grundsätzlichen Charakter des Datensatzes als überwiegend sauberen Bestand nicht verfälscht; so betreffen die zusätzlichen exakten und unscharfen Duplikate jeweils rund vier Prozent der Kundendatensätze, während die fehlenden Länderangaben und die zusätzlichen Leerzeichen in vergleichbar begrenztem Umfang gesetzt werden.

Wie sich diese Beschreibung konkret in den Daten niederschlägt, veranschaulicht Tabelle 5.1 anhand tatsächlicher Zeilen des erzeugten Datensatzes. Sie stellt jedem verfälschten Rohwert den zugehörigen, aus der sauberen Fassung bekannten Sollwert gegenüber und macht damit zugleich sichtbar, weshalb die Aufgaben unterschiedlich schwer sind: Während sich Leerzeichen und Datumsformate durch eine endliche Menge syntaktischer Regeln vollständig auflösen lassen, ist bei den Länderangaben für jeden einzelnen Wert zu entscheiden, welches Land gemeint ist. Fehlende Werte werden nicht gelöscht, sondern einheitlich als UNKNOWN gekennzeichnet, sodass die Zeilenzahl erhalten bleibt und die Vollständigkeit im Sinne von Abschnitt 4.5.2 messbar bleibt.

Tabelle 5.1: Beispiele tatsächlich erzeugter Rohwerte und ihrer Sollwerte, gegliedert nach der zugehörigen Aufgabe. Sichtbare Leerzeichen sind mit _ gekennzeichnet.

Aufgabe	Rohwert	Sollwert
Leerzeichen	_ _Timm Haering-Faust_ _	Timm Haering-Faust
fehlender Wert	(leer)	UNKNOWN
Datumsformat	2025-06-27 (ISO)	2025-06-27
Datumsformat	05.11.2023 (deutsch)	2023-11-05
Datumsformat	December 22 2025 (US)	2025-12-22
Datumsformat	1639353600 (Unix)	2021-12-13
Länderangabe	Deutschland / Germany	DE
Länderangabe	Deutshcland (Tippfehler)	DE
Länderangabe	SUI / Schweiz	CH
Länderangabe	Holland / Niederlande	NL
Preisangabe	134.94 EUR	134.94
Preisangabe	€498.06	498.06
Preisangabe	323,37	323.37
Verfügbarkeit	yes / true / 1	True
Verfügbarkeit	no / false / 0	False

Die Duplikate lassen sich nicht sinnvoll als einzelne Werte darstellen, da ihr Kern gera-

de im Verhältnis zweier Datensätze zueinander liegt; Tabelle 5.2 stellt sie deshalb zeilenweise gegenüber. Bei den exakten Duplikaten ist der eingefügte Datensatz in allen Feldern einschließlich der Kundennummer identisch, sodass er sich durch einen unmittelbaren Abgleich entfernen lässt. Bei den unscharfen Duplikaten dagegen weichen Name und E-Mail-Adresse geringfügig voneinander ab, und zwar durch genau jene Muster, die auch in realen Beständen entstehen: veränderte Groß- und Kleinschreibung, ein eingefügtes zusätzliches Leerzeichen oder ein zusätzlicher Punkt im lokalen Teil der Adresse. Die inhaltliche Identität der Datensätze bleibt für einen Menschen unmittelbar erkennbar, ist über einen exakten Vergleich jedoch nicht mehr zugänglich, weshalb diese Aufgabe die in Abschnitt 4.3.3 beschriebene Ähnlichkeitsbetrachtung erzwingt.

Tabelle 5.2: Eingefügte Duplikate im Vergleich zu ihrem Ausgangsdatensatz. Bei den unscharfen Duplikaten sind die abweichenden Stellen die einzige Änderung gegenüber dem Original.

Art	Nr.	Name	E-Mail
exakt (Original)	145	Angelina Siering	waehnerbirgitt@example.net
exakt (Duplikat)	145	Angelina Siering	waehnerbirgitt@example.net
unscharf (Original)	308	Uta Lange	gkabus@example.org
unscharf (Duplikat)	505	Uta_ Lange	gka.bus@example.org
unscharf (Original)	493	Bernfried Harloff	gunter53@example.org
unscharf (Duplikat)	520	BERNFRIED HARLOFF	GUNTER53@example.org
unscharf (Original)	465	Norma Schmiedecke-Hofmann	elfriede16@example.org
unscharf (Duplikat)	510	norma schmiedecke-hofmann	ELFRIEDE16@example.org

Aus den 500 sauberen Kundendatensätzen entstehen durch das Anfügen von 20 exakten und 20 unscharfen Duplikaten insgesamt 540 Rohdatensätze, was die in Abschnitt 5.2 genannte Zielgröße von 500 eindeutigen Kunden als Sollzustand der Duplikataufgaben festlegt. Die Häufigkeitsverteilung der Länderangaben bestätigt zugleich die beabsichtigte Streuung: Für die zehn im Datensatz vertretenen Länder liegen je Land drei bis sechs verschiedene Schreibweisen vor, von Standardcodes wie DE über ausgeschriebene deutsche und englische Namen bis zu untypischen Kürzeln wie SUI für die Schweiz oder GER für Deutschland, und 45 Angaben fehlen vollständig. Genau diese Streuung ist es, an der sich in Kapitel 8 der Unterschied zwischen regelbasierter Zuordnung und sprachlichem Verständnis bemessen lässt.

5.4 Generierungsverfahren und verwendete Werkzeuge

Die Erzeugung des Datensatzes folgt einem zweistufigen Prinzip, das die in Abschnitt 5.1 geforderte Kontrollierbarkeit unmittelbar umsetzt. Im ersten Schritt werden für jede Tabelle vollständig saubere, strikt schemakonforme Daten erzeugt, die den fehlerfreien Sollzustand verkörpern. Im zweiten Schritt werden aus dieser sauberen Fassung durch eine kontrollierte Verfälschung die tatsächlich zu verarbeitenden Rohdaten abgeleitet, während die saubere Fassung selbst als Referenz erhalten bleibt. Dieses Vorgehen kehrt den üblichen Ablauf einer ETL-Strecke bewusst um: Nicht die Bereinigung wird nachträglich auf unbekannte Rohdaten angewandt, sondern die Verunreinigung wird gezielt auf bekannte saubere Daten aufgebracht, sodass die exakte Umkehrung jeder Verfälschung und damit die eindeutige Soll-Lösung jeder Aufgabe von vornherein feststeht [29, 38].

Für die Erzeugung realistisch wirkender Basiswerte wie Namen und E-Mail-Adressen wird eine etablierte Bibliothek zur Generierung synthetischer Testdaten eingesetzt, deren deutschsprachige Lokalisierung dem Szenario angemessene Werte liefert. Die kontrollierte Verfälschung, also das Einstreuen von Leerzeichen, fehlenden Werten, uneinheitlichen Formaten, verfälschten Länderangaben sowie exakten und unscharfen Duplikaten, erfolgt anschließend durch eigens implementierte Routinen, die jede der in Abschnitt 5.3 beschriebenen Problemklassen erzeugen. Sowohl die Basisdaten als auch die Verfälschung werden über einen zentralen Zufallszahlengenerator gesteuert, der mit einem festen Startwert initialisiert wird. Dadurch ist die Generierung vollständig deterministisch: Derselbe Startwert führt stets zu einem identischen Datensatz, während unterschiedliche Startwerte voneinander unabhängige, aber strukturell gleichartige Datensätze erzeugen. Diese Determiniertheit stellt sicher, dass die untersuchte Variabilität allein den Modellen und nicht der Datengrundlage zuzurechnen ist [6, 7]. Die erzeugten Rohdaten werden in einem gängigen tabellarischen Austauschformat abgelegt und dienen zusammen mit der separat gespeicherten sauberen Referenz als Eingabe für die in Kapitel 6 beschriebene Verarbeitungsumgebung.

5.5 Validierung des Datensatzes

Bevor der Datensatz als Grundlage der Experimente verwendet wird, ist zu prüfen, ob er die in Abschnitt 5.1 formulierten Anforderungen tatsächlich erfüllt. Die Validierung setzt zunächst an der sauberen Referenz an: Da diese als maßgebliche Ground Truth dient, wird ihre strikte Konformität mit dem definierten Zielschema überprüft, sodass

sichergestellt ist, dass die als korrekt geltenden Werte die geforderten Datentypen, Wertebereiche und Formate einhalten [11, 12]. Nur unter dieser Voraussetzung ist der spätere Abgleich der Modellergebnisse mit der Referenz aussagekräftig.

In einem zweiten Schritt wird geprüft, ob die eingebauten Qualitätsprobleme in der beabsichtigten Art und im vorgesehenen Umfang vorhanden sind. Kontrolliert wird insbesondere, dass jede der neun Aufgaben aus Abschnitt 4.3 auf einen tatsächlich vorliegenden Mangel trifft, dass die verschiedenen Datums- und Preisformate wie vorgesehen auftreten und dass sich die exakten und unscharfen Duplikate eindeutig auf ihre jeweiligen Ausgangsdatensätze zurückführen lassen. Damit ist zugleich gewährleistet, dass für jede Aufgabe eine eindeutige Soll-Lösung existiert, was die in Abschnitt 4.5 definierten Kriterien überhaupt erst berechenbar macht [10, 13]. Ergänzend wird die Reproduzierbarkeit der Generierung dadurch bestätigt, dass die wiederholte Ausführung mit identischem Startwert einen bitgenau identischen Datensatz liefert.

Mit dieser Validierung steht ein Datensatz zur Verfügung, dessen Sollzustand, Fehlerbild und Erzeugung vollständig bekannt und nachvollziehbar sind und der damit die kontrollierte Vergleichsgrundlage für die weitere Untersuchung bildet. Zugleich ist die dem Vorgehen innewohnende Einschränkung zu benennen: Ein synthetischer Datensatz kann die Vielfalt und die unvorhergesehenen Unregelmäßigkeiten realer Datenbestände nur annähern, weshalb die aus ihm gewonnenen Ergebnisse unter diesem Vorbehalt zu interpretieren und im Rahmen der Limitationen in Kapitel 10 erneut aufzugreifen sind [29, 31]. Auf der so geschaffenen Grundlage kann im folgenden Kapitel 6 die technische Umsetzung der Verarbeitungsumgebung beschrieben werden, welche die hier erzeugten Daten sowohl der klassischen Pipeline als auch den LLM-gestützten Verfahren zuführt.

6. Implementierung

Das vorangegangene Kapitel 4 hat das Untersuchungsdesign entworfen und der Datensatz aus Kapitel 5 stellt die unveränderliche Datengrundlage bereit. Dieses Kapitel beschreibt, wie dieses Design in ein lauffähiges Softwaresystem überführt wurde, mit dem sich die vollständige Matrix aus Modell, Prompting-Strategie und ETL-Aufgabe automatisiert durchführen und einheitlich bewerten lässt. Zunächst wird die modulare Systemarchitektur und die Versuchsumgebung dargestellt (Abschnitt 6.1), gefolgt von den eingesetzten Technologien und Bibliotheken (Abschnitt 6.2). Anschließend wer-

den die Umsetzung der klassischen Vergleichspipeline (Abschnitt 6.3), die Integration der LLMs über eine einheitliche Schnittstelle (Abschnitt 6.4) sowie die konkrete Realisierung der Prompting-Strategien (Abschnitt 6.5) beschrieben. Den Abschluss bildet das automatisierte Mess- und Auswertungsverfahren (Abschnitt 6.6), das die in Abschnitt 4.5 definierten Kriterien technisch operationalisiert und damit die Grundlage für die Durchführung in Kapitel 7 schafft.

6.1 Systemarchitektur und Versuchsumgebung

Das System ist als modulares Python-Projekt aufgebaut, dessen Komponenten entlang der einzelnen Verarbeitungsschritte getrennt sind und über klar umrissene Schnittstellen zusammenwirken. Diese Trennung folgt dem Ziel, die einzelnen Faktoren des Versuchsdesigns unabhängig voneinander austauschen zu können, ohne den übrigen Ablauf zu verändern, und erleichtert zugleich die Nachvollziehbarkeit der Ergebnisse [4, 10]. Ein Datenmodul kapselt die Erzeugung des synthetischen Datensatzes, die Referenzlösungen sowie die Definition der neun Aufgaben; ein Modellmodul stellt die Anbindung der LLMs bereit; ein Modul für die klassische Pipeline enthält die regelbasierte Vergleichslösung; ein Experimentmodul steuert den Ablauf und die Ausführung des erzeugten Codes; ein Evaluationsmodul bewertet die Ergebnisse; und ein Auswertungsmodul bereitet die erhobenen Messdaten für die Analyse auf.

Der grundlegende Datenfluss folgt für jede Konfiguration demselben Muster: Ausgehend vom synthetischen Datensatz und der Aufgabenbeschreibung wird ein Prompt erzeugt, an das jeweilige Modell übergeben und dessen Antwort weiterverarbeitet. Entsprechend der in Abschnitt 4.1 getroffenen Gestaltungsentscheidung erzeugt das Modell dabei nicht das Ergebnis unmittelbar, sondern ein ausführbares Verarbeitungsskript, das anschließend deterministisch auf den Daten ausgeführt wird. Das so entstandene Ergebnis wird als tabellarische Ausgabedatei gespeichert, gegen die bekannte Soll-Lösung geprüft und gemeinsam mit allen erhobenen Kennzahlen in einem unveränderlichen Ergebnisdatensatz protokolliert [1, 6]. Sämtliche steuerbaren Randbedingungen sind in einer zentralen Konfiguration gebündelt, die unter anderem den Zufallskern für die Datenerzeugung, die konstant niedrig gehaltene Temperatur, die Obergrenze der Ausgabe-Token sowie die Anzahl der Wiederholungen je Konfiguration festlegt. Diese zentrale und explizit hinterlegte Parametrisierung ist eine wesentliche Voraussetzung für die in Abschnitt 4.5 als querschnittliche Dimension eingeführte Reproduzierbarkeit, da sie die Bedingungen jedes Durchlaufs eindeutig dokumentiert [6, 7]. Die Ausführung erfolgt durchgängig in einer kontrollierten lokalen Umgebung.

6.2 Eingesetzte Technologien und Bibliotheken

Als Implementierungssprache dient Python, da es im Umfeld der Datenverarbeitung und des maschinellen Lernens die verbreitetste Sprache ist und mit einem umfangreichen Ökosystem einschlägiger Bibliotheken sowie offiziellen Schnittstellen aller untersuchten Modellanbieter unterstützt wird [17, 39]. Die eigentliche Datenverarbeitung stützt sich sowohl in der klassischen Pipeline als auch im vom Modell erzeugten Code auf die Bibliothek Pandas, die sich als De-facto-Standard für tabellarische Verarbeitung etabliert hat. Ihre einheitliche Verwendung stellt sicher, dass die regelbasierte und die KI-gestützte Lösung auf derselben technischen Grundlage arbeiten und damit unmittelbar vergleichbar bleiben [1, 4]. Sämtliche Zwischen- und Ergebnistabellen werden im spaltenorientierten Parquet-Format ausgetauscht, da dieses die Datentypen der Spalten mitführt und so einen typerhaltenden Abgleich zwischen erzeugter Ausgabe und Soll-Lösung ermöglicht, wie er für das Konsistenzkriterium aus Abschnitt 4.5.3 benötigt wird.

Die Anbindung der Modelle erfolgt über die jeweils offiziellen Softwarebibliotheken der Anbieter für die geschlossenen Modelle von OpenAI, Anthropic und Google sowie über eine lokale Laufzeitumgebung für das offen verfügbare Modell der Llama-Reihe. Diese heterogenen Schnittstellen werden, wie in Abschnitt 6.4 beschrieben, hinter einer einheitlichen Abstraktion zusammengeführt, sodass der Versuchsablauf unabhängig vom konkreten Anbieter bleibt [6, 20]. Ergänzend kommen etablierte Hilfsbibliotheken zum Einsatz: eine typsichere Konfigurationsverwaltung zur Bündelung der Versuchsparameter, eine Auszeichnungssprache zur versionierten Ablage der Prompt-Vorlagen sowie Werkzeuge zur Visualisierung und interaktiven Auswertung, mit denen die in Kapitel 8 dargestellten Tabellen und Abbildungen erzeugt werden. Als repräsentative Vertreter der jeweiligen Anbieter wird je ein aktuelles Modell eingesetzt, dessen exakte Versionskennung bei jedem Aufruf protokolliert wird, um der raschen Weiterentwicklung des Modellfeldes Rechnung zu tragen und die spätere Nachvollziehbarkeit zu sichern [6, 20].

6.3 Umsetzung der klassischen ETL-Pipeline

Die in Abschnitt 4.6 als Vergleichsmaßstab definierte klassische Pipeline ist als regelbasierte Lösung umgesetzt, die dieselben neun Aufgaben ausschließlich mit fest vorgegebener Verarbeitungslogik und ohne Zugriff auf ein Sprachmodell löst. Für die klar formalisierbaren Aufgaben, etwa das Entfernen exakter Duplikate oder die Typkonver-

tierung, nutzt sie dieselben deterministischen Routinen, die auch der Erzeugung der Soll-Lösung zugrunde liegen; für diese Aufgaben ist die explizit formulierte Regel zugleich die korrekte Lösung, sodass hier definitionsgemäß eine hohe Ergebnisqualität zu erwarten ist [3, 4].

Von besonderer Bedeutung für die Aussagekraft des Vergleichs ist die bewusste Gestaltung der Pipeline bei den semantisch geprägten Aufgaben. Für die Vereinheitlichung der Länderangaben greift sie auf eine absichtlich begrenzte Zuordnungstabelle zurück, die zwar ausgeschriebene Ländernamen und Standardcodes abdeckt, jedoch keine Tippfehler, untypischen Abkürzungen oder umgangssprachlichen Bezeichnungen berücksichtigt; nicht auflösbare Werte werden einheitlich als unbekannt gekennzeichnet. Für die Erkennung unscharfer Duplikate bildet sie einen normalisierten Schlüssel aus Name und E-Mail-Bestandteil und entfernt darüber Mehrfachnennungen, kann damit aber nur solche Abweichungen erfassen, die sich durch einfache Normalisierung angleichen lassen. Diese Heuristiken sind realistisch für ein rein regelbasiertes Vorgehen, stoßen jedoch genau dort an ihre Grenzen, wo ein inhaltliches Verständnis der Werte erforderlich wäre. Gerade an dieser Stelle wird der in Hypothese H1 erwartete Unterschied zwischen regelbasierter und KI-gestützter Verarbeitung messbar [7, 13]. Da die Pipeline vollständig deterministisch arbeitet und identische Eingaben stets zu identischen Ausgaben führen, dient sie im weiteren Verlauf als stabiler und nachvollziehbarer Referenzpunkt, gegen den sich die Ergebnisse der Modelle einordnen lassen.

6.4 Integration der LLMs in die Pipeline

Um die in Abschnitt 4.2 ausgewählten Modelle trotz ihrer unterschiedlichen Schnittstellen einheitlich ansprechen zu können, werden sie hinter einer gemeinsamen Abstraktion gekapselt. Jeder Anbieter implementiert dieselbe abstrakte Schnittstelle, die einen Aufruf entgegennimmt und eine standardisierte Antwort zurückliefert, in der neben dem erzeugten Text auch die exakte Modellkennung, der Verbrauch an Eingabe- und Ausgabe-Token sowie die Antwortzeit enthalten sind. Eine vorgeschaltete Erzeugungsinstanz liefert anhand eines Anbieternamens die passende Implementierung, sodass der Versuchsablauf vollständig unabhängig vom konkreten Modell bleibt und die Faktorachse Modell ohne Eingriffe in die übrige Logik variiert werden kann [6, 20]. Diese standardisierte Antwort ist zugleich die Datenbasis für die spätere Effizienzbewertung nach Abschnitt 4.5.4, da sie die dafür benötigten Kennzahlen bei jedem Aufruf unmittelbar mitführt.

Der eigentliche Verarbeitungsschritt folgt dem Ansatz der Code-Generierung: Aus der Modellantwort wird der enthaltene Python-Code extrahiert und anschließend in einem eigenen Betriebssystemprozess mit fester Zeitbegrenzung ausgeführt. Ein Durchlauf gilt nur dann als erfolgreich, wenn der Prozess fehlerfrei endet und tatsächlich die erwartete Ausgabedatei erzeugt wurde; andernfalls wird das Ergebnis als fehlgeschlagene Ausführung protokolliert. Da an dieser Stelle von einem Modell erzeugter Code ausgeführt wird, ist die Ausführung in einen separaten Prozess mit Zeitlimit ausgelagert. Dieses Vorgehen ist für die kontrollierte, lokale Durchführung der Experimente vertretbar, stellt jedoch bewusst keine vollständige Sicherheitsisolation dar und wäre für einen unbeaufsichtigten Produktivbetrieb entsprechend abzusichern [7, 25].

Ein zentrales Element der Integration ist die Behandlung von Fehlern, die aufgrund der probabilistischen Natur der Modelle und der Nutzung externer Dienste unvermeidlich auftreten. Dabei wird bewusst zwischen zwei Arten von Fehlern unterschieden: Vorübergehende Störungen des Dienstes, die sich etwa in bestimmten Statuscodes oder Verbindungsabbrüchen äußern, werden als transient eingestuft und der betreffende Aufruf mit exponentiell wachsender Wartezeit mehrfach wiederholt. Ein inhaltlich fehlerhaftes, aber technisch wohlgeformtes Ergebnis, etwa ein Skript, das mit einem Programmfehler abbricht, wird hingegen nicht wiederholt, da es einen gültigen Messwert über die Fähigkeit des Modells darstellt und eine Wiederholung das Ergebnis verfälschen würde [6, 7]. Ergänzend werden anbieterspezifische Besonderheiten aktueller Modelle abgefangen: Lehnt ein Modell einen Sampling-Parameter wie die Temperatur ab, wird dieser automatisch weggelassen und die Anpassung im Ergebnisdatensatz vermerkt, sodass solche Abweichungen bei der Interpretation der Reproduzierbarkeit sichtbar bleiben. Für jeden Aufruf werden schließlich sämtliche relevanten Angaben, von der Modellkennung über den Token-Verbrauch und die Kosten bis hin zum erzeugten Code und dem Ausführungsergebnis, vollständig protokolliert und bilden so die lückenlose Grundlage für die anschließende Auswertung.

Der in Abschnitt 4.1 eingeführte zweite Ausführungsmodus, die direkte In-Context-Verarbeitung, ist über dieselbe einheitliche Modellschnittstelle umgesetzt, unterscheidet sich jedoch im Ablauf. Hier werden die Eingabedaten unmittelbar in den Prompt eingebettet und das Modell angewiesen, die transformierte Ergebnistabelle direkt in einem klar abgegrenzten Format zurückzugeben, das anschließend eingelesen und gegen die Soll-Lösung geprüft wird. Die Auswertung der Antwort ist dabei bewusst robust gegenüber am Ausgabelimit abgeschnittenen Antworten gestaltet, sodass auch unvollständige Ergebnisse noch verarbeitet und als solche erkannt werden. Da aktuelle

Modelle standardmäßig einen internen Denkprozess ausführen, der das für die Ergebnistabelle verfügbare Ausgabebudget aufzehren würde, bevor die eigentlichen Daten erzeugt werden, wird dieses Verhalten für den Direkt-Modus deaktiviert und das Ausgabelimit angehoben [6, 24]. Anders als bei der wiederholten Code-Erzeugung wird jede Konfiguration in diesem Modus nur einmal ausgeführt, um den durch die eingebetteten Daten erhöhten Token-Verbrauch zu begrenzen; eine Wiederholung erfolgt ausschließlich bei einem technischen Fehlschlag wie einem transienten Anbieterfehler oder einer leeren beziehungsweise nicht auswertbaren Antwort, nicht hingegen bei einem wohlgeformten, aber inhaltlich abweichenden Ergebnis, das einen gültigen Messwert darstellt.

6.5 Umsetzung der Prompting-Strategien

Die in Abschnitt 4.4 konzipierten Strategien sind als versionierte Vorlagen umgesetzt, die getrennt vom Programmcode in einem eigenen, menschenlesbaren Format abgelegt sind. Jede Vorlage besteht aus einer eindeutigen Kennung, einer Systemanweisung und einer Nutzervorlage mit klar benannten Platzhaltern, in die zur Laufzeit die Aufgabenbeschreibung, die Ein- und Ausgabepfade sowie bei Bedarf das erwartete Zielschema eingesetzt werden. Diese versionierte und vom Code entkoppelte Ablage stellt sicher, dass jede eingesetzte Prompt-Variante eindeutig dokumentiert, unverändert wiederverwendbar und damit nachvollziehbar bleibt, was eine wesentliche Voraussetzung für die Vergleichbarkeit und Reproduzierbarkeit der Ergebnisse ist [23, 24].

Alle vier Strategien folgen einem einheitlichen Grundaufbau und unterscheiden sich ausschließlich im Umfang der zusätzlich bereitgestellten Hilfestellung, sodass beobachtete Qualitätsunterschiede eindeutig auf die jeweilige Strategie und nicht auf abweichende Formulierungen zurückführbar sind. Die Zero-Shot-Vorlage beschränkt sich auf die reine Aufgabenanweisung und bildet damit die Referenzbasis. Die Few-Shot-Vorlage ergänzt ein vollständig ausformuliertes Ein- und Ausgabepaar; dieses Beispiel bezieht sich bewusst auf eine andersartige Aufgabe auf einer separaten Beispieltabelle und nicht auf eine der neun evaluierten Aufgaben, damit es keine Musterlösung vorwegnimmt, sondern lediglich Stil und Format der erwarteten Lösung vermittelt [19, 23]. Die Chain-of-Thought-Vorlage leitet das Modell an, seine Lösung in nachvollziehbaren Zwischenschritten zu entwickeln, während die schema-angereicherte Vorlage der Anweisung eine explizite Beschreibung des Zielschemas mit Spaltennamen und Datentypen beifügt, um die strukturelle Korrektheit der Ausgabe gezielt zu unterstützen [22, 23]. Da das Zielschema nur von der schema-angereicherten Variante tatsächlich

genutzt wird, während die übrigen Platzhalter für alle Strategien identisch belegt sind, bleibt der Vergleich über die Strategien hinweg fair und kontrolliert.

6.6 Mess- und Auswertungsverfahren

Die in Abschnitt 4.5 definierten Kriterien werden durch eine automatisierte Bewertungsroutine operationalisiert, die die erzeugte Ausgabedatei mit der bekannten Soll-Lösung abgleicht. Um zu berücksichtigen, dass programmatisch erzeugte Ergebnisse korrekte Werte häufig in abweichender Zeilenreihenfolge liefern, werden beide Tabellen vor dem Abgleich einheitlich sortiert und damit reihenfolgeunabhängig verglichen. Die Genauigkeit wird als Anteil der zellweise übereinstimmenden Werte bestimmt, die Vollständigkeit aus dem Vorhandensein der erwarteten Spalten und der korrekten Datensatzanzahl abgeleitet und die Konsistenz über die Übereinstimmung der Datentypen mit dem Zielschema erfasst [10, 12]. Ein zellweiser Abgleich setzt dabei voraus, dass Spaltenmenge und Datensatzanzahl übereinstimmen; ist diese strukturelle Grundlage nicht gegeben, wird dies gesondert ausgewiesen, statt eine inhaltliche Genauigkeit vorzutäuschen, die mangels vergleichbarer Struktur nicht sinnvoll bestimmbar wäre. Die Effizienzkennzahlen aus Zeit, Token-Verbrauch und Kosten stammen unmittelbar aus der standardisierten Modellantwort und werden ergänzend zur inhaltlichen Bewertung erfasst [20, 24].

Jeder einzelne Durchlauf wird als eigenständiger, unveränderlicher Ergebnisdatensatz abgelegt, der die vollständige Konfiguration, die erhobenen Qualitäts- und Effizienzkennzahlen, die exakte Modellkennung sowie den erzeugten Code umfasst. Diese lückenlose Protokollierung erlaubt es, jede spätere Auswertung ohne erneute Modellaufrufe aus den gespeicherten Datensätzen nachzuvollziehen, und bildet zugleich die Grundlage für die Erhebung der Reproduzierbarkeit, die sich aus der Streuung der Kennzahlen über die wiederholten Ausführungen einer Konfiguration ergibt [6, 7]. Eine gesonderte Auswertungsschicht führt die einzelnen Datensätze zusammen, wählt je Konfiguration den jeweils jüngsten Lauf aus und klassifiziert die Ergebnisse nach ihrem Zustand, um technische Fehlschläge von inhaltlichen Abweichungen zu trennen. Auf dieser Grundlage werden die in Kapitel 8 dargestellten Vergleiche, Übersichten und detaillierten Abweichungsanalysen erstellt. Damit ist die technische Umsetzung des Untersuchungsdesigns abgeschlossen, und die tatsächliche Durchführung der Experimente sowie die dabei erhobenen Messdaten sind Gegenstand des folgenden Kapitels 7.

7. Durchführung der Experimente

Nachdem Kapitel 6 die technische Umsetzung des Untersuchungssystems beschrieben hat, dokumentiert dieses Kapitel die eigentliche Durchführung der Experimente. Es legt offen, in welchem Ablauf die einzelnen Läufe erfolgten (Abschnitt 7.1), welche Messdaten dabei in welcher Form erhoben wurden (Abschnitt 7.2) und unter welchen Bedingungen die Durchführung stattfand, um die Nachvollziehbarkeit und Reproduzierbarkeit der Ergebnisse sicherzustellen (Abschnitt 7.3). Damit schafft es die Grundlage für die anschließende Auswertung in Kapitel 8.

7.1 Versuchsablauf

Ausgangspunkt jeder Durchführung ist der in Kapitel 5 beschriebene synthetische Datensatz, der unverändert für alle Läufe verwendet wird, sowie die zu jeder Aufgabe vorab bestimmte Soll-Lösung. Auf dieser gemeinsamen Grundlage werden die Experimente vollständig automatisiert über den in Abschnitt 6.1 dargestellten Experiment-Runner ausgeführt, sodass jede Konfiguration unter identischen Bedingungen und ohne manuelle Eingriffe durchläuft. Der Ablauf durchmustert die vollständige Faktormatrix systematisch, indem er für jede Kombination aus Modell, Prompting-Strategie und Aufgabe nacheinander die vorgesehenen Wiederholungen ausführt und das Ergebnis unmittelbar festhält [6, 10].

Entsprechend den beiden in Abschnitt 4.1 eingeführten Ausführungsmodi gliedert sich die Durchführung in mehrere Teilläufe. Im primären Modus der Code-Generierung wird für jede Konfiguration ein Verarbeitungsskript erzeugt, ausgeführt und bewertet; daraus ergeben sich bei vier Modellen, vier Prompting-Strategien, neun Aufgaben und je drei Wiederholungen insgesamt 432 Einzelläufe. Im ergänzenden Direkt-Modus wird jede der neun Aufgaben von jedem der vier Modelle einmalig unmittelbar auf den eingebetteten Daten gelöst, woraus sich 36 weitere Läufe ergeben. Zusätzlich löst die regelbasierte Vergleichspipeline dieselben neun Aufgaben in je einem deterministischen Durchlauf. Jeder einzelne Lauf wird direkt nach seiner Ausführung als eigenständiger Ergebnisdatensatz gespeichert, sodass der Gesamttablauf jederzeit unterbrochen und fortgesetzt werden kann, ohne dass bereits erhobene Ergebnisse verloren gehen oder überschrieben werden. Die auf den externen Modellzugriff zurückzuführenden, vorübergehenden Störungen werden dabei, wie in Abschnitt 6.4 beschrieben, durch eine automatische Wiederholung transienter Fehlaufufe abgefangen, während inhalt-

lich fehlerhafte, aber technisch vollständige Ergebnisse bewusst als gültige Messwerte erhalten bleiben [6, 7].

Auf die Faktormatrix folgt die in Abschnitt 4.1 eingeführte verkettete Ausführung, die dieselben Aufgaben als durchgängige Strecke aus neun aufeinander aufbauenden Schritten durchläuft. Sie wird je Modell sowie für die regelbasierte Pipeline einmal ausgeführt und nach jedem Schritt unmittelbar gegen die zugehörige Soll-Zwischentabelle geprüft, sodass sich der Qualitätsverlauf über die Kette hinweg verfolgen lässt. Da ein Schritt seine Nachfolger mit Daten versorgt, muss er strukturell verwertbar sein: Bricht der erzeugte Code ab oder verletzt seine Ausgabe das erwartete Schema, wird der Versuch bis zu dreimal wiederholt und die Kette andernfalls an dieser Stelle abgebrochen, wobei die abhängigen Schritte als blockiert protokolliert werden. Diese Wiederholung dient allein der Herstellung einer ausführbaren Kette und ist nicht mit der zur Reproduzierbarkeit erhobenen Mehrfachausführung zu verwechseln, die weiterhin ausschließlich in der Faktormatrix stattfindet. Ergänzend wird die Kette in einem zweiten, sonst identischen Durchlauf so ausgeführt, dass jeder Schritt die Soll-Ausgabe seines Vorgängers erhält; beide Durchläufe zusammen erlauben in Kapitel 8 die Trennung von eigener Schrittleistung und geerbtem Fehler. Aus den 432 Läufen der Code-Generierung, den 36 Läufen des Direkt-Modus, den neun Durchläufen der regelbasierten Pipeline sowie den verketteten und isolierten Kettendurchläufen ergibt sich damit die vollständige Datenbasis der Auswertung.

7.2 Erhobene Messdaten

Für jeden Lauf wird ein umfassender Ergebnisdatensatz erhoben, der eine spätere Auswertung ohne erneute Modellaufrufe ermöglicht. Er umfasst zum einen die vollständige Konfiguration des Laufs, also das eingesetzte Modell mit seiner exakten Versionskennung, die verwendete Prompting-Strategie, die bearbeitete Aufgabe mit Kategorie und Schwierigkeitsgrad, den zugrunde liegenden Zufallskern, die Nummer der Wiederholung sowie die gewählte Temperatur. Zum anderen enthält er die Ergebnisse der in Abschnitt 4.5 definierten Bewertung, namentlich die Genauigkeit, die Vollständigkeit und die Konsistenz sowie die zugehörigen strukturellen Kennzeichen wie die Übereinstimmung von Schema und Datensatzanzahl, anhand derer die grundsätzliche Vergleichbarkeit eines Ergebnisses beurteilt wird [10, 12].

Ergänzend zu den Qualitätskennzahlen werden für jeden Lauf die Effizienzgrößen aus Abschnitt 4.5.4 festgehalten, also der Verbrauch an Eingabe- und Ausgabe-Token, die

daraus abgeleiteten Nutzungskosten sowie die Antwortzeit des Modells und, im Modus der Code-Generierung, die Dauer und der Erfolg der anschließenden Ausführung [20, 24]. Um über die reinen Kennzahlen hinaus auch die konkrete Lösung nachvollziehbar zu machen, wird zusätzlich das jeweils erzeugte Artefakt gesichert, im Code-Generierungs-Modus der vollständige erzeugte Quelltext und im Direkt-Modus die unmittelbar zurückgegebene Ergebnistabelle. Sämtliche Läufe werden in einem einheitlichen, maschinenlesbaren Format als jeweils unveränderlicher Datensatz abgelegt, während die erzeugten Ergebnistabellen im typerhaltenden Parquet-Format gespeichert werden. Für die verkettete Ausführung wird zusätzlich je Schritt festgehalten, welche Eingaben er erhalten hat, wie viele Versuche bis zu einer verwertbaren Ausgabe erforderlich waren und wie lange Modellaufruf und Ausführung jeweils gedauert haben, sodass sich neben dem Qualitätsverlauf auch der Aufwand der gesamten Strecke beziffern lässt. Insgesamt entsteht so eine geschlossene Datenbasis, auf der die gesamte Auswertung in Kapitel 8 aufsetzt.

7.3 Reproduzierbarkeit und Versuchsbedingungen

Die Reproduzierbarkeit wurde bereits im Untersuchungsdesign als querschnittliche Dimension verankert (Abschnitt 4.5) und schlägt sich unmittelbar in den Versuchsbedingungen nieder. Um vergleichbare Bedingungen zu schaffen, werden alle steuerbaren Einflussgrößen über sämtliche Läufe hinweg konstant gehalten: Der Datensatz wird über einen festen Zufallskern reproduzierbar erzeugt und unverändert verwendet, und die Temperatur wird durchgehend auf einen niedrigen Wert festgelegt, um die nicht durch das Modell selbst bedingte Variabilität zu minimieren [6, 23]. Da sich das Feld der Modelle mit hoher Geschwindigkeit weiterentwickelt und die genaue Modellfassung bei den kommerziell betriebenen Anbietern nicht selbst kontrolliert werden kann, wird die exakte Versionskennung jedes Modells bei jedem Aufruf protokolliert; im Rahmen dieser Arbeit kamen je ein aktueller Vertreter von Anthropic, OpenAI und Google sowie ein lokal ausgeführtes Modell der Llama-Reihe zum Einsatz [6, 20].

Um die probabilistische Variabilität der Modelle nicht nur zu kontrollieren, sondern selbst messbar zu machen, wird im primären Ausführungsmodus jede Konfiguration dreifach ausgeführt, sodass sich die Streuung der Ergebnisse über die Wiederholungen als Maß für die Stabilität auswerten lässt und die in Abschnitt 4.1 als querschnittliche Dimension zu H1 und H2 eingeführte Reproduzierbarkeit überprüfbar wird [6, 7]. Der ergänzende Direkt-Modus wird demgegenüber bewusst nur einfach ausgeführt, um den durch die eingebetteten Daten deutlich erhöhten Token-Verbrauch zu

begrenzen; die dort erhobenen Werte sind entsprechend als einzelne Beobachtung und nicht als über Wiederholungen gemittelte Größe zu verstehen. Zu berücksichtigen ist ferner, dass einige aktuelle Modelle bestimmte Sampling-Parameter nicht mehr akzeptieren oder standardmäßig einen internen Denkprozess ausführen; solche anbieterspezifischen Abweichungen werden, wie in Abschnitt 6.4 beschrieben, abgefangen und im Ergebnisdatensatz vermerkt, sodass sie bei der Interpretation sichtbar bleiben. Das lokal betriebene Modell wurde auf allgemein verfügbarer Endgeräte-Hardware mit begrenztem Grafikspeicher ausgeführt, was insbesondere im Direkt-Modus eine Begrenzung des nutzbaren Kontextfensters erforderte und die praktischen Grenzen eines lokalen Betriebs sichtbar macht. Trotz dieser Vorkehrungen bleibt eine vollständige, bitgenaue Reproduktion aufgrund der probabilistischen Natur der Modelle und möglicher serverseitiger Änderungen der kommerziellen Modelle nicht garantierbar; die protokollierten Versionskennungen, die konstant gehaltenen Bedingungen und die vollständige Ablage aller Läufe stellen jedoch eine im Rahmen des Möglichen weitgehende Nachvollziehbarkeit sicher [6, 7].

8. Ergebnisse und Auswertung

Die in Kapitel 7 beschriebene Durchführung hat eine geschlossene Datenbasis erzeugt, deren Auswertung Gegenstand dieses Kapitels ist. Es führt die Ergebnisse entlang der Struktur des Untersuchungsdesigns aus Kapitel 4 zusammen und bereitet damit die Beantwortung der beiden Forschungsfragen vor. Zunächst werden die Ergebnisse je ETL-Aufgabe und Aufgabenkategorie dargestellt (Abschnitt 8.1), anschließend die Modelle (Abschnitt 8.2) und die Prompting-Strategien (Abschnitt 8.3) gegeneinander verglichen. Darauf folgen die Ergebnisse der verketteten Pipeline mit der Trennung von eigener Schrittleistung und Fehlerfortpflanzung (Abschnitt 8.4) sowie die Auswertung der querschnittlichen Reproduzierbarkeitsdimension (Abschnitt 8.5). Abschnitt 8.6 prüft sämtliche dieser Befunde anschließend auf einem zweiten, während der Entwicklung unbekannten Datensatz nach und beziffert dort zugleich Kosten, Token-Verbrauch und Laufzeit beider Ausführungsmodi. Abschnitt 8.7 stellt die KI-gestützten Verfahren der regelbasierten Pipeline gegenüber, bevor Abschnitt 8.8 die Befunde bündelt, an den Hypothesen H1 und H2 aus Abschnitt 4.1 spiegelt und in den Stand der Forschung einordnet. Die daraus abzuleitenden Empfehlungen sind Gegenstand des folgenden Kapitels 9.

Sämtliche im Folgenden berichteten Genauigkeitswerte beziehen sich auf die in Ab-

schnitt 4.5.1 definierte Genauigkeit in ihrer abgeglichenen Ausprägung: Stimmt das Zielschema, wird auch bei abweichender Datensatzanzahl der Anteil korrekt reproduzierter Werte gemessen, statt das Ergebnis pauschal als unbrauchbar zu verwerfen. Fehlgeschlagene Ausführungen und strukturell nicht vergleichbare Ausgaben gehen mit dem Wert null in die Mittelwerte ein, da sie im praktischen Einsatz ebenfalls kein verwertbares Ergebnis liefern. Ergänzend wird als Erfolgsquote der Anteil derjenigen Läufe ausgewiesen, die überhaupt eine strukturell verwertbare Ausgabe erzeugt haben; die Differenz beider Kennzahlen trennt das Scheitern an der Ausführung von der inhaltlichen Abweichung eines lauffähigen Ergebnisses.

8.1 Ergebnisse je ETL-Aufgabe

Über alle 432 Läufe des Code-Generierungs-Modus hinweg erreichen die vier Modelle eine mittlere Genauigkeit von 0,573 bei einer Erfolgsquote von 62,5 %. Von den 432 Läufen führten 270 zu einer verwertbaren Ausgabe, 134 brachen mit einem Programmfehler ab und 28 lieferten eine lauffähige, aber strukturell abweichende Ausgabe. Der gesamte Aufwand belief sich auf 6,57 US-Dollar bei rund 194.000 Eingabe- und 583.000 Ausgabe-Token sowie einer mittleren Antwortzeit von 11,2 Sekunden je Aufruf. Diese Mittelwerte verdecken jedoch eine erhebliche Spreizung, die erst in der Aufschlüsselung nach Aufgaben sichtbar wird und die zentrale Beobachtung dieser Arbeit bereits vorwegnimmt: Die Ergebnisqualität variiert zwischen den Aufgaben deutlich stärker als zwischen den Modellen.

Abbildung 8.1 zeigt diese Spreizung im Gesamtüberblick. Aufgetragen sind alle neun Aufgaben gegen die vier Modelle in beiden Ausführungsmodi sowie die regelbasierte Vergleichspipeline, wobei für den Modus der Code-Generierung jeweils die beste der klassischen Prompting-Strategien ausgewiesen ist. Drei Muster treten unmittelbar hervor: Die formal beschreibbaren Aufgaben in der oberen Bildhälfte werden von allen Cloud-Modellen nahezu fehlerfrei gelöst; die semantisch geprägten Aufgaben zeigen ein uneinheitliches, zwischen den Modellen stark schwankendes Bild; und die zusammengesetzte Aufgabe aus Join, Bereinigung und Aggregation scheitert im Modus der Code-Generierung durchgängig, während sie im Direkt-Modus immerhin rund die Hälfte der Werte trifft.

Tabelle 8.1 beziffert dieses Bild je Aufgabe. Ausgewiesen sind die über alle Modelle, Strategien und Wiederholungen gemittelte Genauigkeit, die zugehörige Erfolgsquote sowie zum Vergleich das Ergebnis der regelbasierten Pipeline. Die Reihenfolge folgt

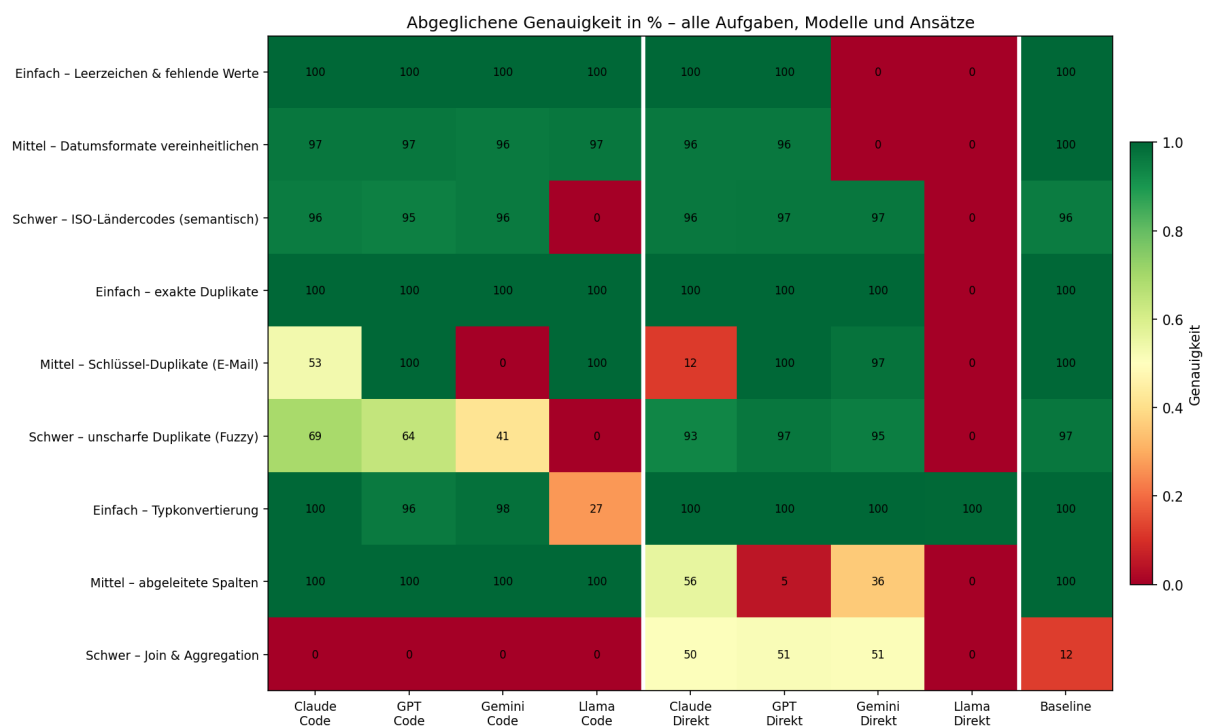


Abbildung 8.1: Abgegliche Genauigkeit in Prozent über alle neun Aufgaben, vier Modelle und beide Ausführungsmodi sowie die regelbasierte Pipeline. Für die Code-Generierung ist jeweils die beste der drei klassischen Prompting-Strategien ausgewiesen.

den drei Schwierigkeitsstufen aus Abschnitt 4.3, sodass sich der Einfluss der Aufgabenkomplexität unmittelbar ablesen lässt.

Tabelle 8.1: Mittlere Genauigkeit und Erfolgsquote je Aufgabe über alle Modelle, Strategien und Wiederholungen des Code-Generierungs-Modus, gegenübergestellt der regelbasierten Pipeline.

Kategorie	Aufgabe	Genauigkeit	Erfolg	Baseline
Bereinigung	Einfach – Leerzeichen und fehlende Werte	81,2 %	81,2 %	100,0 %
Bereinigung	Mittel – Datumsformate	85,6 %	89,6 %	100,0 %
Bereinigung	Schwer – ISO-Ländercodes	61,7 %	66,7 %	95,5 %
Duplikate	Einfach – exakte Duplikate	77,1 %	77,1 %	100,0 %
Duplikate	Mittel – Schlüssel-Duplikate	41,6 %	41,7 %	100,0 %
Duplikate	Schwer – unscharfe Duplikate	19,1 %	54,2 %	96,9 %
Transformation	Einfach – Typkonvertierung	68,0 %	70,8 %	100,0 %
Transformation	Mittel – abgeleitete Spalten	81,2 %	81,2 %	100,0 %
Transformation	Schwer – Join und Aggregation	0,0 %	0,0 %	12,1 %

Bemerkenswert ist zunächst, dass selbst die einfachsten Aufgaben im Mittel deutlich unter der Marke von 100 Prozent bleiben, obwohl jedes einzelne Modell sie mit gut gewähltem Prompt fehlerfrei löst. Diese Differenz geht überwiegend auf das lokal betriebene Modell und auf die schema-angereicherte Prompting-Variante zurück, deren Wirkung in den Abschnitten 8.2 und 8.3 gesondert betrachtet wird. Die Mittelwerte in Tabelle 8.1 sind daher nicht als Aussage über die erreichbare Qualität zu lesen, sondern als Aussage über die Verlässlichkeit einer beliebig herausgegriffenen Konfiguration: Sie messen, was zu erwarten ist, wenn Modell und Prompt nicht gezielt für die Aufgabe ausgewählt wurden.

8.1.1 Datenbereinigung

Die Datenbereinigung ist die Kategorie mit der höchsten und zugleich stabilsten Ergebnisqualität. Das Entfernen überflüssiger Leerzeichen und die Kennzeichnung fehlender Werte lösen alle vier Modelle – einschließlich des lokalen – mit gut gewähltem Prompt vollständig korrekt, ebenso wie die Vereinheitlichung der vier Datumsformate, die bei allen Modellen zwischen 95,9 und 96,5 Prozent erreicht. Beide Aufgaben sind, wie in Abschnitt 5.3 beschrieben, durch eine endliche Menge syntaktischer Regeln vollständig beschreibbar, und genau in diesem Bereich verhält sich der erzeugte Code so zuverlässig wie eine manuell geschriebene Lösung [11, 13].

Gerade hier zeigt sich jedoch auch ein Risiko, das sich in keiner Erfolgsquote niederschlägt. Skript 8.1 stellt zwei Lösungen derselben Aufgabe gegenüber, die beide fehlerfrei durchlaufen und eine schemakonforme Ausgabe erzeugen. Die erste Variante wählt die zu bereinigenden Spalten über deren Datentyp aus – ein in der Praxis verbreitetes und über Jahre korrektes Idiom. In der eingesetzten Pandas-Version werden Textspalten jedoch nicht mehr als generische Objektspalten, sondern als eigener Zeichenkettentyp geführt, sodass die Auswahl leer bleibt und die Bereinigung wirkungslos verpufft, ohne dass ein Fehler auftritt. Die zweite Variante benennt die betroffenen Spalten explizit und ist damit gegenüber dieser Änderung unempfindlich.

```
1 # still wirkungslos: die Spaltenauswahl liefert keine Treffer
2 for col in df.select_dtypes(include=['object']).columns:
3     df[col] = df[col].str.strip()
4
5 # wirksam: die betroffenen Spalten werden explizit benannt
6 for col in ["full_name", "email", "country", "registered_at"]:
7     df[col] = df[col].astype(str).str.strip()
```

Skript 8.1: Zwei lauffähige Lösungen derselben Bereinigungsaufgabe: Die obere Variante bleibt in der eingesetzten Pandas-Version wirkungslos, weil Textspalten nicht mehr als Objektspalten geführt werden; die untere benennt die Spalten explizit.

Dieser Befund ist für die Beurteilung der Eignung von erheblicher Bedeutung: Der Fehler ist nicht inhaltlicher, sondern umgebungsbedingter Natur, er erzeugt weder eine Fehlermeldung noch eine Schemaverletzung und wäre ohne eine bekannte Soll-Lösung im laufenden Betrieb nicht ohne Weiteres zu bemerken. Er entsteht dort, wo das Modell auf ein in seinen Trainingsdaten dominantes, inzwischen jedoch überholtes Idiom zurückgreift, und tritt damit systematisch und nicht zufällig auf [6, 7]. Behoben wurde er im Rahmen der Prompt-Gestaltung nicht durch einen allgemeinen Hinweis auf Versionsunterschiede, sondern erst dadurch, dass dem Modell die Datentypen der Eingabespalten explizit mitgeteilt wurden.

Die semantische Vereinheitlichung der Länderangaben fällt mit 61,7 Prozent im Mittel deutlich ab, obwohl jedes Cloud-Modell mit gut gewähltem Prompt zwischen 94,6 und 95,8 Prozent erreicht. Der Grund für die verbleibende Lücke ist aufschlussreich und lässt sich anhand der Zwischenergebnisse der Pipeline exakt beziffern: Alle drei Cloud-Modelle lösen die Aufgabe, indem sie im erzeugten Code eine Zuordnungstabelle aufbauen und die Rohwerte darüber nachschlagen. Damit ist die Lösung nicht durch das Weltwissen des Modells begrenzt, sondern durch das, was es beim Schreiben des Codes vorab aufzählt. Nicht aufgelöst wurden auf diese Weise das Kürzel *SUI* für die

Schweiz (14 Datensätze) und der Tippfehler Deutschland (11 Datensätze) bei allen drei Modellen, ergänzt um Großbritannien bei einem und Österreich bei einem weiteren Modell (jeweils 7 Datensätze). Der Modus der Code-Generierung reduziert die semantische Aufgabe damit strukturell auf dieselbe Nachschlagelogik, die auch die regelbasierte Pipeline verwendet – und erreicht folgerichtig ein ähnliches Ergebnis. Im Direkt-Modus, in dem das Modell jeden Wert einzeln beurteilt, statt eine verallgemeinernde Vorschrift zu formulieren, steigt die Genauigkeit auf 96,4 bis 96,5 Prozent und liegt damit erstmals über der regelbasierten Lösung [7, 25].

8.1.2 Duplikaterkennung

Die Duplikaterkennung weist die stärkste Spreizung aller drei Kategorien auf, die Abbildung 8.2 nach Aufgaben und Strategien aufschlüsselt. Exakte Duplikate werden von allen vier Modellen und in jeder Strategie fehlerfrei entfernt; die Aufgabe ist mit einem einzigen Funktionsaufruf lösbar und trennt die Ansätze nicht. Bereits die schlüsselbasierte Entdoppelung anhand der E-Mail-Adresse fällt jedoch auf 41,6 Prozent im Mittel, und die unscharfen Duplikate erreichen mit 19,1 Prozent den niedrigsten Wert aller lösbaren Aufgaben.

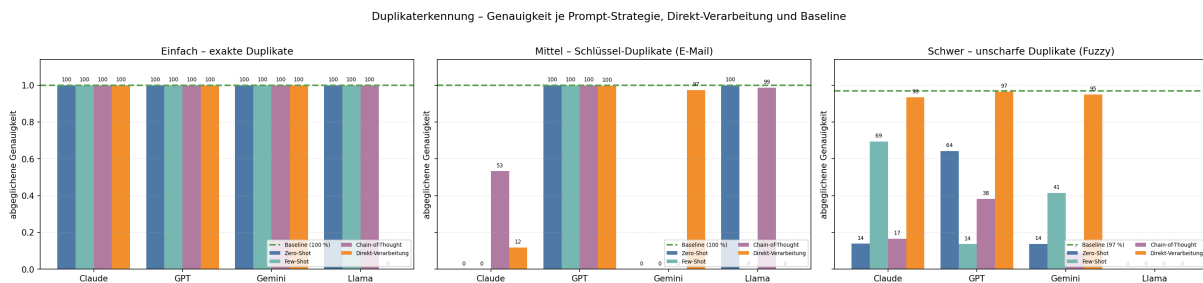


Abbildung 8.2: Duplikaterkennung: Genauigkeit je Modell und Prompting-Strategie im Vergleich zur Direkt-Verarbeitung und zur regelbasierten Pipeline (gestrichelte Linie).

Bei näherer Betrachtung sind die niedrigen Mittelwerte weniger auf ein Unvermögen der Modelle als auf die Zweiseitigkeit der Aufgabe zurückzuführen. Anders als bei der Bereinigung, bei der jede Zeile für sich betrachtet wird, verlangt die Entdoppelung eine Entscheidung über das Verhältnis von Datensätzen zueinander, bei der sowohl zu wenig als auch zu viel entfernt werden kann. Die Ergebnisse zeigen beide Fehlerrichtungen: Bei den unscharfen Duplikaten liegt der beste Wert je Modell zwischen 41,5 und 69,4 Prozent, wobei ein zu enger Ähnlichkeitsschwellenwert echte Zwillinge übersieht, während ein zu weiter Schwellenwert unbeteiligte Datensätze zusammenführt. In der verketteten Pipeline trat dieser zweite Fehlertyp in besonders klarer Form auf: Werden akademische Grade und Anreden nicht als solche erkannt, sondern als Na-

mensbestandteile behandelt, entstehen Vergleichsschlüssel, unter denen bis zu sieben verschiedene Kunden zusammenfallen. Erst die explizite Anweisung, Titel nicht als Teil des Namens zu behandeln und Übereinstimmung sowohl im Namen als auch in der E-Mail-Adresse zu verlangen, löste dieses Problem auf.

Der deutlichste Befund dieser Kategorie ergibt sich aus dem Vergleich der beiden Ausführungsmodi. Während die beste Code-Generierung bei den unscharfen Duplikaten 69,4 Prozent erreicht, liegt der Direkt-Modus bei denselben Modellen zwischen 93,5 und 96,5 Prozent. Das Muster entspricht dem der Länder-Vereinheitlichung und hat dieselbe Ursache: Die Beurteilung, ob die beiden in Tabelle 5.2 gegenübergestellten Datensätze `Uta Lange / gkabus@example.org` und `Uta_Lange / gka.bus@example.org` dieselbe Person bezeichnen, ist für ein Sprachmodell eine unmittelbar zugängliche Einzelfallentscheidung, aber eine schwer zu verallgemeinernde Regel. Sobald das Modell diese Entscheidung in Code fassen muss, ist es gezwungen, sie in einen numerischen Schwellenwert zu übersetzen, und verliert dabei genau die Fähigkeit, die es der regelbasierten Pipeline überlegen macht [7, 13].

8.1.3 Datentransformation

Typkonvertierung und abgeleitete Spalten fügen sich in das Bild der formal beschreibbaren Aufgaben ein: Beide werden von den Cloud-Modellen mit gut gewähltem Prompt vollständig oder nahezu vollständig korrekt gelöst, wobei die Typkonvertierung mit 68,0 Prozent im Mittel vor allem deshalb abfällt, weil das lokale Modell hier lediglich 26,7 Prozent erreicht und die schema-angereicherte Strategie zu abstürzenden Konvertierungen verleitet.

Die zusammengesetzte Aufgabe aus Join, Bereinigung und Aggregation ist dagegen die einzige der neun Aufgaben, die im Modus der Code-Generierung von keinem Modell und in keiner Strategie gelöst wurde. Von den 48 Läufen brachen 33 mit einem Programmfehler ab, die verbleibenden 15 erzeugten eine lauffähige, aber inhaltlich abweichende Ausgabe. Deren Analyse zeigt zwei wiederkehrende Fehlerbilder. Im ersten liefert das Ergebnis 65 statt der erwarteten 60 Gruppen und aggregiert lediglich 485 der 2.000 Bestellungen, weil die Verknüpfung Bestellungen ohne passenden Kundendatensatz verwirft. Im zweiten werden zwar alle 2.000 Bestellungen korrekt aggregiert, die Gruppierung erfolgt jedoch über 42 verschiedene, lediglich in Großbuchstaben überführte Rohschreibweisen der Länderangabe statt über die zehn ISO-Codes, so dass etwa `DE`, `DEUTSCHLAND`, `GERMANY`, `GER` und `DEUTSHCLAND` als fünf getrennte Länder

erscheinen.

Beide Fehlerbilder sind aufschlussreich, weil sie nicht die Aggregation selbst betreffen. Die Modelle beherrschen Verknüpfung und Gruppierung als solche; sie scheitern daran, die vorgelagerte semantische Bereinigung im selben Arbeitsschritt mitzudenken. Dass die Genauigkeit dabei exakt auf null fällt, ist der Zusammensetzung der Aufgabe geschuldet: Weil die bereinigte Länderangabe zugleich Gruppierungsschlüssel ist, wirkt sich jede unvollständige Bereinigung auf sämtliche aggregierten Werte aus, sodass kein einziger Kennzahlenwert korrekt bleibt. Die Kennzahl misst hier folglich nicht die Fähigkeit zur Aggregation, sondern die Fähigkeit zur Komposition mehrerer Verarbeitungsschritte. Dass es sich um genau diese Unterscheidung handelt, belegt Abschnitt 8.4: Wird derselbe Schritt als Glied einer Kette mit bereits bereinigten Eingabedaten gestellt, lösen ihn alle vier Modelle fehlerfrei.

8.2 Vergleich der Modelle

Abbildung 8.3 fasst den Modellvergleich über alle 432 Läufe zusammen. Unter den kommerziell betriebenen Modellen liegen die Ergebnisse eng beieinander: Das Modell von OpenAI erreicht mit einer Genauigkeit von 0,733 bei einer Erfolgsquote von 79,6 Prozent den höchsten Wert, gefolgt von Anthropic mit 0,661 und 73,1 Prozent sowie Google mit 0,635 und 71,3 Prozent. Die Abstände zwischen diesen drei Modellen sind mit rund zehn Prozentpunkten deutlich geringer als die zuvor beobachteten Unterschiede zwischen den Aufgaben, die von 0 bis über 95 Prozent reichen. Für die Frage der Eignung ist damit die Wahl der Aufgabe erheblich folgenreicher als die Wahl des Anbieters – ein Befund, der die in der Literatur verbreitete Fokussierung auf Modellranglisten relativiert [20, 21].

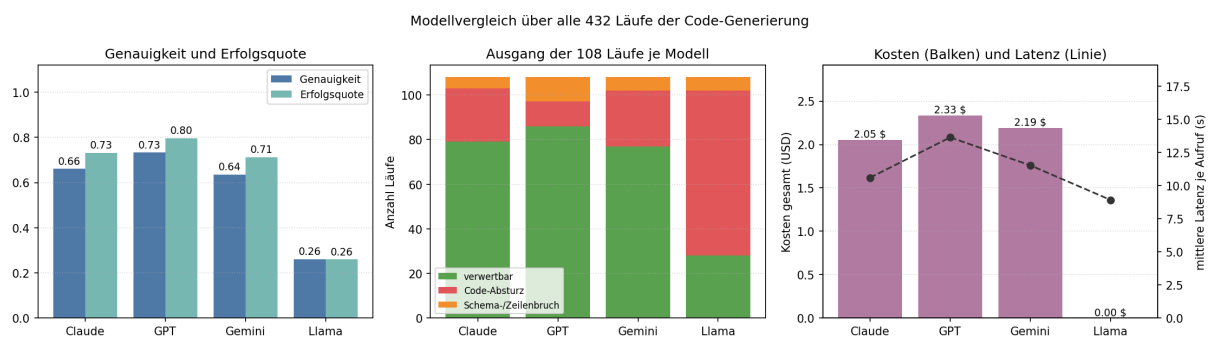


Abbildung 8.3: Modellvergleich über alle 432 Läufe der Code-Generierung: Genauigkeit und Erfolgsquote, Ausgang der je 108 Läufe sowie Gesamtkosten und mittlere Antwortzeit.

Deutlich abgesetzt ist das lokal betriebene Modell der Llama-Reihe mit einer Genauigkeit von 0,261 und einer Erfolgsquote von 25,9 Prozent. Das mittlere Bild in Abbildung 8.3 zeigt, worin dieser Abstand besteht: Von seinen 108 Läufen brachen 74 mit einem Programmfehler ab, während die Cloud-Modelle zwischen 11 und 25 solcher Abbrüche aufweisen. Das lokale Modell scheitert damit weit überwiegend nicht an der inhaltlichen Aufgabe, sondern daran, überhaupt lauffähigen Code zu erzeugen. Diese Trennung ist praktisch bedeutsam, weil sie einen qualitativen Unterschied markiert: Ein inhaltlich abweichendes Ergebnis lässt sich prüfen und korrigieren, ein nicht lauffähiges Skript hingegen erzwingt einen erneuten Aufruf und verschiebt das Problem lediglich [6, 7]. Dass das Modell die Aufgaben durchaus versteht, belegt der in Abschnitt 8.4 dargestellte isolierte Kettendurchlauf, in dem es bei bis zu drei zugestandenen Versuchen je Schritt alle neun Schritte durchläuft.

Diese Beobachtung erfordert eine Einordnung, die zugleich eine Entscheidung über den weiteren Aufbau dieses Kapitels begründet. Der gemessene Abstand ist keine Aussage über lokal betreibbare Modelle im Allgemeinen, sondern über dasjenige Modell, das sich auf der für diese Arbeit verfügbaren Hardware sinnvoll ausführen ließ. Die Größe eines lokal betriebenen Modells ist unmittelbar durch den verfügbaren Grafikspeicher begrenzt, und mit ihr die Fähigkeit, längere Anweisungen zu verarbeiten und syntaktisch fehlerfreien Code zu erzeugen. Die kommerziellen Modelle unterliegen dieser Beschränkung nicht, da die Ausführung beim Anbieter stattfindet. Ein Vergleich zwischen beiden misst somit nicht allein die Modellqualität, sondern zu einem erheblichen Teil die Ausstattung des ausführenden Rechners [6, 20].

Aus diesem Grund wird das lokale Modell in den folgenden Vergleichen dieses Kapitels – dem Vergleich der Prompting-Strategien, der verketteten Pipeline und dem Bestätigungslauf – nicht weiter geführt. Der Grund ist nicht sein schwächeres Abschneiden als solches, sondern dass seine Ausfälle die Mittelwerte in einer Weise verschieben, die sich nicht mehr der untersuchten Einflussgröße zurechnen lässt: Wie Abschnitt 8.3 zeigt, entsteht die dort gemessene Rangfolge der Prompting-Strategien nahezu vollständig aus dem Verhalten dieses einen Modells. Seine Ergebnisse aus der Faktormatrix bleiben in Abschnitt 8.1 und im vorliegenden Abschnitt vollständig ausgewiesen; ausgeschlossen wird es nur dort, wo es eine andere Fragestellung überlagern würde.

Für die Praxis folgt daraus ausdrücklich keine Empfehlung gegen den lokalen Betrieb. Er ist die einzige der untersuchten Optionen, bei der keine Daten das eigene Netz ver-

lassen, und er verursacht keine laufenden Nutzungskosten – gegenüber den 2,05 bis 2,33 US-Dollar je Modell für die Faktormatrix und rund 0,19 US-Dollar je vollständigem Pipeline-Durchlauf ein Vorteil, der bei dauerhaftem Betrieb erheblich wird. Da die beobachtete Schwäche überwiegend in der Erzeugung lauffähigen Codes und nicht im inhaltlichen Verständnis der Aufgaben liegt, ist zu erwarten, dass ein auf leistungsfähigerer Hardware ausführbares, größeres Modell diesen Abstand deutlich verringert. Für Vorhaben mit entsprechender Ausstattung ist eine eigene Erprobung daher lohnend, und die in Abschnitt 9.2 genannten Bedingungen – kleinteiliger Aufgabenzuschnitt und mehrere zugestandene Versuche je Schritt – geben dafür den Rahmen vor [9, 20].

Die Kostenseite relativiert die Qualitätsunterschiede zusätzlich. Über alle 108 Läufe eines Modells liegen die Nutzungskosten zwischen 2,05 und 2,33 US-Dollar und damit in derselben Größenordnung; das lokal ausgeführte Modell verursacht überhaupt keine Nutzungskosten, benötigt dafür jedoch eigene Hardware und lieferte, wie in Abschnitt 7.3 beschrieben, im Direkt-Modus wegen des begrenzten Kontextfensters in den meisten Fällen kein verwertbares Ergebnis. Die Antwortzeiten unterscheiden sich mit 8,9 bis 13,6 Sekunden je Aufruf ebenfalls nur moderat. Ein Modell allein aufgrund von Kosten oder Geschwindigkeit auszuwählen, ist auf dieser Grundlage nicht zu begründen; die Auswahl folgt vielmehr der Zuverlässigkeit bei der konkret vorliegenden Aufgabenart.

8.3 Vergleich der Prompting-Strategien

Der Vergleich der vier Prompting-Strategien in Abbildung 8.4 liefert zunächst ein für die Ausgangserwartung überraschendes Ergebnis. Über alle vier Modelle und Aufgaben hinweg erzielt nicht eine der aufwändigeren Strategien, sondern das einfache Zero-Shot-Prompting mit 0,655 die höchste Genauigkeit, gefolgt vom Chain-of-Thought-Prompting mit 0,641 und dem Few-Shot-Prompting mit 0,609. Die schemaangereicherte Variante fällt mit 0,386 deutlich ab. Der Aufwand wächst dabei in die entgegengesetzte Richtung: Das Few-Shot-Prompting benötigt mit im Mittel 834 Eingabetoken annähernd das Dreifache des Zero-Shot-Prompts mit 290 Token.

Diese Rangfolge hält jedoch einer genaueren Prüfung nicht stand, und die Ursache ist das in Abschnitt 8.2 beschriebene lokale Modell. Beschränkt man dieselbe Auswertung auf die drei kommerziellen Modelle, verschwindet der Unterschied nahezu vollständig: Zero-Shot erreicht 72,6, Few-Shot 73,3 und Chain-of-Thought 73,4 Prozent – eine Spanne von 0,8 statt 4,6 Prozentpunkten. Das lokale Modell allein trägt den

Effekt, und zwar in ausgeprägter Form: Es fällt von 44,1 Prozent unter dem Zero-Shot-Prompt auf 24,0 Prozent unter dem Few-Shot-Prompt, während seine Erfolgsquote von 44,4 auf 25,9 Prozent sinkt. Der dreifach längere Prompt überfordert es, was zu den in Abschnitt 8.2 genannten Grenzen der verfügbaren Hardware passt. Die scheinbar allgemeine Aussage „der einfachste Prompt ist der beste“ ist damit keine Aussage über Prompting-Strategien, sondern über die Kapazität eines bestimmten Modells [23, 24].

Aussagekräftig ist der Strategievergleich folglich erst auf Aufgabenebene und unter den kommerziellen Modellen. Dort zeigt sich, dass sechs der neun Aufgaben zwischen den Strategien überhaupt nicht unterscheiden: Die Bereinigungsaufgaben und die exakten Duplikate werden von jeder Strategie nahezu fehlerfrei gelöst, die zusammengesetzte Aufgabe von keiner. Der gesamte messbare Unterschied konzentriert sich auf die beiden anspruchsvollen Entdoppelungsaufgaben, und dort mit klarer Zuordnung: Bei den unscharfen Duplikaten führt das Few-Shot-Prompting mit 41,5 gegenüber 30,7 Prozent für Zero-Shot und 21,3 für Chain-of-Thought, bei den Schlüssel-Duplikaten hingegen das Chain-of-Thought-Prompting mit 51,1 gegenüber jeweils 33,3 Prozent. Beispiele wirken damit dort, wo eine Ähnlichkeitsentscheidung zu treffen ist, für die eine verbale Beschreibung schlecht taugt; die Aufforderung zur schrittweisen Herleitung wirkt dort, wo eine mehrstufige Auswahlregel umzusetzen ist. Wie Abschnitt 8.6.4 zeigt, ist genau dieses aufgabenbezogene Muster auf dem unbekannten Datensatz reproduzierbar, die Gesamttrangfolge dagegen nicht [19, 22].

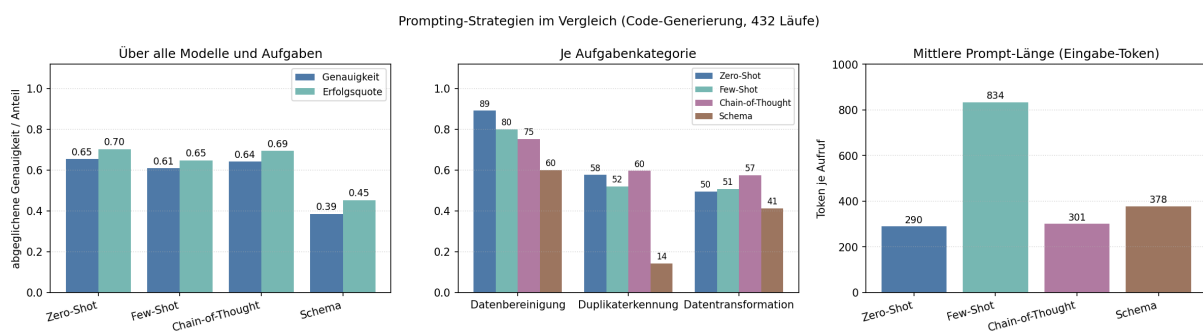


Abbildung 8.4: Prompting-Strategien im Vergleich: Genauigkeit und Erfolgsquote über alle Läufe, Genauigkeit je Aufgabenkategorie sowie mittlere Prompt-Länge.

Das mittlere Bild der Abbildung, das die Kategorien über alle vier Modelle zusammenfasst, ist entsprechend mit Vorsicht zu lesen: Der dort bei der Datenbereinigung sichtbare Vorsprung des Zero-Shot-Promptings von 89,2 gegenüber 80,0 und 75,3 Prozent geht auf dieselbe Ursache zurück und besteht unter den kommerziellen Modellen nicht. Festzuhalten bleibt der allgemeinere Zusammenhang, der sich in beiden Betrachtun-

gen zeigt: Der Nutzen zusätzlicher Anleitung ist nicht generell, sondern an die Eigenart der Aufgabe gebunden. Wo eine Aufgabe in einem einzigen gedanklichen Schritt lösbar ist, stiftet sie keinen Mehrwert und verleitet zu unnötig komplexem und damit fehleranfälligerem Code; wo mehrere aufeinander aufbauende Entscheidungen zu treffen sind oder eine Ermessensgrenze zu setzen ist, wirkt sie [22, 23].

Der Einbruch der schema-angereicherten Strategie verdient gesonderte Betrachtung, da er dem eigentlich beabsichtigten Zweck zuwiderläuft. Die Angabe des Zielschemas mit Spaltennamen und Datentypen sollte die strukturelle Korrektheit der Ausgabe unterstützen; tatsächlich fällt die Genauigkeit bei der Duplikaterkennung auf 14,3 Prozent und damit auf ein Viertel des Werts der übrigen Strategien. Die Ursache liegt darin, dass die Modelle die genannten Zieltypen als unmittelbaren Handlungsauftrag auffassen und Konvertierungen vornehmen, die die Aufgabenstellung gar nicht verlangt – etwa das Umwandeln eines Datumsfeldes, dessen Rohwerte in dieser Aufgabe noch in vier verschiedenen Formaten vorliegen und die Konvertierung folglich zum Abbruch bringen. Zusätzlicher Kontext ist damit nicht per se hilfreich: Er wirkt nur, wenn er zur Aufgabe passt, und richtet Schaden an, wenn er über sie hinausweist [19, 24].

Zusammenfassend ist die Wirkung der Prompt-Gestaltung damit anders zu beurteilen, als es die Gesamttrangfolge nahelegt. Sie ist nicht gleichmäßig über die Aufgaben verteilt, sondern auf wenige konzentriert: Bei sechs der neun Aufgaben ist sie ohne messbare Wirkung, bei zwei entscheidet sie über bis zu 20 Prozentpunkte, und bei einer – der schema-angereicherten Variante in der Duplikaterkennung – richtet sie erheblichen Schaden an. Ihr Gewicht ist damit dort, wo sie überhaupt wirkt, durchaus mit dem der Modellwahl vergleichbar; eine allgemeine Empfehlung für oder gegen aufwändigere Prompts lässt sich daraus jedoch nicht ableiten, sondern nur eine je Aufgabenart [23, 24].

8.4 Ergebnisse der verketteten Pipeline

Die bisher dargestellten Ergebnisse betrachten jede Aufgabe für sich. Die in Abschnitt 4.1 eingeführte dritte Untersuchungsebene prüft, wie sich dieses Bild verändert, wenn dieselben Aufgaben als durchgängige Strecke ausgeführt werden, in der jeder Schritt auf dem Ergebnis seines Vorgängers aufsetzt.

8.4.1 Qualitätsverlauf über die Kette

Abbildung 8.5 zeigt den Verlauf der neun Schritte für alle vier Modelle und die regelbasierte Pipeline. Drei der vier Modelle durchlaufen die Kette vollständig und erreichen in den Schritten 1 bis 8 durchweg Werte zwischen 97,6 und 100 Prozent. Der Aufwand einer vollständigen Strecke ist dabei gering: Er beträgt je Modell zwischen 0,176 und 0,198 US-Dollar sowie 117 bis 153 Sekunden, gegenüber 6,57 US-Dollar für die vollständige Faktormatrix. Die regelbasierte Pipeline benötigt für dieselbe Strecke 0,17 Sekunden und keine Nutzungskosten.



Abbildung 8.5: Abgeglichene Genauigkeit je Schritt der verketteten Pipeline. Grau markierte Felder kennzeichnen Schritte, die infolge eines vorangegangenen Abbruchs blockiert wurden.

Der Endschritt bricht dagegen bei allen Teilnehmern ein: Das Modell von Google erreicht 81,8 Prozent, jenes von OpenAI 72,7 Prozent und jenes von Anthropic 71,2 Prozent, die regelbasierte Pipeline 60,6 Prozent. Bemerkenswert ist zunächst, dass die Werte damit weit über dem Ergebnis derselben Aufgabe als Einzellauf liegen, wo sie bei null lag. Die Zerlegung des Endschritts erklärt beides: Wendet man die Referenzlogik der Aggregation auf die tatsächlich gelieferten Zwischenergebnisse jedes Teilnehmers an, ergibt sich für alle vier eine Übereinstimmung von 1,000. Der von jedem Teilnehmer selbst erzeugte Aggregationscode ist also nachweislich korrekt; der Verlust entsteht ausschließlich aus den Daten, die er verarbeitet. Konkret bleiben bei den Mo-

dellen 5 bis 8 und bei der regelbasierten Pipeline 20 Bestellungen ohne zugehörigen Kundendatensatz zurück, weil in der vorgelagerten Entdoppelung jeweils der andere von zwei unscharfen Zwillingen behalten wurde. Da die Aggregation über Land und Kategorie gruppiert, macht jede solche Bestellung zwei Gruppen in beiden Kennzahlen falsch, sodass wenige fehlplatzierte Datensätze einen überproportionalen Ausschlag erzeugen.

Abbildung 8.6 stellt beide Betrachtungsweisen unmittelbar gegenüber. Für sechs der neun Schritte liegt das Ergebnis in der Kette über dem des unabhängigen Einzellaufs, teilweise erheblich: bei den unscharfen Duplikaten um 54,3, bei der Aggregation um 75,2 und bei den Schlüssel-Duplikaten um 35,3 Prozentpunkte. Dieser zunächst widersprüchlich wirkende Befund hat eine klare Ursache: In der Kette erhält jeder Schritt bereits vorbereitete Daten und eine eng umrissene Teilaufgabe, während er im Einzellauf zusätzlich alle vorgelagerten Mängel mitbehandeln muss. Die Zerlegung einer Strecke in kleine, klar abgegrenzte Schritte verbessert das Ergebnis somit unabhängig vom eingesetzten Modell – ein Befund, der unmittelbar auf die praktische Gestaltung KI-gestützter ETL-Strecken durchschlägt [1, 4].

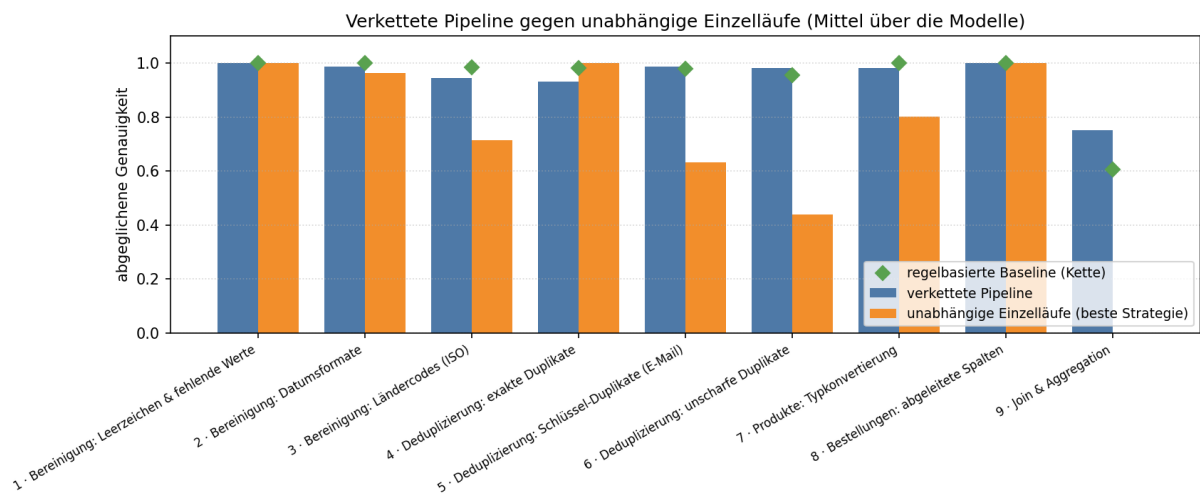


Abbildung 8.6: Dieselben Aufgaben als Glied einer Kette und als unabhängiger Einzellauf, gemittelt über die Modelle. Die Kette schneidet in sechs von neun Schritten besser ab.

8.4.2 Trennung von eigener Leistung und Fehlerfortpflanzung

Der bis hierhin gemessene Verlauf lässt offen, welcher Anteil eines schwachen Schrittergebnisses dem Schritt selbst und welcher den Vorgängerschritten zuzurechnen ist. Der in Abschnitt 4.1 vorgesehene isolierte Durchlauf, in dem jeder Schritt die Soll-Ausgabe seines Vorgängers erhält, beantwortet diese Frage. Abbildung 8.7 stellt beide

Durchläufe je Modell und Schritt gegenüber; die Differenz beider Balken beziffert unmittelbar den Preis der Fehlerfortpflanzung.

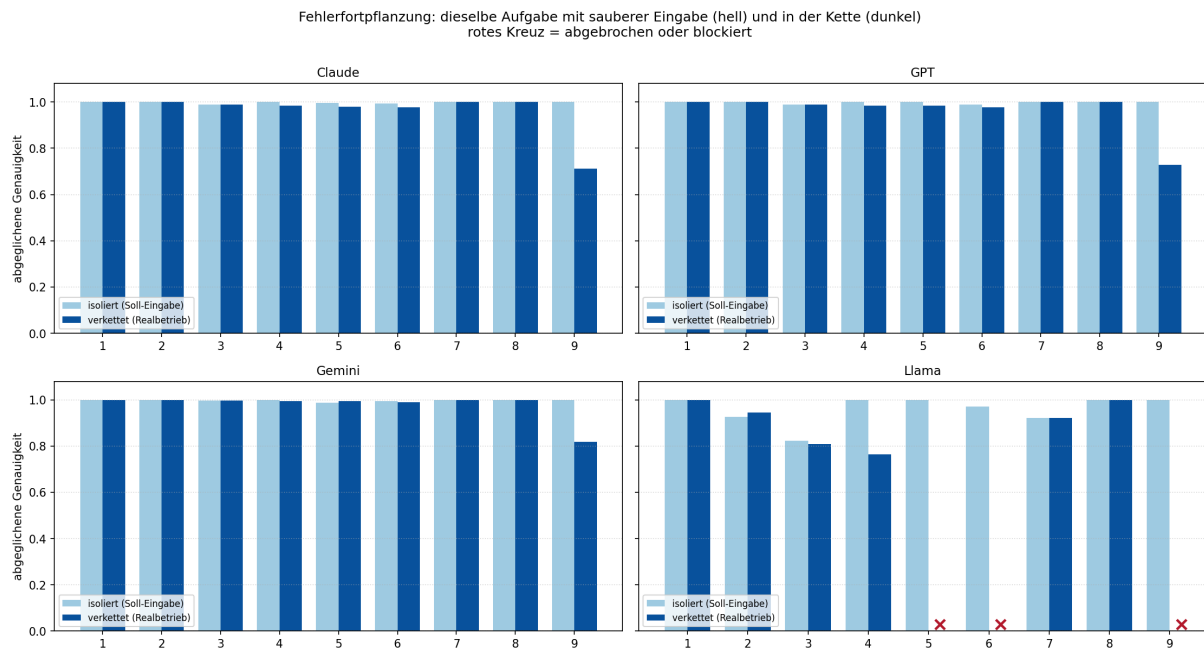


Abbildung 8.7: Dieselbe Aufgabe mit fehlerfreier Eingabe (hell) und in der Kette (dunkel). Rote Kreuze kennzeichnen abgebrochene oder blockierte Schritte.

Das Ergebnis ist eindeutig: Mit fehlerfreier Eingabe lösen alle vier Modelle den Endschritt vollständig korrekt. Der in der Kette gemessene Verlust von 18,2 bis 28,8 Prozentpunkten ist damit zu hundert Prozent geerbt und nicht dem Endschritt anzulasten. Dasselbe gilt in abgeschwächter Form für die Entdoppelungsschritte, die isoliert zwischen 98,8 und 100 Prozent erreichen und in der Kette ein bis zwei Prozentpunkte verlieren. Als eigene, nicht geerbte Schwäche verbleibt allein die semantische Länder-Vereinheitlichung: Sie ist der einzige Schritt, den auch mit fehlerfreier Eingabe kein Modell vollständig löst (99,1 / 98,8 / 98,8 / 82,3 Prozent), und zwar aus dem in Abschnitt 8.1.1 belegten Grund der unvollständig aufgezählten Zuordnungstabelle.

Auch das Verhalten des lokalen Modells klärt sich erst durch diese Trennung auf. In der Kette erreicht es 6 von 9 Schritten und bricht in Schritt 5 ab; isoliert durchläuft es alle neun Schritte und löst den Endschritt ebenfalls fehlerfrei. Die Ursache des Abbruchs liegt drei Schritte früher: Seine Lösung der Datumsvereinheitlichung ließ 145 der 540 Registrierungsdaten unverändert als Unix-Zeitstempel stehen, was für sich genommen eine Genauigkeit von 94,6 Prozent bedeutet und die Kette nicht unterbricht. Erst der Entdoppelungsschritt, der auf diesem Feld eine Datumskonvertierung vornimmt, scheitert an den verbliebenen Zeitstempeln und blockiert die beiden folgen-

den Schritte. Zwei der drei Ausfälle des lokalen Modells sind damit nicht eigene Fehler, sondern Folgen eines früheren, für sich betrachtet unauffälligen Ergebnisses.

Für die Praxis folgt daraus eine unmittelbar umsetzbare Konsequenz: In verketteten Strecken entscheidet die Qualität der frühen, semantisch geprägten Schritte über das Endergebnis, und ein Fehler wird umso teurer, je weiter vorn er entsteht. Eine Prüfung nach jedem Schritt ist deshalb wirksamer als eine Endkontrolle, weil sie einen Mangel dort sichtbar macht, wo er noch mit begrenztem Aufwand zu beheben ist, statt ihn erst in seiner verstärkten Form am Ende der Strecke zu entdecken [10, 12].

8.5 Reproduzierbarkeit der Ergebnisse

Die in Abschnitt 4.1 als querschnittliche Dimension verankerte Reproduzierbarkeit wird aus der Streuung der Genauigkeit über die drei Wiederholungen jeder Konfiguration bestimmt. Abbildung 8.8 zeigt das Ergebnis. Über alle Konfigurationen hinweg beträgt die mittlere Standardabweichung 0,041, und 74,3 Prozent der Konfigurationen liefern über alle drei Wiederholungen hinweg ein exakt identisches Ergebnis. Die niedrig gewählte Temperatur hat die Variabilität somit wirksam begrenzt, aber nicht beseitigt: In gut einem Viertel der Fälle führt derselbe Prompt bei demselben Modell zu messbar unterschiedlichen Ergebnissen.

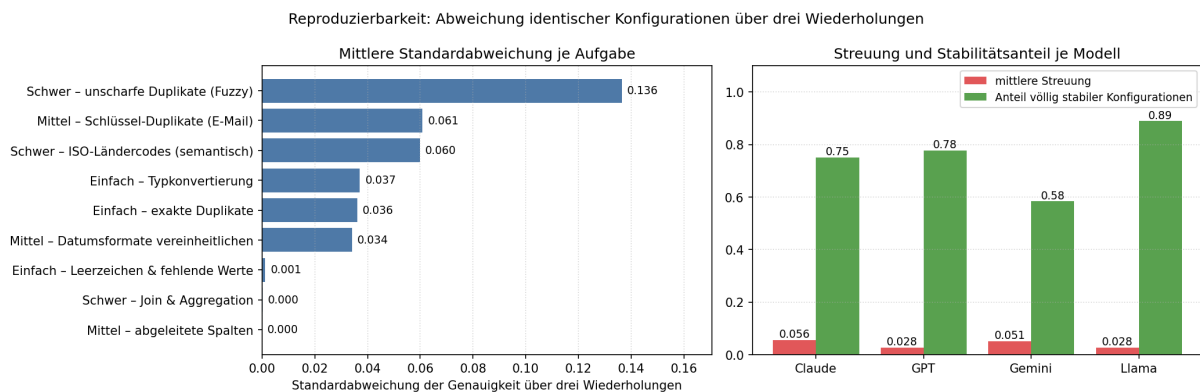


Abbildung 8.8: Streuung der Genauigkeit über die drei Wiederholungen je Konfiguration, aufgeschlüsselt nach Aufgabe und nach Modell.

Die Verteilung dieser Instabilität ist dabei nicht zufällig. Sie konzentriert sich auf genau jene Aufgaben, die auch inhaltlich am schwersten fallen: Die unscharfen Duplikate weisen mit 0,136 die mit Abstand höchste Streuung auf, gefolgt von den Schlüssel-Duplikaten mit 0,061 und der semantischen Länder-Vereinheitlichung mit 0,060. Die abgeleiteten Spalten und die einfache Bereinigung sind dagegen praktisch vollstän-

dig stabil. Der Zusammenhang ist plausibel: Wo eine Aufgabe eine eindeutige, formal beschreibbare Lösung besitzt, konvergieren wiederholte Generierungen auf dieselbe Umsetzung; wo hingegen eine Ermessensentscheidung wie die Wahl eines Ähnlichkeitsschwellenwerts zu treffen ist, schlägt sich die probabilistische Natur des Modells unmittelbar im Ergebnis nieder [6, 7].

Diese Kopplung hat eine unangenehme praktische Konsequenz, die über die reine Kennzahl hinausreicht: Instabilität und Fehleranfälligkeit treten gemeinsam auf. Gerade dort, wo ein Ergebnis besonders sorgfältig zu prüfen wäre, ist es zugleich am wenigsten verlässlich wiederholbar, sodass eine einmalige Abnahme eines Verarbeitungsschritts dessen künftiges Verhalten nicht garantiert. Für den produktiven Einsatz spricht dies dafür, den einmal erzeugten und geprüften Code als festes Artefakt zu übernehmen, statt ihn bei jeder Ausführung neu erzeugen zu lassen – eine Konsequenz, die der Modus der Code-Generierung im Unterschied zur direkten Verarbeitung überhaupt erst zulässt. Die Unterschiede zwischen den Modellen sind demgegenüber nachrangig; die mittlere Streuung liegt zwischen 0,028 und 0,056, ohne dass sich daraus eine belastbare Rangfolge ergäbe.

8.6 Bestätigungslauf auf einem unbekannten Datensatz

Alle bisher berichteten Ergebnisse beruhen auf dem Datensatz mit dem Startwert 1. An ihm wurden im Verlauf der Arbeit die Prompts formuliert, der Aufgabenzuschnitt festgelegt und die Auswertungslogik entwickelt; insbesondere die in Abschnitt 8.1.1 beschriebene Angabe der Eingabe-Datentypen und die in Abschnitt 8.1.2 beschriebene Anweisung zum Umgang mit akademischen Graden sind unmittelbare Reaktionen auf dort beobachtete Fehler. Die an ihm gemessenen Werte sind damit nicht unbefangen: Sie beschreiben die Leistung auf einem Datensatz, gegen den die Konfiguration justiert wurde, und überschätzen die zu erwartende Leistung auf unbekannten Daten in unbekanntem Umfang. Diese Gefahr ist in der methodischen Literatur zur Bewertung von LLMs als zentrale Fehlerquelle beschrieben und wird dort regelmäßig als Grund dafür angeführt, dass berichtete Kennzahlen sich im Einsatz nicht bestätigen [6, 10].

Um die Tragfähigkeit der Befunde zu prüfen, wurde deshalb ein zweiter, unabhängiger Datensatz mit dem Startwert 2 erzeugt und ohne jede weitere Anpassung von Prompts, Aufgabenstellungen oder Auswertungscode durchlaufen. Er folgt derselben

Erzeugungsvorschrift aus Kapitel 5, enthält jedoch andere Namen, andere Adressen und eine andere Zuordnung der eingebauten Abweichungen. Für den Direkt-Modus kommt die in Abschnitt 8.4 eingeführte halbierte Fassung dieses Datensatzes zum Einsatz, die den Startwert 102 trägt, da die vollständige Fassung das Ausgabefenster der Modelle überschreitet. Weil die Erwartung an diesen Lauf vor seiner Ausführung feststand und nachträglich nichts mehr verändert wurde, trägt er die bestätigende Aussage dieser Arbeit, während der Startwert 1 die explorative Grundlage bildet.

8.6.1 Verkettete und isolierte Ausführung

Abbildung 8.9 zeigt alle vier Kombinationen aus Ausführungsmodus und Eingabeform nebeneinander. Der ausgefüllte Balken gibt das Mittel über die Wiederholungen wieder, der hellere Aufsatz reicht bis zur besten Wiederholung; damit sind Durchschnitts- und Spitzenleistung gleichzeitig ablesbar, ohne dass der Mittelwert durch den Bestwert beschönigt würde. Schritte, die ein Lauf nicht mehr erreicht hat, gehen mit dem Wert null ein.

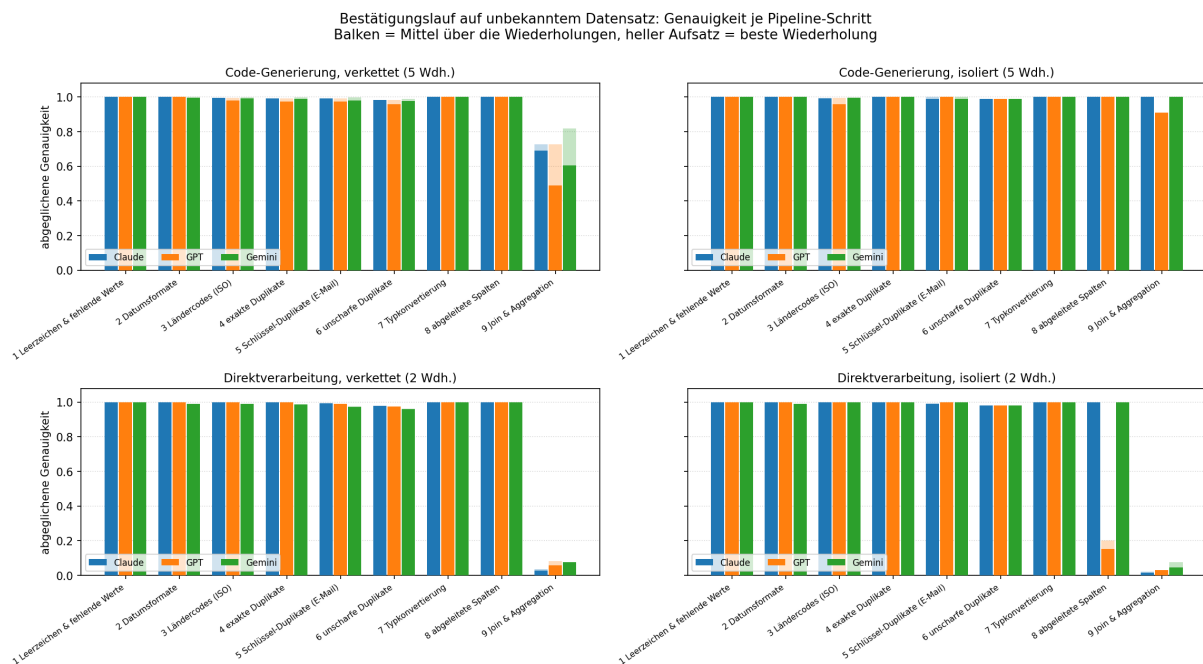


Abbildung 8.9: Abgeglichene Genauigkeit je Schritt im Bestätigungslauf, getrennt nach Ausführungsmodus und Eingabeform. Balken: Mittel über die Wiederholungen; heller Aufsatz: beste Wiederholung.

Der Code-Generierungs-Modus bestätigt das Bild aus Abschnitt 8.4 in allen wesentlichen Zügen. Über fünf Wiederholungen der verketteten Strecke erreichen die drei Cloud-Modelle in den Schritten 1 bis 8 durchweg Werte zwischen 96,0 und 100 Pro-

zent und liegen damit im selben Band wie beim Startwert 1. Über alle neun Schritte gemittelt erzielt das Modell von Anthropic 96,1 Prozent, jenes von Google 94,9 Prozent und jenes von OpenAI 93,1 Prozent, gegenüber 94,2 Prozent für die regelbasierte Vergleichspipeline. Der Endschrift bricht erneut ein, und zwar auf 69,1, 60,6 und 49,1 Prozent bei einem Baseline-Wert von 53,0 Prozent. Auch die Rangfolge bleibt erhalten: Alle drei Modelle lösen die formal beschreibbaren Schritte praktisch fehlerfrei, und der zusammengesetzte Endschrift ist die einzige Stelle, an der die Kette substantiell verliert.

Der isolierte Durchlauf trennt auch hier eigene Leistung von geerbtem Fehler, und das Ergebnis fällt noch deutlicher aus als beim Startwert 1. Mit fehlerfreier Eingabe erreichen die Modelle über alle neun Schritte 99,7, 99,7 und 98,4 Prozent; den Endschrift lösen die Modelle von Anthropic und Google in jeder einzelnen Wiederholung vollständig korrekt, jenes von OpenAI erreicht konstant 90,9 Prozent. Der in der Kette gemessene Verlust von 30,9 bis 41,8 Prozentpunkten im Endschrift ist damit fast vollständig geerbt. Die zentrale Aussage aus Abschnitt 8.4.2 – dass Evaluierungen einzelner ETL-Schritte und Evaluierungen ganzer Strecken systematisch in entgegengesetzte Richtungen verzerren – ist damit an einem Datensatz repliziert, gegen den nichts justiert wurde [2, 10].

Für den Bestätigungslauf wurde zusätzlich die regelbasierte Pipeline isoliert ausgeführt, was in Abschnitt 8.4 noch unterblieben war, und dieser Vergleich stellt den Befund auf eine breitere Grundlage. Ihre Werte sind allerdings asymmetrisch zu lesen: In den sieben formal beschreibbaren Schritten verwendet sie dieselben Routinen, aus denen auch die Kettenreferenz komponiert ist, sodass ein Ergebnis von 1,000 dort definitionsgemäß eintritt und keine Leistung ausweist. Aussagekräftig sind allein die beiden Schritte mit begrenzten Heuristiken, die sie mit 99,3 und 97,6 Prozent löst. Entscheidend ist jedoch der Endschrift: Auch die Regel löst ihn mit fehlerfreier Eingabe vollständig korrekt und fällt in der Kette auf 53,0 Prozent. Ihr Verlust von 47,0 Prozentpunkten übersteigt damit den jedes einzelnen Modells. Die Fehlerfortpflanzung ist folglich keine Eigenheit KI-gestützter Verarbeitung, sondern eine Eigenschaft der verketteten Architektur, die das deterministische Verfahren in gleichem, hier sogar stärkerem Maße trifft [2, 12].

Der Direkt-Modus liefert das gegenteilige Muster und bestätigt ebenfalls den Befund des Startwerts 101. In den Schritten 1 bis 8 arbeitet er auf dem halbierten Datensatz mit 96,3 bis 100 Prozent auf dem Niveau der Code-Generierung, wobei er die seman-

tische Länder-Vereinheitlichung mit 98,8 bis 99,9 Prozent löst und damit die regelbasierte Pipeline mit 99,1 Prozent knapp erreicht oder übertrifft. Im Endschritt jedoch fällt er auf 2,7 bis 7,6 Prozent und damit weit unter jeden anderen Ansatz. Die Ursache ist keine inhaltliche: Die Antworten waren in keinem der Schritte abgeschnitten, alle neun Schritte lieferten eine vollständige, schemakonforme Tabelle. Entscheidend ist, dass der isolierte Durchlauf hier nichts rettet – mit fehlerfreier Eingabe erreicht der Endschritt lediglich 1,5 bis 4,6 Prozent. Anders als bei der Code-Generierung ist dieser Einbruch also gerade nicht geerbt, sondern eigenes Versagen: Das Modell muss im Direkt-Modus die Aggregation über 60 Gruppen selbst rechnen, statt eine Rechenvorschrift zu formulieren, und genau diese arithmetische Zusammenfassung mehrerer hundert Einzelwerte gelingt ihm nicht. Der Modus vertauscht damit die Stärken: Er gewinnt dort, wo Einzelfallverständnis gefragt ist, und verliert dort, wo verlässliche Rechenoperationen über viele Datensätze verlangt werden [7, 25].

8.6.2 Reproduzierbarkeit über die Wiederholungen

Der Bestätigungslauf wurde im Code-Generierungs-Modus je fünfmal verkettet und isoliert ausgeführt, im Direkt-Modus je zweimal, wodurch sich die in Abschnitt 8.5 auf Aufgabenebene betrachtete Reproduzierbarkeit erstmals auf Ebene ganzer Strecken beziffern lässt. Abbildung 8.10 zeigt jede einzelne Wiederholung als Punkt, links als Gesamtwert der Strecke und rechts für den Endschritt.

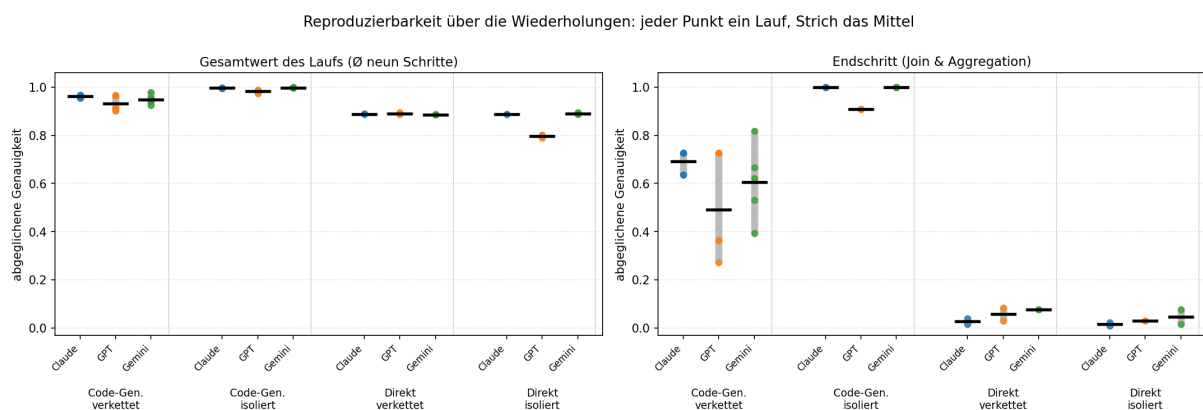


Abbildung 8.10: Streuung über die Wiederholungen des Bestätigungslaufs. Jeder Punkt ist ein vollständiger Durchlauf, der waagerechte Strich das Mittel.

Das Ergebnis ist der aufschlussreichste Einzelbefund des Bestätigungslaufs. Auf der Ebene des Gesamtwerts wirken die Läufe stabil: Die Standardabweichung über die fünf verketteten Wiederholungen beträgt lediglich 0,6 Prozentpunkte beim Modell von Anthropic, 2,0 bei Google und 3,2 bei OpenAI. Der Endschritt zeigt jedoch ein völlig

anderes Bild. Dort schwankt das Ergebnis desselben Modells bei identischem Prompt und identischen Eingabedaten zwischen 27,3 und 72,7 Prozent bei OpenAI sowie zwischen 39,4 und 81,8 Prozent bei Google; lediglich das Modell von Anthropic bleibt mit 63,6 bis 72,7 Prozent in einem engeren Band. Eine einzelne Messung des Endergebnisses ist damit praktisch nicht aussagekräftig: Je nach Wiederholung ließe sich dasselbe Modell als klar besser oder klar schlechter als die regelbasierte Pipeline darstellen.

Entscheidend ist der Vergleich mit dem isolierten Durchlauf, denn dort verschwindet diese Streuung vollständig. Alle drei Modelle liefern über sämtliche fünf Wiederholungen hinweg exakt denselben Endschrift-Wert, und die Standardabweichung des Gesamtwerts sinkt auf 0,08 bis 0,62 Prozentpunkte – bei den Modellen von Anthropic und Google um mehr als eine Größenordnung. Die beobachtete Instabilität entsteht folglich nicht aus der probabilistischen Natur der Textgenerierung selbst, sondern aus deren Verstärkung entlang der Kette: Eine kleine, für sich genommen belanglose Abweichung in der Wahl des Duplikat-Repräsentanten legt fest, welche Bestellungen im Endschrift einen Kundendatensatz finden, und schlägt dort um ein Vielfaches verstärkt durch. Die Reproduzierbarkeit einer KI-gestützten ETL-Strecke ist damit keine Eigenschaft des eingesetzten Modells, sondern eine Eigenschaft ihrer Architektur [6, 12].

Praktisch folgt daraus eine schärfere Fassung der bereits in Abschnitt 8.5 formulierten Empfehlung. Es genügt nicht, den erzeugten Code als festes Artefakt zu übernehmen; ebenso festzuschreiben sind die Zwischenergebnisse der frühen Schritte, da bereits deren Variation ausreicht, um das Endergebnis um über 40 Prozentpunkte zu verschieben, ohne dass sich an einer der beteiligten Vorschriften etwas geändert hätte.

8.6.3 Kosten, Token-Verbrauch und Laufzeit

Abbildung 8.11 stellt den Aufwand einer vollständigen Strecke in beiden Ausführungsmodi gegenüber. Die Werte sind Mittel über die Wiederholungen und beziehen sich jeweils auf einen kompletten Durchlauf aller neun Schritte.

Eine vollständige Strecke im Code-Generierungs-Modus kostet zwischen 0,176 und 0,216 US-Dollar und benötigt zwischen 102 und 135 Sekunden. Der Direkt-Modus liegt mit 1,32 bis 1,71 US-Dollar und 435 bis 854 Sekunden um den Faktor sieben beziehungsweise vier darüber – und das, obwohl er nur den halbierten Datensatz verarbeitet. Auf die Zeile bezogen beträgt der Kostenunterschied damit rund das Fünf-

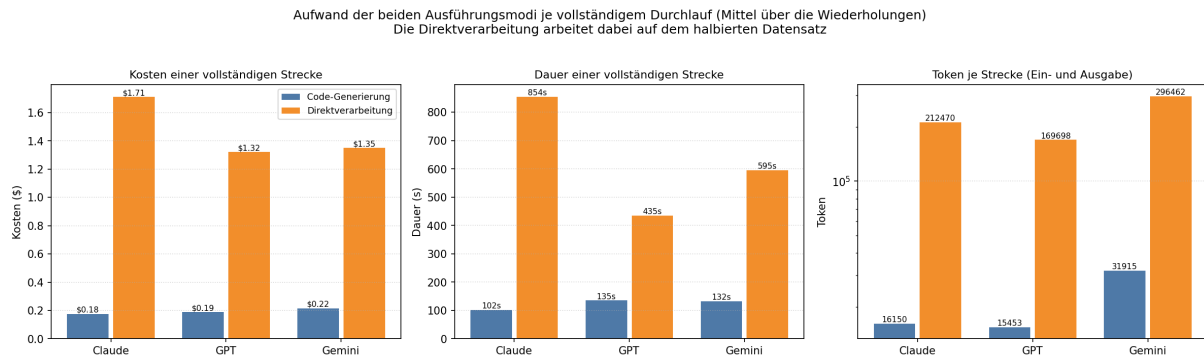


Abbildung 8.11: Kosten, Laufzeit und Token-Verbrauch einer vollständigen Strecke je Ausführungsmodus. Die Direktverarbeitung arbeitet dabei auf dem halbierten Datensatz.

zehnfache. Der Grund liegt in der Struktur der Verarbeitung: Im Code-Generierungs-Modus durchläuft nur die Aufgabenbeschreibung das Modell, während die Daten lokal verarbeitet werden; im Direkt-Modus wandert die vollständige Tabelle in jedem der neun Schritte einmal hinein und einmal wieder heraus. Entsprechend steigt der Token-Umsatz von rund 15.000 bis 32.000 auf 169.000 bis 285.000 je Strecke.

Der Token-Verbrauch macht zugleich einen Unterschied zwischen den Modellen sichtbar, der sich in der Genauigkeit nicht niederschlägt. Das Modell von Google benötigt im Code-Generierungs-Modus mit rund 28.000 Ausgabe-Token annähernd das Dreifache der beiden anderen Modelle, obwohl seine erzeugten Skripte nicht länger sind. Die Ursache sind die verborgenen Zwischenschritte des Modells, die als Ausgabe abgerechnet, aber nicht ausgeliefert werden. Für die Kostenplanung ist dies erheblich, weil sich der Aufwand eines solchen Modells nicht aus der Länge der sichtbaren Antwort abschätzen lässt: Bei der Auslegung des Ausgabebudgets im Direkt-Modus führte genau diese Eigenschaft dazu, dass Antworten abbrachen, obwohl die erwartete Tabelle rechnerisch weit in das gewählte Budget gepasst hätte, und erforderte dessen Anhebung auf 96.000 Token [20, 24].

In der Zusammenschau relativiert der Aufwand den in Abschnitt 8.8.1 beschriebenen Vorteil des Direkt-Modus erheblich. Sein Gewinn bei den semantischen Einzelschritten beträgt wenige Prozentpunkte, sein Verlust im zusammengesetzten Endschritt über 50 und sein Mehraufwand je verarbeiteter Zeile mehr als eine Größenordnung. Als durchgängiger Verarbeitungsmodus für eine vollständige ETL-Strecke ist er damit nicht zu begründen; sinnvoll einsetzbar ist er allein als gezielt vorgeschalteter Einzelschritt für genau jene Teilaufgaben, die Einzelfallverständnis verlangen.

8.6.4 Prompting-Strategien im Bestätigungslauf

Die dritte Untersuchungsebene des Bestätigungslaufs wiederholt die Faktormatrix aus Abschnitt 8.3 auf dem unbekannten Datensatz, beschränkt auf die drei klassischen Prompting-Strategien und die drei Cloud-Modelle. Die schema-angereicherte Variante wurde nicht erneut ausgeführt, da ihr in Abschnitt 8.3 belegter Einbruch nicht auf eine zufällige Schwankung, sondern auf einen benennbaren systematischen Mechanismus zurückgeht und eine Wiederholung folglich keinen zusätzlichen Erkenntnisgewinn verspricht. Es verbleiben 3 Strategien $\times$ 9 Aufgaben $\times$ 3 Modelle $\times$ 2 Wiederholungen, also 162 Läufe. Die Zahl der Wiederholungen wurde gegenüber dem explorativen Lauf von drei auf zwei verringert, da diese Ebene die Rangfolge der Strategien bestätigen soll und die Reproduzierbarkeit bereits in Abschnitt 8.5 über 432 Läufe sowie in Abschnitt 8.6.2 über fünf vollständige Streckendurchläufe bestimmt wurde; zwei Wiederholungen zeigen hier folglich an, ob ein beobachteter Strategieunterschied überhaupt Bestand hat, ohne die Streuungsaussage selbst tragen zu müssen.

Anders als bei der Pipeline ist hier kein gesonderter isolierter Durchlauf erforderlich, denn die Faktormatrix ist bereits ihrer Anlage nach isoliert: Jede Aufgabe liest unmittelbar die Rohdaten, und kein Ergebnis wird an eine folgende Aufgabe weitergereicht. Die in Abschnitt 8.4.2 untersuchte Fehlerfortpflanzung kann in dieser Ebene folglich gar nicht auftreten, weshalb die gemessenen Unterschiede vollständig der jeweiligen Strategie zuzurechnen sind.

Das Ergebnis bestätigt die in Abschnitt 8.3 vorgenommene Einordnung und widerlegt zugleich die dort zunächst berichtete Gesamtangfolge. Über alle Aufgaben gemittelt führt auf dem unbekannten Datensatz das Few-Shot-Prompting mit 77,9 Prozent vor dem Chain-of-Thought-Prompting mit 75,1 und dem Zero-Shot-Prompting mit 71,8 Prozent – also in genau umgekehrter Reihenfolge zu den 65,5, 64,1 und 60,9 Prozent des explorativen Laufs. Dieser Widerspruch ist jedoch kein Widerspruch der Sache, sondern eine Folge der Grundmenge: Der explorative Lauf umfasste das lokale Modell, der Bestätigungslauf aus den in Abschnitt 8.2 genannten Gründen nicht. Beschränkt man den explorativen Lauf ebenfalls auf die drei kommerziellen Modelle, liegen dessen Werte mit 72,6, 73,3 und 73,4 Prozent so eng beieinander, dass er zwischen den Strategien überhaupt nicht unterscheidet.

Was sich dagegen über beide Datensätze hinweg reproduziert, ist das aufgabenbezogene Muster, und zwar mit deutlich vergrößertem Abstand. Bei den unscharfen Du-

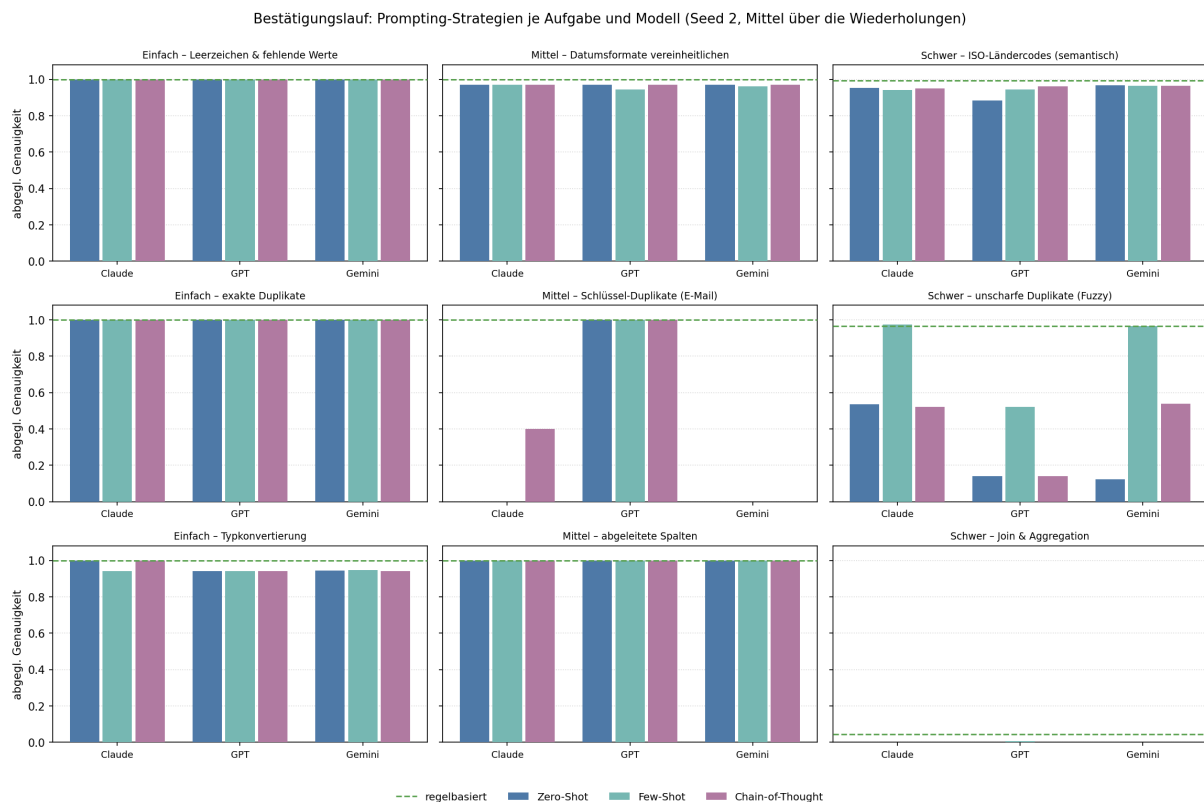


Abbildung 8.12: Abgegliche Genauigkeit der drei klassischen Prompting-Strategien je Aufgabe und Modell auf dem unbekannten Datensatz, gemittelt über zwei Wiederholungen. Die gestrichelte Linie markiert die regelbasierte Pipeline.

plikaten erreicht das Few-Shot-Prompting 82,0 Prozent gegenüber 40,0 für Chain-of-Thought und 26,6 für Zero-Shot; im explorativen Lauf lagen dieselben Werte bei 41,5, 21,3 und 30,7 Prozent. Bei den Schlüssel-Duplikaten führt erneut das Chain-of-Thought-Prompting mit 46,7 gegenüber jeweils 33,3 Prozent für die beiden anderen Strategien – dieselbe Konstellation wie zuvor mit 51,1 gegenüber 33,3. Die übrigen sieben Aufgaben unterscheiden auch hier nicht zwischen den Strategien: Die Bereinigungsaufgaben und die exakten Duplikate werden von allen drei nahezu fehlerfrei gelöst, die zusammengesetzte Aufgabe aus Join, Bereinigung und Aggregation von keiner – sie scheitert erneut in allen 18 Läufen. Die Wirkung der Prompt-Gestaltung ist damit reproduzierbar an die Aufgabenart gebunden, ihre Rangfolge über alle Aufgaben hinweg dagegen nicht [22, 24].

8.6.5 Prompting-Strategien ohne Fehlerfortpflanzung

Die bis hierhin berichteten Strategieunterschiede unterliegen einer Einschränkung, die aus dem Aufbau der Faktormatrix folgt. Jede Aufgabe wird dort auf den unveränderten Rohdaten gestellt, sodass die anspruchsvollen Stufen zusätzlich alle vorgelagerten Mängel mitbehandeln müssen. Bei der zusammengesetzten Aufgabe führt das dazu, dass jede Strategie gleichermaßen auf null fällt – eine Aufgabe, die alle Verfahren scheitern lässt, unterscheidet sie nicht mehr. Die Messung ist an dieser Stelle blind, und zwar nicht zufällig, sondern systematisch bei genau den Aufgaben, bei denen ein Strategieeffekt am ehesten zu erwarten wäre.

Um diese Blindstelle zu schließen, wurde eine vierte Untersuchungsebene ergänzt, die zwischen der Einzelaufgabe und der vollständigen Neunkette liegt. Die neun Pipeline-Schritte werden zu drei Gruppen von je drei Schritten zusammengefasst – Bereinigung, Deduplizierung und Transformation –, und jede Gruppe erhält die Soll-Ausgabe ihrer Vorgängergruppe als Eingabe. Die Anweisungen der drei zugehörigen Schritte werden dabei wörtlich aus der Pipeline übernommen und nummeriert in einem Prompt zusammengefasst, sodass die Gruppe inhaltlich dasselbe verlangt wie die drei Einzelschritte. Damit bleibt die Komposition mehrerer Verarbeitungsschritte als Anforderung erhalten – genau jene Eigenschaft, an der sich eine Strategie wie das Chain-of-Thought-Prompting bewähren sollte –, ohne dass ein Fehler aus einer früheren Gruppe das Ergebnis überdeckt. Ausgeführt wurden 3 Strategien × 3 Modelle × 3 Gruppen, also 27 Läufe.

Das Ergebnis ist eindeutig und in seiner Deutlichkeit unerwartet: Sämtliche 27 Läufe

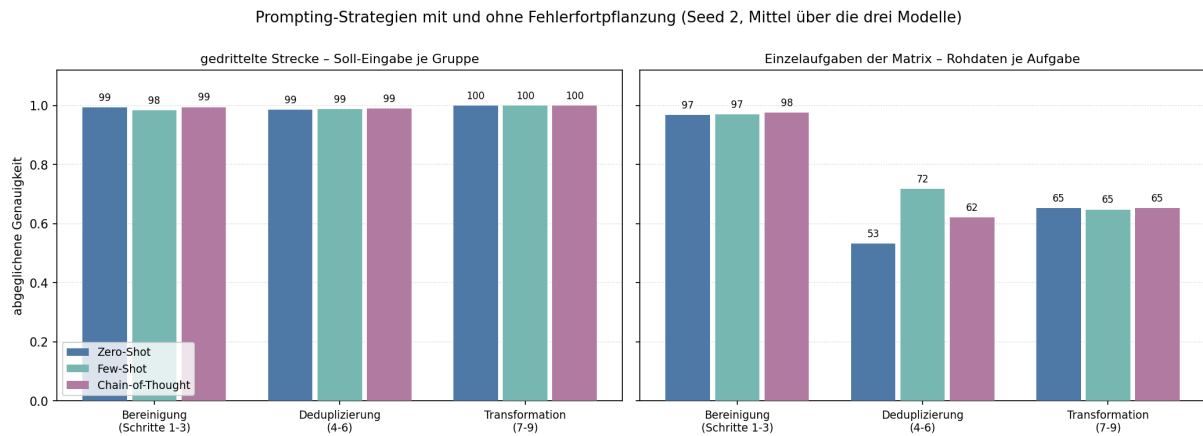


Abbildung 8.13: Dieselben Aufgaben als gedrittelte Strecke mit Soll-Eingabe je Gruppe (links) und als unabhängige Einzelaufgaben auf den Rohdaten (rechts), gemittelt über die drei Modelle.

lieferten ein verwertbares Ergebnis, und die Genauigkeit liegt durchgängig zwischen 98,4 und 100 Prozent. Über die drei Gruppen gemittelt erreicht das Chain-of-Thought-Prompting 99,5 Prozent, das Zero-Shot-Prompting 99,3 und das Few-Shot-Prompting 99,0 – eine Spanne von einem halben Prozentpunkt. Auch zwischen den Modellen ist kein belastbarer Unterschied mehr erkennbar. Besonders aufschlussreich ist die Transformationsgruppe: Sie enthält als letzten Schritt genau jene Aggregation, die als Einzelaufgabe in allen 18 Läufen scheiterte, und wird hier von jedem Modell in jeder Strategie und im ersten Versuch vollständig korrekt gelöst.

Abbildung 8.13 stellt beide Betrachtungsweisen unmittelbar gegenüber und macht damit die eigentliche Konsequenz sichtbar. Der auffälligste Strategieunterschied der Faktormatrix – 53, 72 und 62 Prozent bei der Duplikaterkennung – verschwindet vollständig, sobald dieselben Schritte mit sauberer Eingabe und in einem klar abgegrenzten Zuschnitt gestellt werden. Was in der Matrix als Wirkung der Prompt-Gestaltung erscheint, ist demnach zu einem erheblichen Teil eine Wirkung des Aufgabenzuschnitts: Beispiele und schrittweise Herleitung helfen einem Modell, eine überladene Aufgabe zu bewältigen, sind aber entbehrlich, sobald die Aufgabe angemessen zugeschnitten ist. Für die Beantwortung von FF2 ist das ein Befund von eigenständigem Gewicht, denn er kehrt die übliche Reihenfolge der Gestaltungsentscheidungen um: Nicht der Prompt ist zuerst zu optimieren, sondern der Zuschnitt der Aufgabe – und ist dieser einmal richtig gewählt, verliert die Wahl der Prompting-Strategie ihre Bedeutung weitgehend [1, 23].

Der Aufwand dieser Ebene unterstreicht den Befund zusätzlich. Die 27 Läufe kosteten

zusammen 1,27 US-Dollar bei 13,1 Minuten Laufzeit, und von den 27 Läufen benötigten 25 nur einen einzigen Versuch. Dem stehen 162 Läufe der Faktormatrix mit 3,14 US-Dollar gegenüber, von denen 37 kein verwertbares Ergebnis lieferten. Der kleinteiligere Zuschnitt ist damit nicht nur genauer, sondern zugleich günstiger und zuverlässiger – eine Kombination, die in dieser Untersuchung sonst an keiner Stelle auftritt.

8.7 Vergleich KI-basiert vs. klassisch

Abbildung 8.14 stellt die drei untersuchten Ansätze je Aufgabenkategorie gegenüber. Für die Code-Generierung ist dabei die jeweils beste klassische Prompting-Strategie ausgewiesen, sodass der Vergleich das jeweils Erreichbare und nicht den Durchschnitt einer beliebigen Konfiguration abbildet.

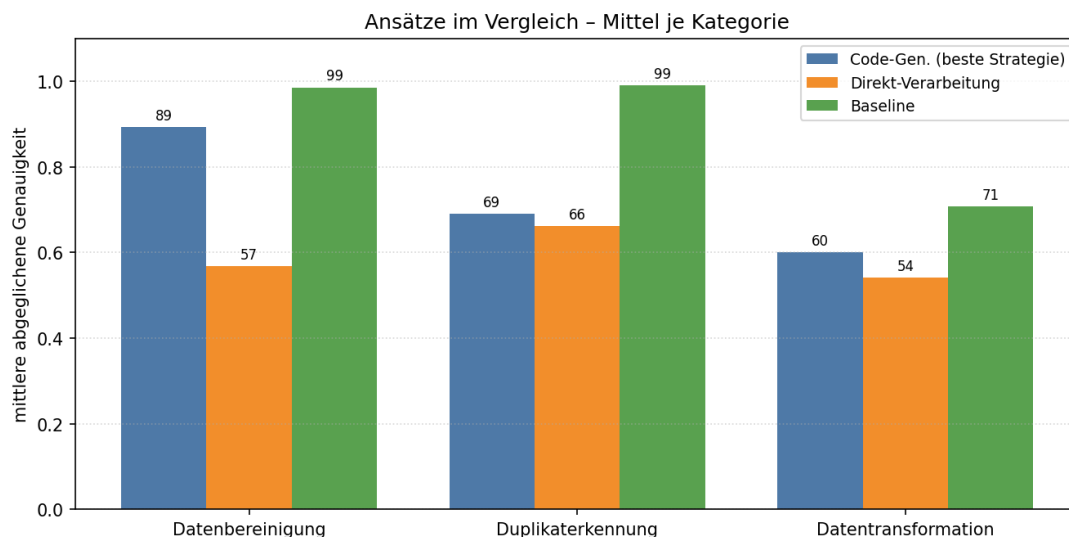


Abbildung 8.14: Mittlere Genauigkeit der drei Ansätze je Aufgabenkategorie. Für die Code-Generierung ist jeweils die beste klassische Prompting-Strategie ausgewiesen.

Das Ergebnis fällt für die KI-gestützten Verfahren zunächst ernüchternd aus: Die regelbasierte Pipeline liegt in allen drei Kategorien vorn, bei der Datenbereinigung mit 98,5 gegenüber 89,3 und 56,9 Prozent, bei der Duplikaterkennung mit 99,0 gegenüber 69,0 und 66,2 Prozent und bei der Datentransformation mit 70,7 gegenüber 60,1 und 54,1 Prozent. Dieser Befund ist jedoch differenziert zu lesen. Für die sechs formal beschreibbaren Aufgaben ist er weitgehend definitorisch bedingt: Wie in Abschnitt 6.3 dargelegt, verwendet die regelbasierte Pipeline für diese Aufgaben dieselben deterministischen Routinen, die auch der Erzeugung der Soll-Lösung zugrunde liegen; die

explizit formulierte Regel *ist* dort die korrekte Lösung. Ein Ergebnis von 100 Prozent ist folglich kein Leistungsnachweis, sondern die zu erwartende Untergrenze für ein Verfahren, dessen Vorschrift mit der Aufgabendefinition zusammenfällt. Interessant ist an diesen Aufgaben nicht, dass die Regel gewinnt, sondern dass die Modelle mit gut gewähltem Prompt gleichauf liegen, ohne dass die Regel zuvor formuliert werden musste.

Aussagekräftig ist der Vergleich daher erst bei den drei Aufgaben, für die die regelbasierte Pipeline bewusst mit begrenzten Heuristiken ausgestattet wurde. Bei der Länder-Vereinheitlichung erreicht sie 95,5 Prozent: Ihre Zuordnungstabelle deckt ISO-Codes in zwei- und dreistelliger Form, IOC-Kürzel sowie deutsche und englische Ländernamen ab, scheitert aber an 42 von 540 Datensätzen, nämlich an `England` (17), `Holland` (14) und dem Tippfehler `Deutshcland` (11). Genau hier liegt die Trennlinie zwischen beiden Verfahrensarten: Eine Zuordnungstabelle kann nur erfassen, was zuvor jemand aufgezählt hat, während ein Sprachmodell erschließt, dass „Holland“ umgangssprachlich für die Niederlande steht. Dass die Code-Generierung diesen Vorteil dennoch nicht ausspielt, liegt daran, dass sie das Weltwissen des Modells in genau dieselbe aufzählende Form zwingt und dabei ebenfalls Lücken lässt – lediglich andere. Erst der Direkt-Modus, in dem jeder Wert einzeln beurteilt wird, übertrifft die regelbasierte Lösung mit 96,4 bis 96,5 Prozent.

Noch klarer zeigt sich dieses Muster bei den unscharfen Duplikaten. Die regelbasierte Pipeline erreicht hier 96,9 Prozent, weil ihr normalisierter Schlüssel aus Name und E-Mail-Bestandteil genau jene Abweichungen erfasst, die durch Groß- und Kleinschreibung, zusätzliche Leerzeichen oder Punkte im lokalen Teil der Adresse entstehen – und der Datensatz eben diese Muster enthält. Die Code-Generierung bleibt mit 69,0 Prozent deutlich zurück, der Direkt-Modus erreicht mit bis zu 96,5 Prozent nahezu das Niveau der Regel. Diese Konstellation macht zugleich eine Grenze der Untersuchungsanlage sichtbar, die in Kapitel 10 aufzugreifen ist: Die eingebauten Abweichungen entstammen einer bekannten, endlichen Menge von Mustern, was regelbasierte Verfahren systematisch begünstigt. In realen Beständen, deren Abweichungsmuster nicht vorab bekannt sind, verschiebt sich dieses Verhältnis zugunsten der semantischen Verfahren [12, 13].

Der dritte aussagekräftige Fall ist die zusammengesetzte Aufgabe aus Join, Bereinigung und Aggregation, bei der auch die regelbasierte Pipeline mit 12,1 Prozent nahezu vollständig scheitert – und zwar aus derselben Ursache wie die Modelle: Ihre begrenzt-

te Zuordnungstabelle lässt 57 Länderangaben als unbekannt zurück, die anschließend eine eigene, in der Soll-Lösung nicht vorgesehene Gruppe bilden und die Aggregation über alle betroffenen Kategorien verfälschen. Die Aufgabe trennt somit nicht zwischen den Ansätzen, sondern zeigt, dass ein Fehler in einem frühen semantischen Schritt in einer zusammengesetzten Verarbeitung unabhängig vom gewählten Verfahren dieselbe verstärkende Wirkung entfaltet. In der zerlegten Kette hingegen lag die regelbasierte Pipeline mit 60,6 Prozent im Endschritt hinter allen drei Cloud-Modellen, weil sie mehr Bestellungen ohne zugehörigen Kunden zurückließ.

Zusammengefasst ergibt sich kein Verfahren, das über alle Aufgaben hinweg vorzuziehen wäre. Für formal beschreibbare Aufgaben ist die regelbasierte Umsetzung der KI-gestützten mindestens ebenbürtig und in Kosten, Geschwindigkeit und Determiniertheit deutlich überlegen – 0,17 Sekunden ohne Nutzungskosten gegenüber gut zwei Minuten und rund 0,19 US-Dollar für dieselbe Strecke. Ihr Vorteil endet jedoch genau dort, wo die Verarbeitungsvorschrift nicht mehr vollständig vorab formulierbar ist, und in diesem Bereich liegt der Beitrag der Modelle weniger im Ersetzen als im Ergänzen der Regel [9, 32].

8.8 Diskussion der Ergebnisse

8.8.1 Stärken des LLM-Einsatzes

Die deutlichste Stärke der untersuchten Modelle liegt nicht in der erreichten Genauigkeit, sondern im Aufwand, mit dem sie erreicht wird. Die regelbasierte Vergleichspipeline setzt voraus, dass sämtliche Verarbeitungsvorschriften einschließlich der Zuordnungstabelle für Länderangaben und der Normalisierungsregel für Duplikate zuvor formuliert werden; ihre hohe Ergebnisqualität ist das Resultat dieser vorgelagerten Arbeit. Die Modelle erreichen bei den formal beschreibbaren Aufgaben mit gut gewähltem Prompt vergleichbare Werte, ohne dass eine solche Spezifikation vorliegt, und zwar in Sekunden und zu Kosten im Bereich weniger Cent. Für Vorhaben, in denen die Verarbeitungslogik noch nicht feststeht oder sich häufig ändert, verschiebt dies die Aufwandsrechnung erheblich [9, 36].

Die zweite Stärke betrifft die semantisch geprägten Aufgaben, allerdings in einer präziseren Form als zu Beginn erwartet. Der Vorteil des Sprachverständnisses tritt nur dann zutage, wenn das Modell sein Wissen auf den einzelnen Wert anwenden kann, wie es der Direkt-Modus zulässt: Bei den unscharfen Duplikaten steigt die Genauigkeit

dort von 69,0 auf bis zu 96,5 Prozent, bei der Länder-Vereinheitlichung von 95,8 auf 96,5 Prozent und damit erstmals über die regelbasierte Lösung. Sobald dieselbe Aufgabe in generierten Code gefasst werden muss, geht dieser Vorteil weitgehend verloren, weil das Modell sein Wissen in eine aufzählende Tabelle oder einen numerischen Schwellenwert übersetzen muss und damit auf die Ausdrucksmittel des regelbasierten Verfahrens zurückfällt. Der Ausführungsmodus ist somit kein technisches Detail, sondern bestimmt maßgeblich, ob die semantische Fähigkeit des Modells überhaupt wirksam werden kann [7, 25].

Als dritte Stärke erweist sich die Zerlegbarkeit der Aufgabenstellung. Der Vergleich zwischen verketteter Pipeline und unabhängigem Einzellauf in Abbildung 8.6 zeigt, dass dieselben Modelle bei kleinteilig gestellten Schritten erheblich besser abschneiden, in einzelnen Schritten um über 50 Prozentpunkte. Die vollständig gescheiterte zusammengesetzte Aufgabe wird in zerlegter Form von allen vier Modellen fehlerfrei gelöst. Nicht die Schwierigkeit der einzelnen Operation begrenzt die Leistungsfähigkeit, sondern die Zahl der Entscheidungen, die in einem Arbeitsschritt gleichzeitig zu treffen sind – eine Eigenschaft, die sich durch den Zuschnitt der Aufgabenstellung unmittelbar beeinflussen lässt.

8.8.2 Schwächen und Risiken

Das gravierendste Risiko ist das stille Versagen. Der in Skript 8.1 dokumentierte Fall zeigt einen erzeugten Code, der fehlerfrei durchläuft, eine schemakonforme Ausgabe erzeugt und dennoch nichts bewirkt. Weder die Erfolgsquote noch eine Schemaprüfung noch eine Sichtprüfung des Codes hätten diesen Fehler aufgedeckt; sichtbar wurde er allein durch den Abgleich mit einer bekannten Soll-Lösung, die im produktiven Einsatz gerade nicht vorliegt. Verschärfend kommt hinzu, dass die Ursache – der Rückgriff auf ein in den Trainingsdaten dominantes, inzwischen überholtes Idiom – systematisch und damit reproduzierbar ist. Jede Produktivnutzung generierten ETL-Codes setzt folglich eine unabhängige Ergebnisprüfung voraus, die nicht auf der Abwesenheit von Fehlermeldungen beruhen darf [7, 17].

Ein zweites Risiko liegt in der Fehlerfortpflanzung. Der isolierte Kettendurchlauf hat gezeigt, dass ein Schritt, der für sich betrachtet 94,6 Prozent erreicht und damit unauffällig wirkt, drei Schritte später einen vollständigen Abbruch auslösen kann. Die üblichen Qualitätskennzahlen eines einzelnen Schritts sind daher kein verlässlicher Indikator für die Verwendbarkeit seines Ergebnisses in nachgelagerten Schritten, da nicht der

Umfang, sondern die Art der verbliebenen Abweichung entscheidet. Ein Ergebnis mit wenigen, aber typfremden Restwerten kann schädlicher sein als eines mit vielen, aber gleichartigen Abweichungen.

Das dritte Risiko ist die begrenzte Reproduzierbarkeit, die sich, wie in Abschnitt 8.5 gezeigt, ausgerechnet bei den schwierigsten Aufgaben am stärksten ausprägt. Hinzu tritt die Abhängigkeit von serverseitig gepflegten Modellen, deren Verhalten sich ohne Zutun der Anwendenden ändern kann; diesem Umstand wurde durch die Protokollierung der exakten Versionskennung Rechnung getragen, er lässt sich damit jedoch nur dokumentieren und nicht beheben [6, 20].

Schließlich ist die Erwartung zu korrigieren, mehr Kontext im Prompt führe zu besseren Ergebnissen. Die schema-angereicherte Strategie hat die Genauigkeit bei der Duplikaterkennung auf ein Viertel der übrigen Strategien gesenkt, und das aufwändige Few-Shot-Prompting verursachte den dreifachen Token-Aufwand ohne Qualitätsgewinn. Die Gestaltung des Prompts ist damit ein zweischneidiges Werkzeug, dessen Wirkung sich nicht aus dem investierten Aufwand ableiten lässt, sondern empirisch je Aufgabenart zu bestimmen ist [23, 24].

8.8.3 Einordnung in den Stand der Forschung

Die Ergebnisse bestätigen die in Kapitel 3 referierte grundsätzliche Einschätzung, dass LLMs für Aufgaben der Datenbereinigung und -integration nutzbar sind, präzisieren sie jedoch in einem für die Praxis wesentlichen Punkt. Während die Literatur den Nutzen überwiegend konzeptionell oder anhand einzelner Anwendungsfälle beschreibt, zeigt der hier durchgeführte kontrollierte Vergleich, dass die Eignung nicht vom Modell, sondern vom Zusammenspiel aus Aufgabentyp, Ausführungsmodus und Aufgabenschnitt abhängt [7, 9]. Die Spreizung zwischen den Aufgaben übersteigt jene zwischen den Modellen um ein Vielfaches, was der verbreiteten Orientierung an Modellranglisten eine begrenzte Aussagekraft für konkrete ETL-Vorhaben bescheinigt [20, 21].

Hypothese H1 gilt damit in ihrem Kern als bestätigt und in einem Teilaspekt als eingeschränkt bestätigt. Bestätigt ist die starke Abhängigkeit vom Aufgabentyp, die Ebenbürtigkeit bei formal beschreibbaren Aufgaben, der Verlust an Zuverlässigkeit mit zunehmender struktureller Komplexität sowie die Aussage, dass kein Ansatz über alle Aufgaben hinweg überlegen ist. Der erwartete Vorteil bei semantisch geprägten Aufgaben trat hingegen nur im Direkt-Modus ein, während er im primären Modus der

Code-Generierung ausblieb – eine Präzisierung, die in der Literatur bislang kaum Beachtung findet, weil dort die Wahl des Ausführungsmodus selten als eigenständige Gestaltungsdimension behandelt wird [4, 25].

Hypothese H2 ist in ihrer allgemeinen Form zu verwerfen. Aufwändigere Prompting-Strategien verbessern die Ergebnisqualität gegenüber dem Zero-Shot-Prompt nicht durchgängig; über alle Läufe hinweg schneidet der einfachste Prompt am besten ab, und die aufwändigste Variante fällt deutlich zurück. Zutreffend bleibt der erwartete abnehmende Grenznutzen, allerdings in verschärfter Form: Der Nutzen ist nicht nur abnehmend, sondern bei unpassendem Zuschnitt negativ. Bestätigt hat sich zugleich der in der Literatur beschriebene aufgabenabhängige Charakter der Wirkung, denn das Chain-of-Thought-Prompting führte bei den mehrstufigen Aufgaben der Duplikaterkennung und Transformation, während es bei den einstufigen Bereinigungsaufgaben schädete [19, 22].

Über die beiden Hypothesen hinaus liefert die verkettete Betrachtung einen Beitrag, der in der Literatur bislang wenig ausgeprägt ist. Die getrennte Messung von eigener Schrittleistung und geerbtem Fehler zeigt, dass ein erheblicher Teil der in Einzelbetrachtungen berichteten Schwächen gar nicht dem bewerteten Schritt zuzurechnen ist: Der Endschrift der Kette, der in der verketteten Ausführung zwischen 71,2 und 81,8 Prozent erreicht, ist mit fehlerfreier Eingabe von allen vier Modellen vollständig lösbar. Evaluierungen, die einzelne ETL-Operationen isoliert betrachten, überschätzen die Leistungsfähigkeit in Bezug auf reale Strecken daher ebenso, wie rein verkettete Betrachtungen sie unterschätzen; erst die Gegenüberstellung beider Betrachtungsweisen erlaubt eine belastbare Zuordnung [2, 10].

Ein methodischer Beitrag ergibt sich schließlich aus dem in Abschnitt 8.6 berichteten Bestätigungslauf. Er zeigt, dass die inhaltlichen Befunde auf einem unbekannten Datensatz Bestand haben, macht zugleich aber sichtbar, wie wenig belastbar Einzelmessungen an verketteten Strecken sind: Der Endschrift derselben Konfiguration schwankte über fünf identische Wiederholungen um bis zu 45 Prozentpunkte, während er im isolierten Durchlauf in jeder Wiederholung exakt denselben Wert lieferte. Vergleichende Bewertungen von LLMs in ETL-Strecken, die auf einem einzelnen Durchlauf beruhen – wie es in der referierten Literatur überwiegend der Fall ist –, messen unter diesen Bedingungen zu einem erheblichen Teil Zufall statt Leistungsfähigkeit [10, 20].

Auf dieser Grundlage lassen sich im folgenden Kapitel 9 konkrete Empfehlungen für die

Auswahl zwischen klassischem, KI-gestütztem und hybridem Vorgehen je Aufgabentyp ableiten, wie sie FF1 einfordert, ebenso wie Empfehlungen zur Gestaltung der Prompts im Sinne von FF2.

9. Handlungsempfehlungen für die Praxis

Die in Kapitel 8 dargestellten Befunde erlauben eine Reihe von Empfehlungen, die über die Beantwortung der Forschungsfragen hinaus unmittelbar auf die Gestaltung KI-gestützter ETL-Strecken anwendbar sind. Sie werden im Folgenden entlang dreier Entscheidungen entwickelt, die in einem konkreten Vorhaben nacheinander zu treffen sind: ob ein LLM für eine gegebene Aufgabe überhaupt das geeignete Mittel ist (Abschnitt 9.1), wie Modell, Ausführungsmodus und Prompt zu wählen sind, wenn die Entscheidung dafür gefallen ist (Abschnitt 9.2), und wie sich beide Verfahrensarten in einer Gesamtarchitektur so verbinden lassen, dass die Stärken der einen die Schwächen der anderen auffangen (Abschnitt 9.3).

Vorauszuschicken ist eine Einschränkung, die für alle folgenden Empfehlungen gilt. Sie beruhen auf einem synthetischen Datensatz mit bekannten, endlich vielen Abweichungsmustern und auf drei kommerziellen sowie einem lokal betriebenen Modell zu einem bestimmten Zeitpunkt. Die konkreten Zahlenwerte sind daher nicht zu übertragen; übertragbar sind die Wirkungszusammenhänge, aus denen sie entstehen, sowie die Verfahren, mit denen sich die Werte im eigenen Kontext bestimmen lassen.

9.1 Eignung von LLMs für unterschiedliche ETL-Szenarien

Der über alle Untersuchungsebenen hinweg stabilste Befund dieser Arbeit ist, dass die Eignung eines LLM weit stärker von der Art der Aufgabe abhängt als von der Wahl des Modells. Zwischen dem besten und dem schwächsten Cloud-Modell liegen rund zehn Prozentpunkte, zwischen der am besten und der am schlechtesten gelösten Aufgabe hingegen mehr als neunzig. Die erste Frage in einem ETL-Vorhaben lautet daher nicht, welches Modell einzusetzen ist, sondern für welche Teilaufgaben sich der Einsatz überhaupt lohnt [9, 20].

Für diese Entscheidung hat sich in den Ergebnissen eine einzige Unterscheidung als tragfähig erwiesen: ob die Verarbeitungsvorschrift vollständig vorab formulierbar ist oder nicht. Aufgaben der ersten Art – das Entfernen von Leerzeichen, das Vereinheitlichen bekannter Datumsformate, das Entfernen exakter Duplikate, Typkonvertierungen, das Berechnen abgeleiteter Spalten – werden von einer regelbasierten Implementierung definitionsgemäß korrekt gelöst, deterministisch, in Sekundenbruchteilen und ohne laufende Kosten. Ein LLM erreicht hier bestenfalls dasselbe Ergebnis und bringt zusätzlich Nichtdeterminismus, Kosten und Prüfaufwand mit sich. Sein Beitrag liegt bei diesen Aufgaben ausschließlich in der *Erstellung* der Regel, nicht in ihrer Ausführung – ein Unterschied, der in der Diskussion über KI-gestützte Datenverarbeitung häufig verwischt wird [13, 36].

Aufgaben der zweiten Art sind dadurch gekennzeichnet, dass die Vorschrift eine Aufzählung erfordern würde, die niemand vollständig angeben kann. Die Vereinheitlichung der Länderangaben ist der klarste Fall: Eine Zuordnungstabelle erfasst nur, was zuvor jemand eingetragen hat, und scheiterte in der regelbasierten Vergleichspipeline an „England“, „Holland“ und dem Tippfehler `Deutshccland`. Genau hier liegt der originäre Beitrag des Modells, das erschließt, dass „Holland“ umgangssprachlich für die Niederlande steht, ohne dass diese Gleichsetzung je hinterlegt wurde. Dasselbe gilt für die unscharfe Duplikaterkennung, bei der die Entscheidung über zwei konkrete Datensätze einem Modell unmittelbar zugänglich ist, während ihre Verallgemeinerung zu einem Schwellenwert Information vernichtet.

Diese Zuordnung bleibt jedoch unvollständig, solange sie den Ausführungsmodus außer Acht lässt, denn der Vorteil bei semantischen Aufgaben tritt nur dann ein, wenn das Modell sein Wissen auf den einzelnen Wert anwenden kann. In generierten Code gefasst, fiel dieselbe Aufgabe auf das Niveau der regelbasierten Lösung zurück, weil das Modell gezwungen war, sein Wissen in eine ebenfalls unvollständige Aufzählung zu übersetzen. Die Eignungsfrage ist deshalb nicht mit „LLM oder Regel“ zu beantworten, sondern erst mit der Angabe, in welchem Modus das Modell arbeiten soll. Tabelle 9.1 fasst die daraus folgende Zuordnung zusammen.

Die letzte Zeile dieser Tabelle verdient besondere Beachtung, weil sie den folgenreichsten Gestaltungshebel benennt. Die zusammengesetzte Aufgabe aus Join, Bereinigung und Aggregation wurde im Code-Generierungs-Modus von keinem Modell und in keiner Strategie gelöst; dieselben Modelle lösten sie in zerlegter Form mit fehlerfreier Eingabe vollständig korrekt. Nicht die Schwierigkeit der einzelnen Operation

Tabelle 9.1: Empfohlenes Verfahren je Aufgabentyp auf Grundlage der Ergebnisse aus Kapitel 8.

Aufgabentyp	Merkmal	Empfehlung
Syntaktisch regelbar	Vorschrift vollständig aufzählbar	Regelbasiert ausführen; LLM allenfalls zur Erstellung des Codes
Semantisch, wertweise	Entscheidung je Einzelwert, Verallgemeinerung verlustbehaftet	LLM im Direkt-Modus, gezielt als einzelner Schritt
Rechnerisch aggregierend	viele Datensätze, exakte Arithmetik	Regelbasiert oder LLM im Code-Generierungs-Modus; Direkt-Modus ungeeignet
Zusammengesetzt	mehrere Entscheidungen in einem Schritt	In Einzelschritte zerlegen, dann je Schritt entscheiden

begrenzt die Leistungsfähigkeit, sondern die Zahl der Entscheidungen, die gleichzeitig zu treffen sind. Bevor ein Vorhaben ein Modell auswählt oder einen Prompt optimiert, sollte es daher seine Aufgabenstellung zerlegen – das ist der Eingriff mit dem mit Abstand größten Ertrag [1, 4].

9.2 Empfehlungen zur Modell- und Prompt-Auswahl

Steht fest, dass ein LLM eingesetzt werden soll, sind drei Festlegungen zu treffen. Sie werden hier in der Reihenfolge ihrer Wirkung behandelt, die der verbreiteten Aufmerksamkeitsverteilung genau entgegengesetzt ist: Der Ausführungsmodus wirkt am stärksten, die Prompt-Gestaltung deutlich, die Modellwahl am wenigsten.

Ausführungsmodus. Die Wahl zwischen Code-Generierung und Direktverarbeitung ist keine technische Detailfrage, sondern bestimmt, welche Fähigkeit des Modells überhaupt zur Wirkung kommt. Für den Regelbetrieb ist der Code-Generierungs-Modus vorzuziehen, und zwar aus drei unabhängigen Gründen. Er ist um mehr als eine Größenordnung günstiger – 0,18 bis 0,22 US-Dollar gegenüber 1,27 bis 1,70 für dieselbe Strecke auf der halben Datenmenge. Er erzeugt ein prüfbares, versionierbares Artefakt, das sich einmal abnehmen und danach unverändert ausführen lässt, während die Direktverarbeitung bei jedem Lauf neu und potenziell anders entscheidet. Und er skaliert unabhängig vom Datenvolumen, während der Direkt-Modus durch das Ausgabefenster begrenzt ist und im Rahmen dieser Arbeit eine Halbierung des Datensatzes

erzwang. Der Direkt-Modus ist demnach nicht als durchgängiger Verarbeitungsmodus zu empfehlen, sondern gezielt für einzelne, semantisch geprägte Schritte – dort allerdings mit messbarem Gewinn.

Prompt-Gestaltung. Hier ist die wichtigste Empfehlung eine über die Reihenfolge: Der Zuschnitt der Aufgabe ist vor dem Prompt zu optimieren, nicht danach. Die in Abschnitt 8.6.5 berichtete Gegenüberstellung zeigt, dass der auffälligste Strategieunterschied der Faktormatrix – 53 gegenüber 72 Prozent bei der Duplikaterkennung – vollständig verschwindet, sobald dieselben Schritte in angemessenem Zuschnitt und mit sauberer Eingabe gestellt werden: Dort liegen alle drei Strategien zwischen 99,0 und 99,5 Prozent. Beispiele und schrittweise Herleitung helfen einem Modell, eine überladene Aufgabe zu bewältigen; sie werden entbehrlich, sobald die Aufgabe nicht mehr überladen ist. Wer zuerst am Prompt feilt, optimiert daher an einem Symptom.

Wo eine Aufgabe sich nicht weiter zerlegen lässt, gilt die Wirkung als aufgabengebunden und nicht als allgemein. Über beide Datensätze hinweg reproduzierbar sind genau zwei Zuordnungen: Beispiele wirken bei Entscheidungen über Ähnlichkeit, wie sie die unscharfe Duplikaterkennung verlangt – dort führte das Few-Shot-Prompting mit 82,0 gegenüber 26,6 Prozent für den Zero-Shot-Prompt; die Aufforderung zur schrittweisen Herleitung wirkt bei mehrstufigen Auswahlregeln wie der schlüsselbasierten Entdoppelung. Bei den übrigen sieben Aufgaben war zwischen den Strategien kein Unterschied messbar, sodass dort der einfachste und billigste Prompt zu wählen ist. Ausdrücklich zu warnen ist vor Kontext, der über die Aufgabe hinausweist: Die schema-angereicherte Variante senkte die Genauigkeit bei der Duplikaterkennung auf ein Viertel, weil die Modelle die genannten Zieltypen als Handlungsauftrag auffassten. Zusätzlicher Kontext wirkt, wenn er einen konkreten beobachteten Mangel adressiert – die Angabe der Eingabe-Datentypen behob ein stilles Versagen der Spaltenauswahl –, und schadet, wenn er darüber hinausgeht [23, 24].

Modellwahl. Die drei untersuchten Cloud-Modelle liegen so eng beieinander, dass sich eine allgemeine Empfehlung nicht begründen lässt; auch die Kosten und Antwortzeiten unterscheiden sich nur moderat. Sinnvoll ist stattdessen eine Auswahl anhand der konkret vorliegenden Aufgabenart, wobei die Modelle in ihren Fehlermustern durchaus unterscheidbar sind. In der gedrittelten Strecke, also bei angemessenem Aufgabenzuschnitt, verschwindet auch dieser Unterschied: Dort liegen alle drei zwischen

98,5 und 100 Prozent.

Für das lokal betriebene Modell gilt eine eigene Empfehlung. Sein in Abschnitt 8.2 berichteter Rückstand ist maßgeblich durch die verfügbare Hardware bedingt und keine Aussage über lokal betreibbare Modelle im Allgemeinen; er entstand zudem weit überwiegend daraus, überhaupt lauffähigen Code zu erzeugen, und nicht aus einem mangelnden Verständnis der Aufgaben. Für Vorhaben, in denen Daten das eigene Netz nicht verlassen dürfen oder in denen dauerhaft hohe Volumina verarbeitet werden, ist eine eigene Erprobung auf leistungsfähigerer Hardware daher lohnend – der Wegfall laufender Nutzungskosten wiegt bei anhaltendem Betrieb schwer. Die Bedingungen dafür lassen sich aus den Ergebnissen benennen: ausschließlich im Code-Generierungs-Modus, mit kleinteilig zugeschnittenen Aufgaben und mit mehreren zugestandenem Versuchen je Schritt. Unter genau diesen Bedingungen bewältigte auch das hier eingesetzte, hardwarebegrenzte Modell im isolierten Kettendurchlauf alle neun Schritte [9, 20].

Messung statt Übernahme. Keine dieser Empfehlungen ersetzt die Messung im eigenen Kontext, und der in Abschnitt 8.6 berichtete Bestätigungslauf zeigt, welche Anforderungen eine solche Messung erfüllen muss. Eine Konfiguration ist an Daten zu bewerten, gegen die sie nicht justiert wurde, und mehrfach auszuführen: Der Endschritt derselben Konfiguration schwankte über fünf identische Wiederholungen zwischen 27,3 und 72,7 Prozent. Wer auf dieser Grundlage eine einzelne Messung zur Entscheidung heranzieht, wählt zu einem erheblichen Teil zufällig.

9.3 Hybride Architekturen

Aus den vorstehenden Empfehlungen ergibt sich, dass weder ein rein regelbasiertes noch ein rein KI-gestütztes Vorgehen der jeweils anderen Variante über alle Aufgaben hinweg überlegen ist. Die naheliegende Konsequenz ist eine Architektur, die beide Verfahren nicht als Alternativen, sondern als Bausteine derselben Strecke behandelt und je Schritt dasjenige einsetzt, dessen Stärken zur Aufgabe passen [9, 32].

Der erste Baustein einer solchen Architektur ist die Zerlegung. Die Untersuchung hat gezeigt, dass dieselbe Strecke als Folge klar abgegrenzter Schritte erheblich besser abschneidet denn als eine zusammengesetzte Aufgabe, in einzelnen Schritten um über fünfzig Prozentpunkte. Die Zerlegung ist damit nicht nur eine Frage der Übersichtlich-

keit, sondern die Voraussetzung dafür, dass je Schritt überhaupt eine Verfahrenswahl möglich wird – und zugleich die einzige Stelle, an der sich Qualität ohne zusätzliche Kosten gewinnen lässt.

Der zweite Baustein ist die Zuordnung nach Tabelle 9.1: regelbasierte Ausführung für die syntaktisch beschreibbaren Schritte, ein Modell im Direkt-Modus für die wenigen semantisch geprägten Schritte, und die abschließende Aggregation wieder regelbasiert oder über generierten Code. Für die untersuchte Strecke bedeutet dies konkret, dass sieben der neun Schritte regelbasiert auszuführen wären und lediglich die Länder-Vereinheitlichung sowie die unscharfe Duplikaterkennung ein Modell benötigen. Der Kostenanteil einer solchen Architektur liegt damit bei einem Bruchteil des durchgängigen LLM-Einsatzes, während sie dessen Vorteil genau dort behält, wo er entsteht.

Der dritte und wichtigste Baustein ist die Prüfung nach jedem Schritt. Sie folgt unmittelbar aus dem Befund zur Fehlerfortpflanzung: Ein Schritt, der für sich betrachtet 94,6 Prozent erreicht und damit unauffällig wirkt, löste drei Schritte später einen vollständigen Abbruch aus, und der Endschrift der Kette verlor bis zu 50 Prozentpunkte, die vollständig geerbt und nicht ihm selbst zuzurechnen waren. Entscheidend ist dabei, dass diese Prüfung nicht auf der Abwesenheit von Fehlermeldungen beruhen darf. Der in Skript 8.1 dokumentierte Fall zeigt einen Code, der fehlerfrei durchläuft, eine schemakonforme Ausgabe erzeugt und dennoch nichts bewirkt; weder Erfolgsquote noch Schemaprüfung hätten ihn aufgedeckt. Zu prüfen sind daher inhaltliche Eigenschaften des Ergebnisses – etwa dass die Länderspalte ausschließlich zweistellige ISO-Codes enthält, dass die Datensatzanzahl im erwarteten Bereich liegt oder dass keine Bestellung ohne zugehörigen Kunden zurückbleibt [12, 17].

Der vierte Baustein betrifft den Umgang mit dem Nichtdeterminismus und ergibt sich aus dem Bestätigungslauf. Da die Streuung des Endergebnisses fast vollständig aus der Verstärkung entlang der Kette und nicht aus der Textgenerierung selbst entstand – im isolierten Durchlauf lieferten alle Wiederholungen exakt denselben Wert –, ist sie durch Festschreibung beherrschbar. Sowohl der einmal geprüfte Code als auch die Zwischenergebnisse der frühen, semantisch geprägten Schritte sind als feste Artefakte zu übernehmen, statt sie bei jeder Ausführung neu erzeugen zu lassen. Ein Modell wird damit zum Werkzeug der Entwicklungszeit, dessen Ergebnis in den Betrieb übernommen wird, statt zu einer Komponente des Betriebs, die bei jedem Lauf neu entscheidet.

Diese Aufteilung beschreibt zugleich die realistische Rolle von LLMs in der Datenintegration, wie sie sich aus den Ergebnissen dieser Arbeit ergibt. Sie ersetzen die klassische Verarbeitung nicht, sondern verschieben die Grenze dessen, was ohne vorherige vollständige Spezifikation automatisierbar ist. Ihr Beitrag liegt in den wenigen Schritten, in denen eine Vorschrift nicht vollständig formulierbar ist, sowie in der Verkürzung der Zeit, die von der Aufgabenstellung bis zu einer lauffähigen ersten Umsetzung vergeht – und beides ist erheblich, ohne die Aufgabe der prüfenden Instanz zu ersetzen [2, 25].

10. Fazit und Ausblick

10.1 Zusammenfassung der Ergebnisse

Ziel dieser Arbeit war eine experimentelle Evaluierung, inwieweit sich aktuelle LLMs für ausgewählte ETL-Teilaufgaben eignen. Dazu wurde ein synthetischer Datensatz mit kontrolliert eingebauten Qualitätsproblemen erstellt, neun ETL-Aufgaben in drei Kategorien und drei Schwierigkeitsstufen definiert und eine regelbasierte Vergleichspipeline implementiert, deren Ergebnis als Maßstab dient. Die Modelle wurden auf drei Untersuchungsebenen gemessen: in einer vollständig gekreuzten Faktormatrix aus Modell, Prompting-Strategie und Aufgabe, in einer verketteten Pipeline aus neun aufeinander aufbauenden Schritten sowie in einem sonst identischen isolierten Durchlauf, der jedem Schritt fehlerfreie Eingabedaten bereitstellt und damit eigene Leistung von geerbtem Fehler trennt. Ergänzend wurden zwei Ausführungsmodi gegenübergestellt: die Erzeugung von Verarbeitungscode und die unmittelbare Verarbeitung der Daten im Prompt. Alle Befunde wurden abschließend auf einem zweiten, während der Entwicklung unbekannten Datensatz mit mehreren Wiederholungen nachgeprüft.

Der über alle Ebenen hinweg stabilste Befund ist die Dominanz des Aufgabentyps. Zwischen dem besten und dem schwächsten kommerziellen Modell liegen rund zehn Prozentpunkte, zwischen der am besten und der am schlechtesten gelösten Aufgabe mehr als neunzig. Für ein konkretes ETL-Vorhaben ist die Frage, welche Teilaufgaben einem Modell übertragen werden, damit erheblich folgenreicher als die Frage, welches Modell dafür gewählt wird – ein Ergebnis, das der verbreiteten Orientierung an Modellranglisten eine begrenzte Aussagekraft bescheinigt.

Als zweiter Befund erwies sich der Ausführungsmodus als eigenständige und bislang

unterschätzte Gestaltungsdimension. Der erwartete Vorteil der Modelle bei semantisch geprägten Aufgaben trat nur in der direkten Verarbeitung ein, in der das Modell sein Wissen auf jeden einzelnen Wert anwendet. Sobald dieselbe Aufgabe in generierten Code zu fassen war, fiel das Ergebnis auf das Niveau der regelbasierten Lösung zurück, weil das Modell sein Wissen in eine ebenfalls unvollständige Aufzählung oder einen numerischen Schwellenwert übersetzen musste. Umgekehrt scheiterte die direkte Verarbeitung an der abschließenden Aggregation über mehrere hundert Datensätze nahezu vollständig und kostete je verarbeiteter Zeile rund das Fünfzehnfache. Die beiden Modi sind damit nicht besser oder schlechter, sondern für unterschiedliche Aufgabenarten geeignet.

Der dritte Befund betrifft den Zuschnitt der Aufgabenstellung. Die zusammengesetzte Aufgabe aus Join, Bereinigung und Aggregation wurde im Code-Generierungs-Modus von keinem Modell und in keiner Strategie gelöst; dieselben Modelle lösten sie als Glied einer Kette mit fehlerfreier Eingabe vollständig korrekt. Nicht die Schwierigkeit der einzelnen Operation begrenzt die Leistungsfähigkeit, sondern die Zahl der Entscheidungen, die in einem Arbeitsschritt gleichzeitig zu treffen sind. Die Zerlegung einer Strecke ist damit der Eingriff mit dem größten Ertrag und zugleich der einzige, der ohne zusätzliche Kosten wirkt.

Der vierte Befund ergibt sich aus der Trennung von eigener Schrittleistung und Fehlerfortpflanzung. Der Endschrift der verketteten Strecke verlor je nach Modell zwischen 31 und 51 Prozentpunkte, die ausnahmslos aus vorgelagerten Schritten stammten und ihm selbst nicht zuzurechnen waren. Ein Schritt, der für sich betrachtet 94,6 Prozent erreichte und damit unauffällig wirkte, löste drei Schritte später einen vollständigen Abbruch aus. Für die Bewertung folgt daraus, dass Evaluierungen einzelner ETL-Operationen die Leistungsfähigkeit in realen Strecken systematisch überschätzen, während rein verkettete Betrachtungen sie unterschätzen.

Der fünfte Befund korrigiert eine verbreitete Erwartung an die Prompt-Gestaltung, allerdings anders als zunächst angenommen. Die über alle Läufe gemittelte Rangfolge der Strategien erwies sich als nicht belastbar: Sie kehrte sich auf dem zweiten Datensatz um und ging, wie die Aufschlüsselung zeigte, nahezu vollständig auf das hardwarebegrenzte lokale Modell zurück, das unter den dreifach längeren Few-Shot-Prompts von 44,1 auf 24,0 Prozent einbrach. Unter den kommerziellen Modellen allein unterscheiden sich die Strategien über alle Aufgaben gemittelt kaum. Reproduzierbar ist stattdessen ein aufgabengebundenes Muster: Beispiele wirken bei Ähnlichkeitsent-

scheidungen, schrittweise Herleitung bei mehrstufigen Auswahlregeln, und bei sieben der neun Aufgaben wirkt keine der Strategien messbar. Zusätzlicher Kontext wirkt damit nicht durch seinen Umfang, sondern allein durch seine Passung zur Aufgabe – und schadet, wo er über sie hinausweist.

Der sechste Befund ordnet diesem Ergebnis seinen eigentlichen Rang zu. Werden die neun Schritte zu drei Gruppen zusammengefasst und erhält jede Gruppe eine fehlerfreie Eingabe, so verschwindet der Strategieunterschied vollständig: Alle 27 Läufe lieferten ein verwertbares Ergebnis, die Genauigkeit lag zwischen 98,4 und 100 Prozent, die Spanne zwischen den Strategien bei einem halben Prozentpunkt. Was in der Einzelaufgaben-Matrix als Wirkung der Prompt-Gestaltung erscheint, ist demnach zu einem erheblichen Teil eine Wirkung des Aufgabenzuschnitts. Für die Praxis kehrt das die übliche Reihenfolge der Gestaltungsentscheidungen um: Zuerst ist die Aufgabe zuzuschneiden, erst danach der Prompt – und ist der Zuschnitt richtig gewählt, verliert die Wahl der Strategie ihre Bedeutung weitgehend.

Der siebte Befund schließlich betrifft die Verlässlichkeit der Messung selbst. Im Bestätigungslauf schwankte der Endschrift derselben Konfiguration über fünf identische Wiederholungen zwischen 27,3 und 72,7 Prozent, während er im isolierten Durchlauf in jeder Wiederholung exakt denselben Wert lieferte. Die beobachtete Instabilität entstand folglich nicht aus der probabilistischen Textgenerierung selbst, sondern aus deren Verstärkung entlang der Kette – und ist damit keine Eigenschaft des Modells, sondern eine Eigenschaft der Architektur, in die es eingebettet ist.

10.2 Beantwortung der Forschungsfragen

FF1. Die erste Forschungsfrage richtete sich auf die Eignung aktueller LLMs für die Bearbeitung typischer ETL-Teilaufgaben im Vergleich zu einer klassischen, regelbasierten Pipeline sowie auf die Bedingungen, unter denen je nach Aufgabentyp ein klassischer, KI-gestützter oder hybrider Ansatz zu empfehlen ist.

Auf den ersten Teil der Frage lautet die Antwort, dass die Eignung nicht pauschal, sondern ausschließlich je Aufgabentyp und Ausführungsmodus zu beurteilen ist. Für die sechs formal beschreibbaren Aufgaben erreichen die kommerziellen Modelle mit gut gewähltem Prompt das Niveau der regelbasierten Pipeline, ohne dieses zu übertreffen; ihr Beitrag liegt dort nicht im Ergebnis, sondern darin, dass die Verarbeitungsvorschrift nicht zuvor formuliert werden musste. Für die semantisch geprägten Aufgaben

übertreffen sie die regelbasierte Lösung, allerdings nur im Direkt-Modus und nur um wenige Prozentpunkte. Für die zusammengesetzte Aufgabe versagen alle untersuchten Ansätze einschließlich der regelbasierten Pipeline, solange sie ungeteilt gestellt wird. Die in H1 formulierte Erwartung ist damit in ihrem Kern bestätigt – die starke Abhängigkeit vom Aufgabentyp, die Ebenbürtigkeit bei formalisierbaren Aufgaben, der Zuverlässigkeitsverlust bei zunehmender struktureller Komplexität und die Abwesenheit eines durchgängig überlegenen Ansatzes –, in einem Teilaspekt jedoch einzuschränken: Der erwartete Vorteil bei semantischen Aufgaben tritt nicht generell ein, sondern ist an den Ausführungsmodus gebunden und bleibt im primär untersuchten Modus der Code-Generierung aus.

Auf den zweiten Teil der Frage folgt die Empfehlung eines hybriden Vorgehens, das in Abschnitt 9.3 ausgeführt ist: regelbasierte Ausführung für die syntaktisch beschreibbaren Schritte, ein Modell im Direkt-Modus für die wenigen semantisch geprägten Schritte, regelbasierte oder codegenerierte Ausführung für die abschließende Aggregation, und dies durchgehend in kleinteilig zugeschnittenen Schritten mit einer inhaltlichen Prüfung nach jedem einzelnen. Für die untersuchte Strecke bedeutet dies, dass sieben der neun Schritte regelbasiert auszuführen wären und lediglich zwei ein Modell benötigen. Ein durchgängiger LLM-Einsatz ist auf Grundlage dieser Ergebnisse nicht zu empfehlen, ein durchgängiger Verzicht ebenso wenig.

FF2. Die zweite Forschungsfrage richtete sich auf den Einfluss der Prompting-Strategie auf die Ergebnisqualität und auf die daraus abzuleitende Empfehlung.

Der Einfluss ist erheblich, aber ungleich verteilt und stark vom Aufgabenzuschnitt abhängig. Bei sieben der neun Aufgaben war zwischen den Strategien kein Unterschied messbar; bei den beiden anspruchsvollen Entdoppelungsaufgaben entschied die Wahl über bis zu 55 Prozentpunkte, und die schema-angereicherte Variante senkte die Genauigkeit dort auf ein Viertel. Wo die Prompt-Gestaltung überhaupt wirkt, ist ihr Gewicht demnach mit dem der Modellwahl vergleichbar.

H2 ist in ihrer allgemeinen Form zu verwerfen, allerdings mit einer Begründung, die von der zunächst naheliegenden abweicht. Aufwändigere Strategien verbessern die Ergebnisqualität gegenüber dem Zero-Shot-Prompt nicht durchgängig – aber sie verschlechtern sie unter den kommerziellen Modellen auch nicht durchgängig. Die über alle Aufgaben gemittelte Rangfolge erwies sich schlicht als nicht belastbar: Sie kehrte

sich zwischen den beiden Datensätzen um und ging fast vollständig auf das hardwarebegrenzte lokale Modell zurück. Zutreffend bleibt der erwartete abnehmende Grenznutzen in verschärfter Form – bei unpassendem Zuschnitt ist er negativ –, ebenso die erwartete Aufgabenabhängigkeit, die sich über beide Datensätze hinweg reproduzierte.

Die Empfehlung, die daraus folgt, betrifft weniger die Wahl der Strategie als die Reihenfolge der Entscheidungen. Wird dieselbe Verarbeitung in angemessen zugeschnittenen Gruppen statt als überladene Einzelaufgabe gestellt, liegen alle drei Strategien zwischen 99,0 und 99,5 Prozent, und die Frage nach der besten Strategie erübrigt sich. Zuerst ist daher die Aufgabe zu zerlegen; bleibt danach eine Aufgabe übrig, die sich nicht weiter teilen lässt, gelten die beiden reproduzierten Zuordnungen: Beispiele bei Ähnlichkeitsentscheidungen, schrittweise Herleitung bei mehrstufigen Auswahlregeln, im Übrigen der einfachste und billigste Prompt. Zusätzlicher Kontext ist ausschließlich als Reaktion auf einen konkret beobachteten Mangel zu ergänzen – so behob die Angabe der Eingabe-Datentypen ein stilles Versagen der Spaltenauswahl –, niemals vorsorglich.

Reproduzierbarkeit. Die querschnittlich erhobene Reproduzierbarkeitsdimension fügt beiden Antworten eine Einschränkung hinzu. Die niedrig gewählte Temperatur begrenzte die Variabilität wirksam, beseitigte sie aber nicht: In gut einem Viertel der Konfigurationen führte derselbe Prompt bei demselben Modell zu messbar unterschiedlichen Ergebnissen, und die Instabilität konzentrierte sich auf genau jene Aufgaben, die auch inhaltlich am schwersten fielen. Gerade dort, wo ein Ergebnis besonders sorgfältig zu prüfen wäre, ist es zugleich am wenigsten verlässlich wiederholbar. Da die Verstärkung entlang der Kette sich als der dominierende Beitrag zu dieser Streuung erwies, ist sie durch Festschreibung des geprüften Codes und der frühen Zwischenergebnisse jedoch weitgehend beherrschbar.

10.3 Limitationen der Arbeit

Die vorstehenden Aussagen unterliegen mehreren Einschränkungen, die bei ihrer Übertragung zu berücksichtigen sind.

Die erste und gewichtigste betrifft den Datensatz. Er ist synthetisch erzeugt, und seine Qualitätsprobleme entstammen einer bekannten, endlichen Menge von Mustern.

Diese Anlage war notwendig, um eine eindeutige Soll-Lösung zu besitzen und die Ansätze überhaupt vergleichbar messen zu können, begünstigt jedoch systematisch die regelbasierte Vergleichspipeline: Deren Zuordnungstabelle und Normalisierungsregel wurden gegen genau diese Muster entworfen und erreichen deshalb Werte, die in einem realen Bestand mit unbekannten Abweichungsmustern nicht zu erwarten wären. Der in dieser Arbeit gemessene Vorsprung der regelbasierten Lösung ist damit als Obergrenze zu lesen, und das Verhältnis verschiebt sich in realen Beständen zugunsten der semantischen Verfahren – um welchen Betrag, lässt sich aus diesen Daten nicht bestimmen.

Die zweite Einschränkung betrifft die Größenordnung. Der Datensatz umfasst einige hundert Kunden und einige tausend Bestellungen, was für die inhaltliche Bewertung ausreicht, aber keine Aussage über das Verhalten bei produktionstypischen Datenmengen erlaubt. Für den Code-Generierungs-Modus ist dies unkritisch, da die Datenmenge dort nicht durch das Modell hindurchgeht; für den Direkt-Modus hingegen ist es die bestimmende Grenze, die im Rahmen dieser Arbeit bereits bei wenigen hundert Datensätzen eine Halbierung des Datensatzes erzwang.

Eng damit verbunden ist eine Einschränkung zum lokal betriebenen Modell. Sein Rückstand ist maßgeblich durch den verfügbaren Grafikspeicher bestimmt, der die Größe des ausführbaren Modells begrenzt; gemessen wurde insofern nicht allein Modellqualität, sondern auch die Ausstattung des ausführenden Rechners. Aus diesem Grund wurde es aus den Vergleichen der Prompting-Strategien, der Pipeline und des Bestätigungslaufs herausgenommen – eine Entscheidung, die die Vergleichbarkeit dieser Ebenen sichert, zugleich aber bedeutet, dass für lokal betreibbare Modelle nur die Ergebnisse der Faktormatrix vorliegen. Ob ein größeres, auf leistungsfähigerer Hardware ausführbares Modell den Abstand schließt, bleibt hier offen; die Fehlerverteilung – weit überwiegend nicht lauffähiger Code statt inhaltlicher Fehler – legt es nahe, belegt es aber nicht.

Die dritte Einschränkung betrifft die Momentaufnahme. Untersucht wurden vier Modelle zu einem bestimmten Zeitpunkt in einem Feld, das sich in Monatsabständen verändert. Die exakte Versionskennung wurde bei jeder Ausführung protokolliert, sodass die Läufe dokumentiert bleiben; da die kommerziellen Modelle jedoch serverseitig gepflegt werden, lässt sich ihr Verhalten weder einfrieren noch nach einer Modellaktualisierung wiederherstellen. Die konkreten Zahlenwerte sind daher nicht als dauerhaft gültig zu betrachten – die Wirkungszusammenhänge, aus denen sie entstehen, hingegen sehr

wohl, da sie in ihren Ursachen benannt und im Bestätigungslauf repliziert wurden.

Die vierte Einschränkung betrifft die Zahl der Wiederholungen. Für die Faktormatrix wurden drei Wiederholungen je Konfiguration ausgeführt, für die verkettete Pipeline fünf. Angesichts der beobachteten Streuung – allein der Endschritt schwankte über fünf Wiederholungen um bis zu 45 Prozentpunkte – ist dies eine schmale Grundlage für Aussagen über einzelne Konfigurationen. Die Befunde dieser Arbeit sind deshalb bewusst auf Ebene der Muster formuliert, die über Modelle und Aufgaben hinweg wiederkehren, und nicht auf Ebene einzelner Zahlenwerte.

Die fünfte Einschränkung betrifft den Zuschnitt der Untersuchung. Betrachtet wurde die Transformationsphase des ETL-Prozesses; Extraktion und Ladevorgang blieben ebenso außen vor wie Aspekte des produktiven Betriebs, etwa Fehlerbehandlung, Überwachung, inkrementelle Verarbeitung oder Datenschutzanforderungen beim Übertragen von Daten an externe Anbieter. Gerade der letzte Punkt ist für die praktische Anwendbarkeit erheblich und wurde hier nur insoweit berührt, als mit dem lokal betriebenen Modell eine Option ohne Datenabfluss mituntersucht wurde.

Schließlich ist die Rolle der prüfenden Instanz zu benennen. Die Untersuchung hat die Modelle weitgehend autonom arbeiten lassen und ihre Ergebnisse gegen eine bekannte Soll-Lösung bewertet. Der in der Praxis dominierende Einsatz als Assistenz einer Fachkraft, die den erzeugten Code liest, korrigiert und verantwortet, wurde damit nicht abgebildet. Die in dieser Arbeit gemessenen Werte beschreiben folglich, was ohne menschliche Nachbearbeitung entsteht – was zugleich die Untergrenze dessen markiert, was im assistierten Einsatz zu erwarten ist.

10.4 Ausblick auf weitere Forschung

Aus den Ergebnissen und ihren Grenzen ergeben sich mehrere Anschlussfragen.

Am unmittelbarsten anschlussfähig ist die Übertragung auf reale Datenbestände. Da der Vorsprung der regelbasierten Pipeline maßgeblich daraus resultiert, dass ihre Regeln gegen bekannte Abweichungsmuster entworfen wurden, wäre eine Wiederholung des Vergleichs auf einem gewachsenen Bestand mit unbekannten Mustern der aussagekräftigste nächste Schritt. Zu erwarten ist, dass sich das Verhältnis zugunsten der semantischen Verfahren verschiebt; die Größe dieser Verschiebung ist die eigentlich interessante Messgröße und zugleich diejenige, die über die wirtschaftliche Tragfähig-

keit eines LLM-Einsatzes entscheidet.

Ein zweiter Ansatzpunkt ist die in Abschnitt 9.3 skizzierte hybride Architektur, die aus den Ergebnissen abgeleitet, aber nicht mehr als Ganzes implementiert und gemessen wurde. Eine Umsetzung, die regelbasierte Schritte, gezielt eingesetzte Direkt-Modus-Schritte und eine Prüfung nach jedem Schritt tatsächlich verbindet, ließe sich unmittelbar gegen die hier gemessenen rein regelbasierten und rein KI-gestützten Varianten stellen und würde zeigen, ob die erwartete Kombination der Stärken auch in der Ausführung eintritt.

Der dritte Ansatzpunkt betrifft die Fehlerfortpflanzung, deren Messung diese Arbeit auf den Vergleich zweier Durchläufe stützt. Offen bleibt, welche Eigenschaften eines Zwischenergebnisses seine Verwendbarkeit in nachgelagerten Schritten bestimmen. Die Ergebnisse legen nahe, dass nicht der Umfang, sondern die Art der verbliebenen Abweichung entscheidet – 145 unveränderte Zeitstempel unter 540 Datumsangaben brachten die Kette zum Abbruch, während gleichmäßig verteilte kleinere Ungenauigkeiten folgenlos blieben. Eine Kennzahl, die diese Unterscheidung abbildet, wäre für die Überwachung KI-gestützter Strecken erheblich wertvoller als die übliche Genauigkeit je Schritt.

Ein vierter Ansatzpunkt ergibt sich aus dem Befund zum Ausführungsmodus. Da die direkte Verarbeitung bei semantischen Einzelentscheidungen überlegen, bei Aggregationen über viele Datensätze jedoch unbrauchbar ist, liegt eine Kombination nahe, in der das Modell die semantische Entscheidung je Wert trifft und deren Ergebnis anschließend deterministisch weiterverarbeitet wird – etwa als generierte Zuordnungstabelle statt als generierte Aufzählungslogik. Ob eine solche Zwischenform den Genauigkeitsvorteil des Direkt-Modus bei dessen Kostenstruktur vermeidet, ist eine unmittelbar prüfbare Frage.

Schließlich verdient der in Abschnitt 10.3 angesprochene assistierte Einsatz eigene Aufmerksamkeit. Diese Arbeit hat gemessen, was Modelle ohne menschliche Nachbearbeitung leisten; die praktisch relevantere Frage ist, wie viel Aufwand eine Fachkraft benötigt, um ein erzeugtes Ergebnis auf ein verantwortbares Niveau zu bringen, und ob dieser Aufwand geringer ausfällt als der einer eigenständigen Implementierung. Gerade der in dieser Arbeit dokumentierte Fall des stillen Versagens – eines Codes, der fehlerfrei durchläuft, schemakonform ausgibt und dennoch wirkungslos bleibt – legt nahe, dass diese Frage nicht allein über die Ergebnisqualität, sondern wesentlich über

den Prüfaufwand zu beantworten ist.

Literatur

- [1] Bilal Khan u. a. „An Overview of ETL Techniques, Tools, Processes and Evaluations in Data Warehousing“. In: *Journal on Big Data* 6 (2024), S. 1–20. ISSN: 25790048. DOI: 10.32604/jbd.2023.046223. URL: <https://www.proquest.com/docview/3200128360/abstract/ABA55C04CFAC48B2PQ/1> (besucht am 08. 05. 2026).
- [2] Afef Walha, Faiza Ghazzi und Faiez Gargouri. „Data Integration from Traditional to Big Data: Main Features and Comparisons of ETL Approaches“. In: *The Journal of Supercomputing* 80.19 (1. Dez. 2024), S. 26687–26725. ISSN: 1573-0484. DOI: 10.1007/s11227-024-06413-1. URL: <https://doi.org/10.1007/s11227-024-06413-1> (besucht am 08. 05. 2026).
- [3] Sarbaree Mishra. „Automating the Data Integration and ETL Pipelines through Machine Learning to Handle Massive Datasets in the Enterprise“. In: *International Journal of Emerging Research in Engineering and Technology* 1.2 (30. Juni 2020), S. 69–78. DOI: 10.63282/3050-922X.IJERET-V1I2P109. URL: <https://ijeret.org/index.php/ijeret/article/view/231> (besucht am 08. 05. 2026).
- [4] Kushvanth Chowdary Nagabhyru. *Bridging Traditional ETL Pipelines with AI Enhanced Data Workflows: Foundations of Intelligent Automation in Data Engineering*. 24. Nov. 2022. DOI: 10.2139/ssrn.5505199. Social Science Research Network: 5505199. URL: <https://papers.ssrn.com/abstract=5505199> (besucht am 08. 05. 2026). Vorveröffentlichung.
- [5] Chandran Ravi. „ETL (Extract, Transform & Load) Automation“. In: *International Journal of Emerging Trends in Computer Science and Information Technology* 6.1 (12. Feb. 2025), S. 52–55. ISSN: 3050-9246. DOI: 10.63282/3050-9246.IJETCSIT-V6I1P106. URL: <https://www.ijetcsit.org/index.php/ijetcsit/article/view/37> (besucht am 08. 05. 2026).
- [6] Wayne Xin Zhao u. a. *A Survey of Large Language Models*. 18. März 2026. DOI: 10.48550/arXiv.2303.18223. arXiv: 2303.18223 [cs]. URL: <http://arxiv.org/abs/2303.18223> (besucht am 08. 05. 2026). Vorveröffentlichung.
- [7] Hossein Hassani und Emmanuel Sirmal Silva. „The Role of ChatGPT in Data Science: How AI-Assisted Conversational Interfaces Are Revolutionizing the Field“. In: *Big Data and Cognitive Computing* 7.2 (Juni 2023), S. 62. ISSN: 2504-2289. DOI: 10.3390/bdcc7020062. URL: <https://www.mdpi.com/2504-2289/7/2/62> (besucht am 08. 05. 2026).

-
- [8] Soumen Chakraborty. „Beyond ETL: How AI Agents Are Building Self-Healing Data Pipelines“. In: *Journal of Computer Science and Technology Studies* 7.3 (8. Mai 2025), S. 741–756. ISSN: 2709-104X. DOI: 10.32996/jcsts.2025.7.3.81. URL: <https://al-kindipublishers.org/index.php/jcsts/article/view/9427> (besucht am 08.05.2026).
- [9] Jahangir Khan. „Leveraging Artificial Intelligence to Automate ETL Pipelines: Evolving Legacy Data Systems into Intelligent Workflows“. In: (9. Juni 2025).
- [10] Chaimae Boulahia u. a. „The Multi-Criteria Evaluation of Research Efforts Based on ETL Software: From Business Intelligence Approach to Big Data and Semantic Approaches“. In: *Evolutionary Intelligence* 17.4 (1. Aug. 2024), S. 2099–2124. ISSN: 1864-5917. DOI: 10.1007/s12065-023-00899-z. URL: <https://doi.org/10.1007/s12065-023-00899-z> (besucht am 08.05.2026).
- [11] Sandeep Rangineni u. a. „A Review on Enhancing Data Quality for Optimal Data Analytics Performance“. In: *International Journal of Computer Sciences and Engineering* 11 (31. Okt. 2023), S. 51–58. DOI: 10.26438/ijcse/v11i10.5158.
- [12] Fakhitah Ridzuan und Wan Mohd Nazmee Wan Zainon. „A Review on Data Quality Dimensions for Big Data“. In: *Procedia Computer Science*. Seventh Information Systems International Conference (ISICO 2023) 234 (1. Jan. 2024), S. 341–348. ISSN: 1877-0509. DOI: 10.1016/j.procs.2024.03.008. URL: <https://www.sciencedirect.com/science/article/pii/S187705092400365X> (besucht am 08.05.2026).
- [13] Pierre-Olivier Côté u. a. „Data Cleaning and Machine Learning: A Systematic Literature Review“. In: *Automated Software Engineering* 31.2 (11. Juni 2024), S. 54. ISSN: 1573-7535. DOI: 10.1007/s10515-024-00453-w. URL: <https://doi.org/10.1007/s10515-024-00453-w> (besucht am 08.05.2026).
- [14] Raghavender Maddali. „Automating Data Quality Assurance Using Machine Learning in ETL Pipelines“. In: *IJLRP - International Journal of Leading Research Publication* 2.6 (o.J.). ISSN: 2582-8010. DOI: 10.5281/zenodo.15107533. URL: <https://www.ijlrp.com/research-paper.php?id=1455> (besucht am 08.05.2026).
- [15] Muhammad Fakhrol Safitra u. a. „Advancements in Artificial Intelligence and Data Science: Models, Applications, and Challenges“. In: *Procedia Computer Science*. Seventh Information Systems International Conference (ISICO 2023) 234 (1. Jan. 2024), S. 381–388. ISSN: 1877-0509. DOI: 10.1016/j.procs.2024.03.018. URL: <https://www.sciencedirect.com/science/article/pii/S1877050924003752> (besucht am 08.05.2026).

-
- [16] Jasmin Praful Bharadiya. „The Role of Machine Learning in Transforming Business Intelligence“. In: *International Journal of Computing and Artificial Intelligence* 4.1 (1. Jan. 2023), S. 16–24. ISSN: 27076571, 2707658X. DOI: 10.33545/27076571.2023.v4.i1a.60. URL: <https://www.computersciencejournals.com/ijcai/archives/2023.v4.i1.A.60> (besucht am 08.05.2026).
- [17] Alhassan Mumuni und Fuseini Mumuni. „Automated Data Processing and Feature Engineering for Deep Learning and Big Data Applications: A Survey“. In: *Journal of Information and Intelligence* 3.2 (1. März 2025), S. 113–153. ISSN: 2949-7159. DOI: 10.1016/j.jiixd.2024.01.002. URL: <https://www.sciencedirect.com/science/article/pii/S2949715924000027> (besucht am 08.05.2026).
- [18] Zijing Liang u. a. „A Survey of Multimodal Large Language Models“. In: *Proceedings of the 3rd International Conference on Computer, Artificial Intelligence and Control Engineering. CAICE '24*. New York, NY, USA: Association for Computing Machinery, 6. Aug. 2024, S. 405–409. ISBN: 979-8-4007-1694-2. DOI: 10.1145/3672758.3672824. URL: <https://dl.acm.org/doi/10.1145/3672758.3672824> (besucht am 08.05.2026).
- [19] Pengfei Liu u. a. „Pre-Train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing“. In: *ACM Comput. Surv.* 55.9 (16. Jan. 2023), 195:1–195:35. ISSN: 0360-0300. DOI: 10.1145/3560815. URL: <https://dl.acm.org/doi/10.1145/3560815> (besucht am 08.05.2026).
- [20] *The 2026 AI Index Report | Stanford HAI*. o.J. URL: <https://hai.stanford.edu/ai-index/2026-ai-index-report> (besucht am 08.05.2026).
- [21] *Comparison of AI Models across Intelligence, Performance, and Price*. o.J. URL: <https://artificialanalysis.ai/models> (besucht am 08.05.2026).
- [22] Pranab Sahoo u. a. „A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications“. In: (o.J.).
- [23] Sander Schulhoff u. a. *The Prompt Report: A Systematic Survey of Prompt Engineering Techniques*. 26. Feb. 2025. DOI: 10.48550/arXiv.2406.06608. arXiv: 2406.06608 [cs]. URL: <http://arxiv.org/abs/2406.06608> (besucht am 08.05.2026). Vorveröffentlichung.
- [24] Kaiyan Chang u. a. *Efficient Prompting Methods for Large Language Models: A Survey*. 2. Dez. 2024. DOI: 10.48550/arXiv.2404.01077. arXiv: 2404.01077 [cs]. URL: <http://arxiv.org/abs/2404.01077> (besucht am 08.05.2026). Vorveröffentlichung.

-
- [25] Jahangir Khan. *Cognitive ETL: Using AI to Interpret Complex Data Relationships*. 10. Nov. 2025. DOI: 10.2139/ssrn.5728984. Social Science Research Network: 5728984. URL: <https://papers.ssrn.com/abstract=5728984> (besucht am 08.05.2026). Vorveröffentlichung.
- [26] Naveen Reddy Singi Reddy und Mahitha Adapa. „AI-Driven Data Integration: Transforming Enterprise Data Pipelines through Machine Learning“. In: *Journal of Computer Science and Technology Studies* 7.12 (26. Nov. 2025), S. 110–119. ISSN: 2709-104X. DOI: 10.32996/jcsts.2025.7.12.16. URL: <https://al-kindipublishers.org/index.php/jcsts/article/view/11533> (besucht am 08.05.2026).
- [27] Lin Long u. a. „On LLMs-Driven Synthetic Data Generation, Curation, and Evaluation: A Survey“. In: *Findings of the Association for Computational Linguistics: ACL 2024*. Findings 2024. Hrsg. von Lun-Wei Ku, Andre Martins und Vivek Sri-kumar. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, S. 11065–11082. DOI: 10.18653/v1/2024.findings-acl.658. URL: <https://aclanthology.org/2024.findings-acl.658/> (besucht am 08.05.2026).
- [28] Hsin-Yu Chang u. a. *A Survey of Data Synthesis Approaches*. 4. Juli 2024. DOI: 10.48550/arXiv.2407.03672. arXiv: 2407.03672 [cs]. URL: <http://arxiv.org/abs/2407.03672> (besucht am 08.05.2026). Vorveröffentlichung.
- [29] Zhuoyan Li u. a. „Synthetic Data Generation with Large Language Models for Text Classification: Potential and Limitations“. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. EMNLP 2023. Hrsg. von Houda Bouamor, Juan Pino und Kalika Bali. Singapore: Association for Computational Linguistics, Dez. 2023, S. 10443–10461. DOI: 10.18653/v1/2023.emnlp-main.647. URL: <https://aclanthology.org/2023.emnlp-main.647/> (besucht am 08.05.2026).
- [30] Joao Fonseca und Fernando Bacao. „Tabular and Latent Space Synthetic Data Generation: A Literature Review“. In: *Journal of Big Data* 10.1 (10. Juli 2023), S. 115. ISSN: 2196-1115. DOI: 10.1186/s40537-023-00792-7. URL: <https://doi.org/10.1186/s40537-023-00792-7> (besucht am 08.05.2026).
- [31] Mandeep Goyal und Qusay H. Mahmoud. „A Systematic Review of Synthetic Data Generation Techniques Using Generative AI“. In: *Electronics* 13.17 (Jan. 2024), S. 3509. ISSN: 2079-9292. DOI: 10.3390/electronics13173509. URL: <https://www.mdpi.com/2079-9292/13/17/3509> (besucht am 08.05.2026).

-
- [32] Janardhan Reddy Kasireddy. „The Role of AI in Modern Data Engineering: Automating ETL and Beyond“. In: *ICT for Global Innovations and Solutions*. Hrsg. von Saurav Bhattacharya. Cham: Springer Nature Switzerland, 2026, S. 667–693. ISBN: 978-3-032-02853-2. DOI: 10.1007/978-3-032-02853-2_47.
- [33] Raghav Maddali. „Autonomous AI Agents for Real-Time Data Transformation and ETL Automation“. In: *INTERNATIONAL JOURNAL OF RESEARCH AND ANALYTICAL REVIEWS* 10 (23. Juli 2023), S. 12. DOI: 10.5281/zenodo.15096256.
- [34] Jahangir Khan. *Intelligent Anomaly Detection in ETL Workflows Using AI*. 9. Nov. 2025. DOI: 10.2139/ssrn.5729002. Social Science Research Network: 5729002. URL: <https://papers.ssrn.com/abstract=5729002> (besucht am 08.05.2026). Vorveröffentlichung.
- [35] Chiara Van Der Putten. „Transforming Data Flow: Generative AI in ETL Pipeline Automatization“. laurea. Politecnico di Torino, 11. Apr. 2024. 83 S. URL: <https://webthesis.biblio.polito.it/30855/> (besucht am 08.05.2026).
- [36] Shashank A. „AI-Enhanced ETL Processes: Leveraging Artificial Intelligence for Optimized Data Integration Systems“. In: *Journal Of Multidisciplinary* 5.8 (10. Aug. 2025), S. 219–225. ISSN: 2945-3445. URL: <https://sarcouncil.com/2025/8/ai-enhanced-etl-processes-leveraging-artificial-intelligence-for-optimized-data-integration-systems> (besucht am 08.05.2026).
- [37] Dhamotharan Seenivasan. *AI Driven Enhancement of ETL Workflows for Scalable and Efficient Cloud Data Engineering*. 28. Juni 2024. Social Science Research Network: 5153853. URL: <https://papers.ssrn.com/abstract=5153853> (besucht am 08.05.2026). Vorveröffentlichung.
- [38] Indu Joshi u. a. „Synthetic Data in Human Analysis: A Survey“. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46.7 (Juli 2024), S. 4957–4976. ISSN: 1939-3539. DOI: 10.1109/TPAMI.2024.3362821. URL: <https://ieeexplore.ieee.org/abstract/document/10423161> (besucht am 08.05.2026).
- [39] Jing Lin und Kee Yuan Ngiam. „How Data Science and AI-based Technologies Impact Genomics“. In: *Singapore Medical Journal* 64.1 (Jan. 2023), S. 59. ISSN: 0037-5675. DOI: 10.4103/singaporemedj.SMJ-2021-438. URL: https://journals.lww.com/smj/fulltext/2023/01000/How_data_science_and_AI_based_technologies_impact.10.aspx?context=LatestArticles (besucht am 08.05.2026).